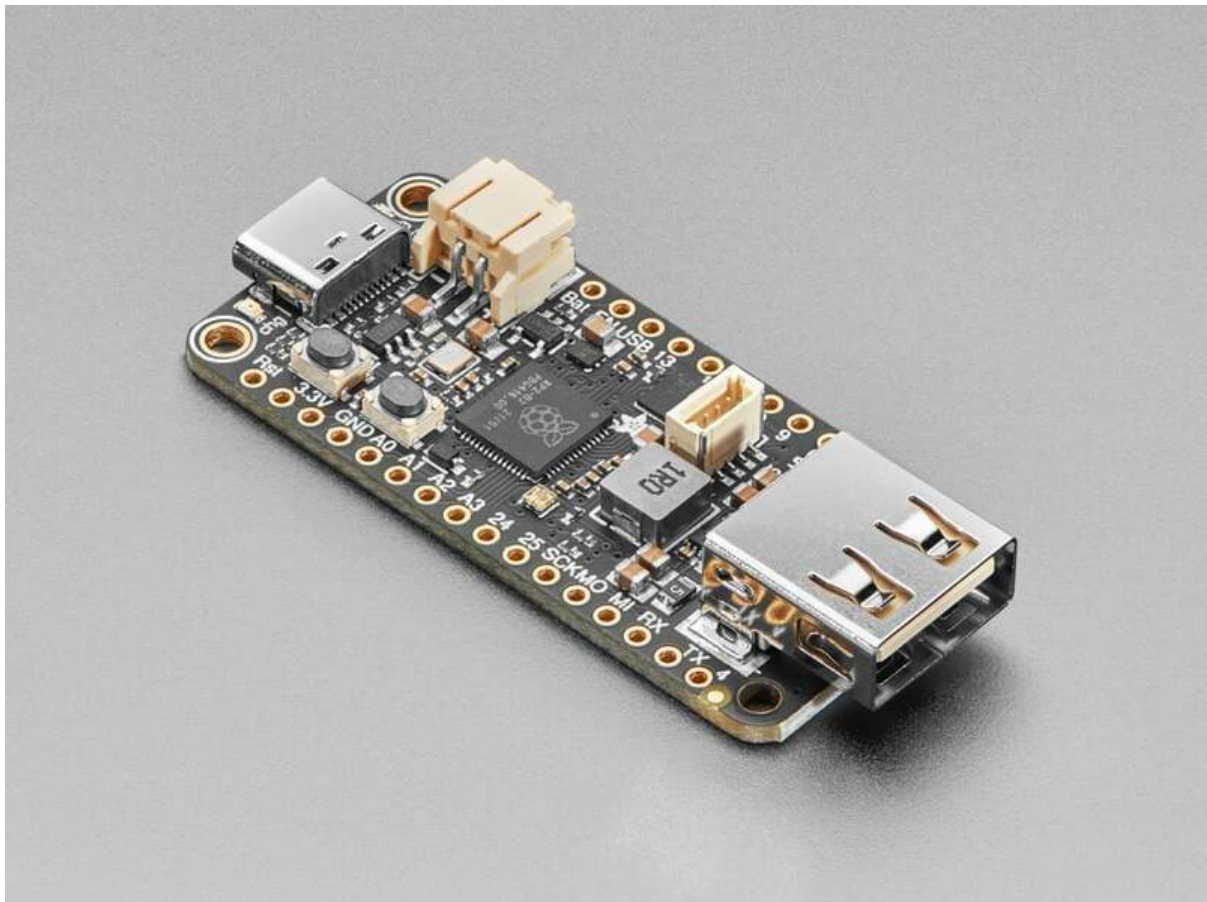




Adafruit Feather RP2040 with USB Type A Host

Created by Liz Clark



<https://learn.adafruit.com/adafruit-feather-rp2040-with-usb-type-a-host>

Last updated on 2025-02-24 01:42:11 PM EST

Table of Contents

Overview	7
Pinouts	11
• Power Pins, Connections, and Charge LED • Logic Pins • GPIO Pins by Pin Functionality • USB Type A Input and 5V LED • Microcontroller and Flash • Buttons and RST Pin • NeoPixel and Red LED • STEMMA QT	
Power Management	19
• Battery + USB Power • Power Supplies • Measuring Battery • ENable pin • Alternative Power Options	
CircuitPython	23
• CircuitPython Quickstart • Safe Mode • Flash Resetting UF2	
Installing the Mu Editor	26
• Download and Install Mu • Starting Up Mu • Using Mu	
The CIRCUITPY Drive	28
• Boards Without CIRCUITPY	
Creating and Editing Code	29
• Creating Code • Editing Code • Back to Editing Code... • Naming Your Program File	
Exploring Your First CircuitPython Program	34
• Imports & Libraries • Setting Up The LED • Loop-de-loops • What Happens When My Code Finishes Running? • What if I Don't Have the Loop?	
Connecting to the Serial Console	37
• Are you using Mu? • Serial Console Issues or Delays on Linux • Setting Permissions on Linux • Using Something Else?	
Interacting with the Serial Console	40

The REPL	43
• Entering the REPL • Interacting with the REPL • Returning to the Serial Console	
CircuitPython Libraries	47
• The Adafruit Learn Guide Project Bundle • The Adafruit CircuitPython Library Bundle • Downloading the Adafruit CircuitPython Library Bundle • The CircuitPython Community Library Bundle • Downloading the CircuitPython Community Library Bundle • Understanding the Bundle • Example Files • Copying Libraries to Your Board • Understanding Which Libraries to Install • Example: ImportError Due to Missing Library • Library Install on Non-Express Boards • Updating CircuitPython Libraries and Examples • CircUp CLI Tool	
CircuitPython Documentation	58
• CircuitPython Core Documentation • CircuitPython Library Documentation	
Recommended Editors	64
• Recommended editors • Recommended only with particular settings or add-ons • Editors that are NOT recommended	
Advanced Serial Console on Windows	66
• Windows 7 and 8.1 • What's the COM? • Install Putty	
Advanced Serial Console on Mac	70
• What's the Port? • Connect with screen	
Advanced Serial Console on Linux	72
• What's the Port? • Connect with screen • Permissions on Linux	
Troubleshooting	76
• Always Run the Latest Version of CircuitPython and Libraries • I have to continue using CircuitPython 7.x or earlier. Where can I find compatible libraries? • macOS Sonoma before 14.4: Errors Writing to CIRCUITPYmacOS 14.4 - 15.1: Slow Writes to CIRCUITPY • Bootloader (boardnameBOOT) Drive Not Present • Windows Explorer Locks Up When Accessing boardnameBOOT Drive • Copying UF2 to boardnameBOOT Drive Hangs at 0% Copied • CIRCUITPY Drive Does Not Appear or Disappears Quickly • Device Errors or Problems on Windows • Serial Console in Mu Not Displaying Anything • code.py Restarts Constantly • CircuitPython RGB Status Light • CircuitPython 7.0.0 and Later	

- [CircuitPython 6.3.0 and earlier](#)
- [Serial console showing ValueError: Incompatible .mpy file](#)
- [CIRCUITPY Drive Issues](#)
- [Safe Mode](#)
- [To erase CIRCUITPY: storage.erase_filesystem\(\)](#)
- [Erase CIRCUITPY Without Access to the REPL](#)
- [For the specific boards listed below:](#)
- [For SAMD21 non-Express boards that have a UF2 bootloader:](#)
- [For SAMD21 non-Express boards that do not have a UF2 bootloader:](#)
- [Running Out of File Space on SAMD21 Non-Express Boards](#)
- [Delete something!](#)
- [Use tabs](#)
- [On macOS?](#)
- [Prevent & Remove macOS Hidden Files](#)
- [Copy Files on macOS Without Creating Hidden Files](#)
- [Other macOS Space-Saving Tips](#)
- [Device Locked Up or Boot Looping](#)

Frequently Asked Questions 94

- [Using Older Versions](#)
- [Python Arithmetic](#)
- [Wireless Connectivity](#)
- [Asyncio and Interrupts](#)
- [Status RGB LED](#)
- [Memory Issues](#)
- [Unsupported Hardware](#)

Welcome to the Community! 100

- [Adafruit Discord](#)
- [CircuitPython.org](#)
- [Adafruit GitHub](#)
- [Adafruit Forums](#)
- [Read the Docs](#)

CircuitPython Essentials 109

Blink 110

- [LED Location](#)
- [Blinking an LED](#)

Digital Input 112

- [LED and Button](#)
- [Controlling the LED with a Button](#)

NeoPixel 114

- [NeoPixel Location](#)
- [NeoPixel Color and Brightness](#)
- [RGB LED Colors](#)
- [NeoPixel Rainbow](#)

I2C 120

- [I2C and CircuitPython](#)
- [Necessary Hardware](#)
- [Wiring the MCP9808](#)
- [Find Your Sensor](#)
- [I2C Sensor Data](#)

- [Where's my I2C?](#)

[USB Host Device Info](#) 127

- [CircuitPython Usage](#)
- [Example Code](#)

[USB Host Read Data](#) 129

- [Example Code](#)

[Arduino IDE Setup](#) 133

- [Arduino IDE Download](#)
- [Adding the Philhower Board Manager URL](#)
- [Add Board Support Package](#)
- [Choose Your Board](#)

[Arduino Usage](#) 136

- [RP2040 Arduino Pins](#)
- [Choose Your Board](#)
- [Load the Blink Sketch](#)
- [Manually Enter the Bootloader](#)

[Blink](#) 137

- [Pre-Flight Check: Get Arduino IDE & Hardware Set Up](#)
- [Start up Arduino IDE and Select Board/Port](#)
- [New Blink Sketch](#)
- [Verify \(Compile\) Sketch](#)
- [Upload Sketch](#)
- [Native USB and manual bootloading](#)
- [Enter Manual Bootload Mode](#)
- [Finally, a Blink!](#)

[I2C Scan Test](#) 145

- [Common I2C Connectivity Issues](#)
- [Perform an I2C scan!](#)
- [Wiring the MCP9808](#)

[USB Host Device Info](#) 150

- [Install Adafruit_TinyUSB Library](#)
- [Install Pico PIO USB Library](#)
- [Board Upload Settings](#)
- [Device Info Example](#)

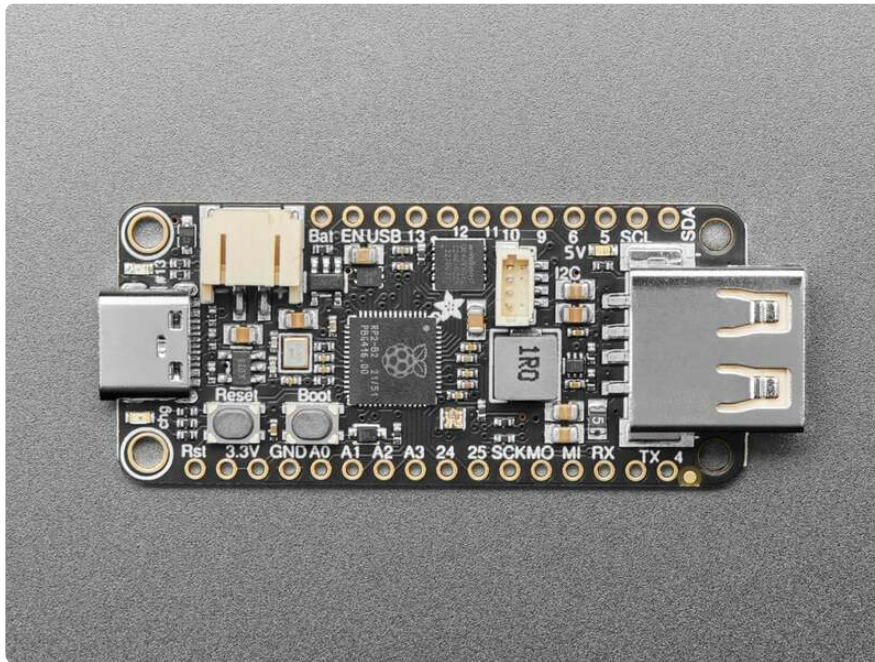
[Factory Reset](#) 159

- [Step 1. Download the factory-reset.uf2 file](#)
- [Step 2. Enter RP2040 bootloader mode](#)
- [Step 3. Drag UF2 file to RPI-RP2](#)
- [Flash Resetting UF2](#)

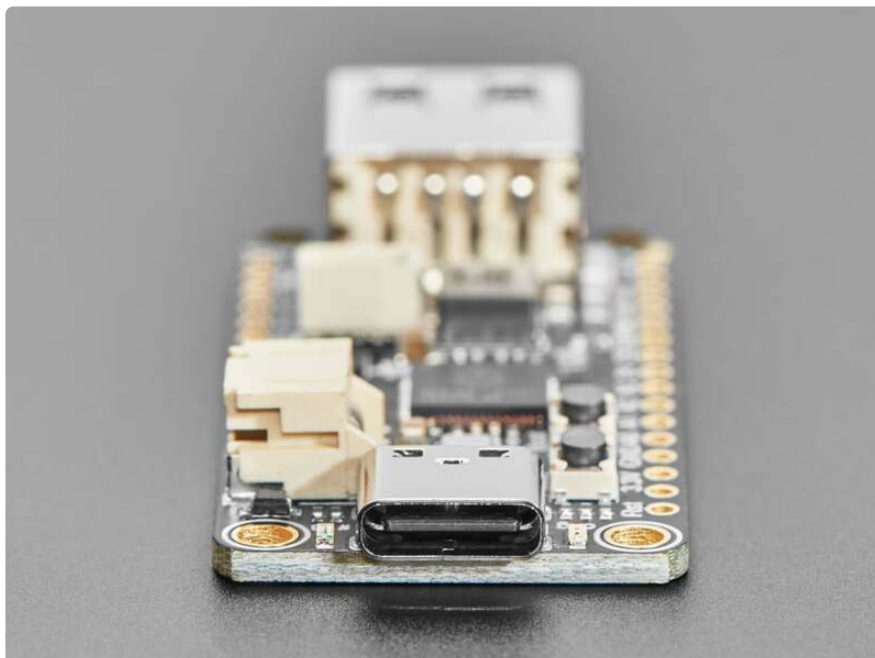
[Downloads](#) 162

- [Files](#)
- [Schematic and Fab Print](#)

Overview

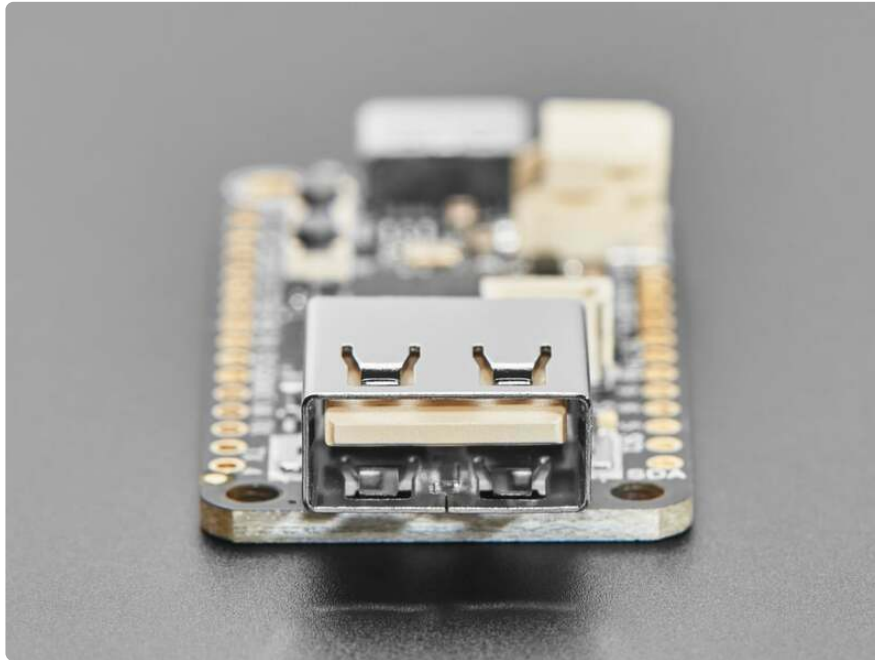


You're probably really used to microcontroller boards with USB, but what about a dev board with two? Two is more than one, so that makes it twice as good! And the **Adafruit Feather RP2040 with USB Host** is double-the-fun of our other Feather RP2040 boards, with a USB Type A port on the end for connecting USB devices to.



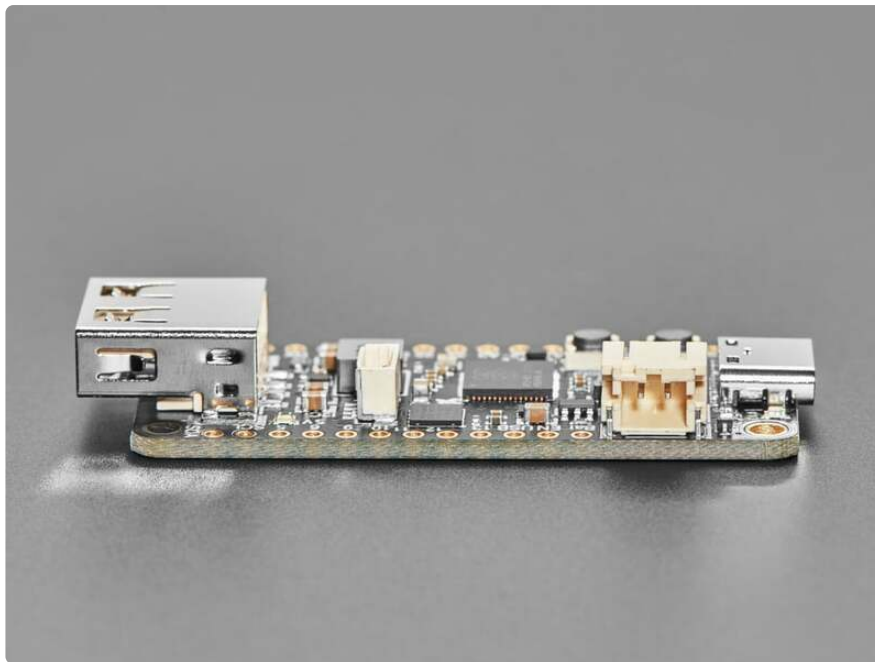
Now you might be thinking "hey waitaminute, the RP2040 doesn't have two USB port peripherals???" and you'd be correct! But what it does have is a nifty PIO peripheral that can be (ab)used to emulate a USB host peripheral. You get to keep the main USB port for uploading, debugging, and data communication, while at the same time

sending and receiving data to just-about-any USB device. [This work is originally by sekigon on GitHub \(https://adafru.it/18Dn\)](https://adafru.it/18Dn), and if you're using the Pico C/C++ SDK, that's still the recommended library to use.

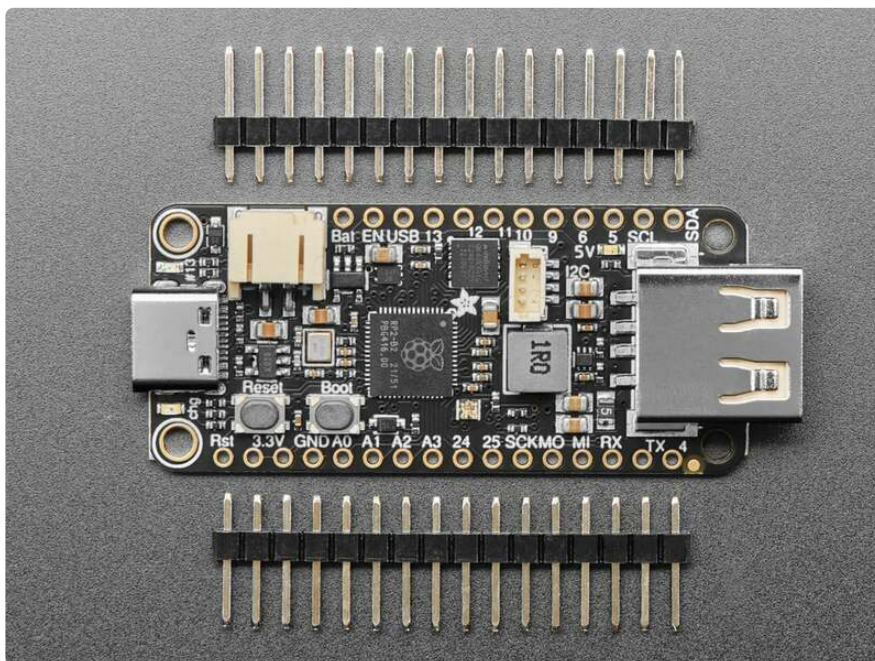


Check out the [TinyUSB 'dual role' examples \(https://adafru.it/18Do\)](https://adafru.it/18Do) for some things you can do! For example, [datalogging to a USB Key \(https://adafru.it/18Dp\)](https://adafru.it/18Dp). Or [reading from another device/microcontroller that has USB CDC serial interface \(https://adafru.it/18Dq\)](https://adafru.it/18Dq). Or [creating an HID re-mapper \(https://adafru.it/18Dr\)](https://adafru.it/18Dr). Or [connecting to weird devices that require firmware-updates like the Cypress EZ-USB based Intellikeys \(https://adafru.it/18Ds\)](https://adafru.it/18Ds) communications board.

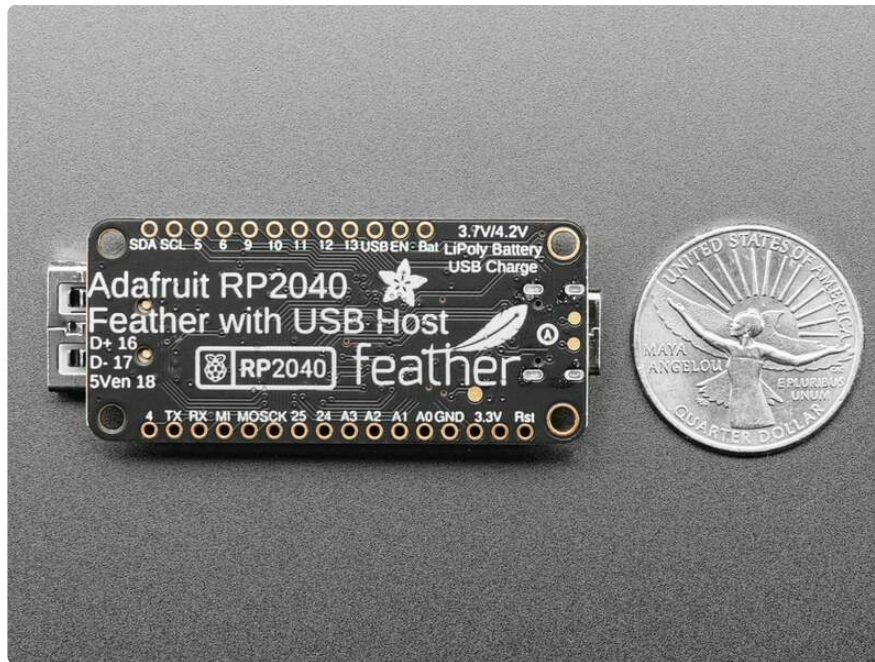
Note that this is definitely a firmware hack: you will need to dedicate the second ARM core and both PIO peripherals to just handling the USB messages, but we find that it does work fairly well, or at least as well as most microcontroller's USB Host peripherals!



We also include a 1 Amp boost converter based on the TPS61023 (<http://adafruit.it/4654>) so you can run from Lipo battery and get a nice clean 5V output for the USB devices. The booster has the enable pin tied to one of the extra GPIO on the RP2040 so power can be manually turned on and off to hard-reset whatever is connected.



At the Feather's heart is an RP2040 chip, clocked at 133 MHz and at 3.3V logic, the same one used in the [Raspberry Pi Pico](http://adafruit.it/4864) (<http://adafruit.it/4864>). This chip has 264K of RAM and is paired with a whopping 8MB of external QSPI FLASH! There's even room left over for a STEMMA QT connector for plug-and-play of I2C devices!



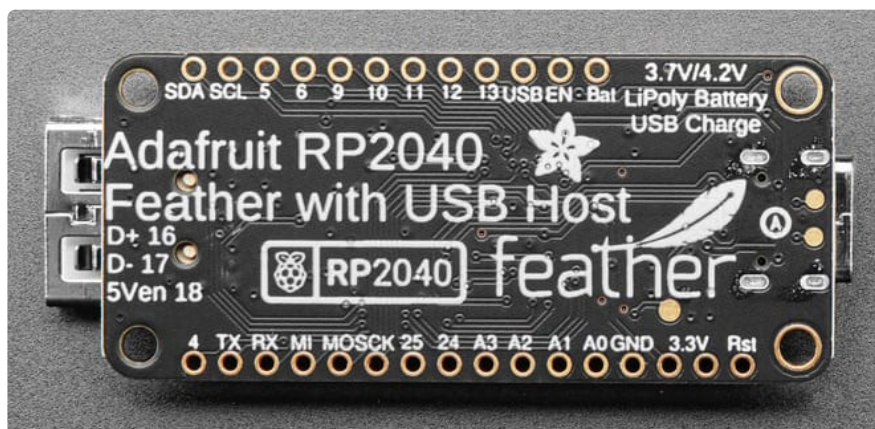
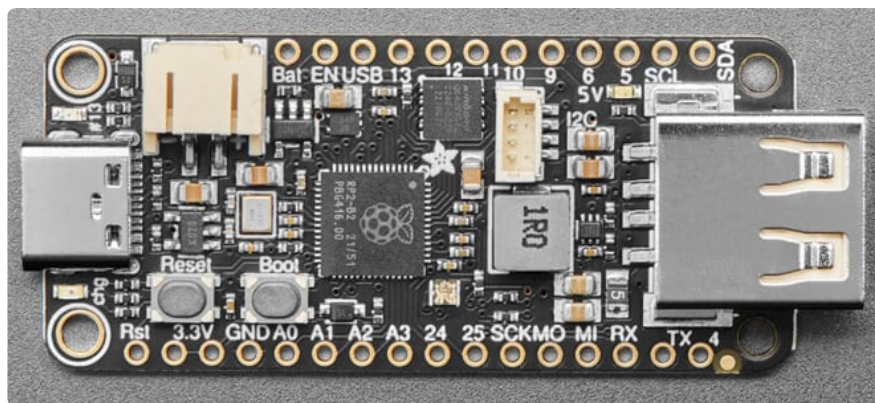
To make it easy to use for portable projects, we added a connector for any of our 3.7V Lithium polymer batteries and built-in battery charging. You don't need a battery, it will run just fine straight from the USB Type C connector. But, if you do have a battery, you can take it on the go, then plug in the USB to recharge. The Feather will automatically switch over to USB power when it's available.

Here're some handy specs! You get:

- Measures 2.0" x 0.9" x 0.28" (50.8mm x 22.8mm x 7mm) without headers soldered in
- Light as a (large?) feather - 6.3 grams
- RP2040 32-bit Cortex M0+ dual core running at ~133 MHz @ 3.3V logic and power
- 264 KB RAM
- **8 MB SPI FLASH** chip for storing files and CircuitPython/MicroPython code storage.
- No EEPROM
- **Tons of GPIO! 21 x GPIO pins with following capabilities:**
 - **Four** 12-bit ADCs (one more than Pico)
 - Two I2C, Two SPI, and two UART peripherals, we label one for the 'main' interface in standard Feather locations
 - 16 x PWM outputs - for servos, LEDs, etc
- **Built-in 200mA+ lipoly charger** with charging status indicator LED
- **Pin #13 red LED** for general purpose blinking
- **RGB NeoPixel** for full-color indication.

- On-board **STEMMA QT connector** that lets you quickly connect any Qwiic, STEMMA QT or Grove I2C devices with no soldering!
- **Both Reset button and Bootloader select button** for quick restarts (no unplugging-replugging to relaunch code)
- **USB Type C connector** lets you access built-in ROM USB bootloader and serial port debugging
- **USB Type A connector** for USB host capability. D+ on GPIO 16, D- on GPIO 17
- **5V Boost converter**, up to 1 Amp peak output for USB peripheral power, with 500mA resettable fuse. Enable on GPIO 18.
- 3.3V Power/enable pin
- 4 mounting holes
- 12 MHz crystal for perfect timing.
- 3.3V regulator with 500mA peak current output

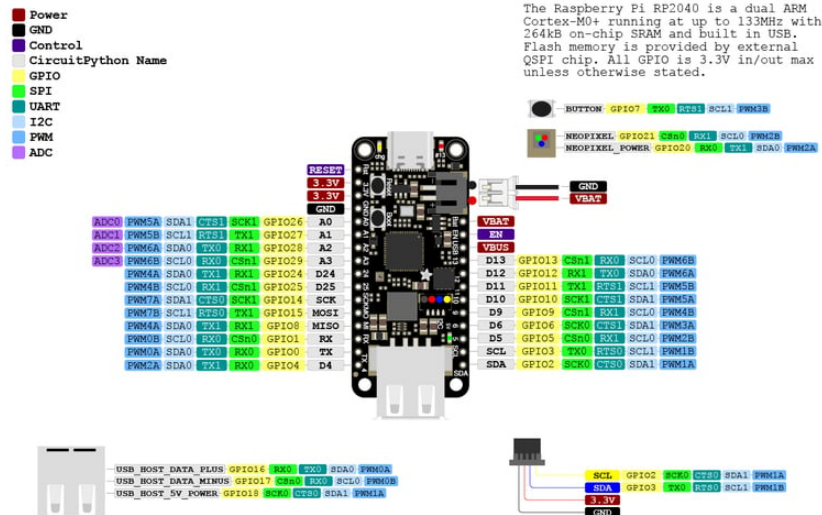
Pinouts



This Feather has a lot going on! This page details all of the pin-specific information and various capabilities.

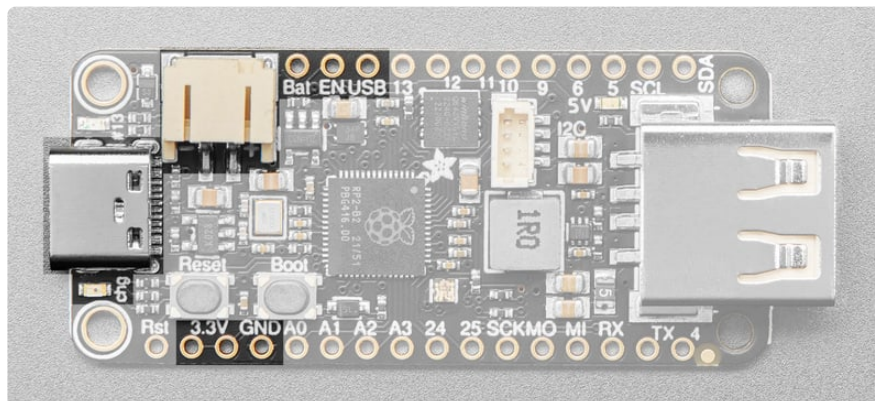
Adafruit Feather RP2040 USB Host

<https://www.adafruit.com/product/5723>



[PrettyPins PDF on GitHub. \(https://adafru.it/18Dt\)](https://adafru.it/18Dt)

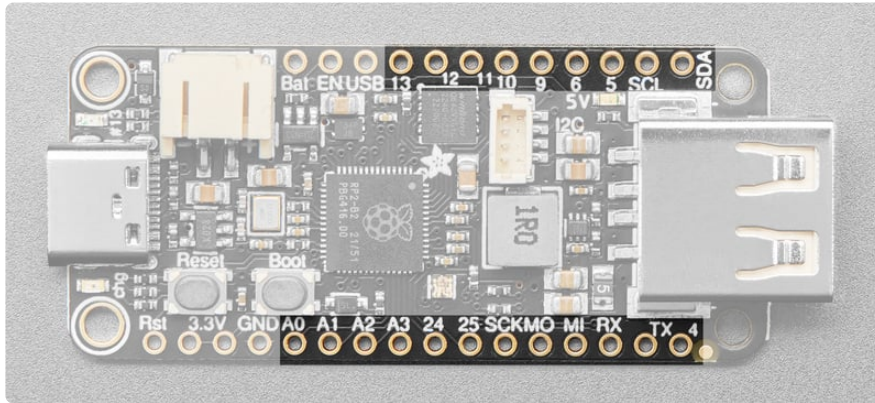
Power Pins, Connections, and Charge LED



- **USB C connector** - This is used for power and data. Connect to your computer via a USB C cable to update firmware and edit code.
- **LiPoly Battery connector** - This 2-pin JST PH connector allows you to plug in LiPoly batteries to power the Feather. The Feather is also capable of charging batteries plugged into this port via USB.
- **chg LED** - This small LED is located below the USB C connector. This indicates the charge status of a connected LiPoly battery when charging over USB. Note, it's normal for this LED to flicker when no battery is in place, that's the charge circuitry trying to detect whether a battery is there or not.
- **GND** - This is the common ground for all power and logic.
- **BAT** - This is the positive voltage to/from the 2-pin JST PH jack for the optional LiPoly battery.
- **USB** - This is the positive voltage to/from the USB C connector, if USB is connected.

- **EN** - This is the 3.3V regulator's enable pin. It's pulled up, so connect to ground to disable the 3.3V regulator.
- **3.3V** - These pins are the output from the 3.3V regulator, they can supply 500mA peak.

Logic Pins



I2C and SPI on RP2040

The RP2040 is capable of handling I2C, SPI and UART on many pins. However, there are really only two peripherals each of I2C, SPI and UART: I2C0 and I2C1, SPI0 and SPI1, and UART0 and UART1. So while many pins are capable of I2C, SPI and UART, you can only do two at a time, and only on separate peripherals, 0 and 1. I2C, SPI and UART peripherals are included and numbered below.

PWM on RP2040

The RP2040 supports PWM on all pins. However, it is not capable of PWM on all pins at the same time. There are 8 PWM "slices", each with two outputs, A and B. Each pin on the Feather is assigned a PWM slice and output. For example, A0 is PWM5 A, which means it is first output of the fifth slice. You can have up to 17 PWM objects on this Feather. The important thing to know is that **you cannot use the same slice and output more than once at the same time**. So, if you have a PWM object on pin A0, you cannot also put a PWM object on D10, because they are both PWM5 A. The PWM slices and outputs are indicated below. Note that PWM3 B is not available on the this Feather because the pin is not broken out.

Analog Pins

The RP2040 has four ADCs. These pins are the only pins capable of handling analog, and they can also do digital.

- **A0/GPIO26** - This pin is ADC0. It is also SPI1 SCK, I2C1 SDA and PWM5 A.
- **A1/GPIO27** - This pin is ADC1. It is also SPI1 MOSI, I2C1 SCL and PWM5 B.
- **A2/GPIO28** - This pin is ADC2. It is also SPI1 MISO, I2C1 SDA and PWM6 A.

- **A3/GPIO29** - This pin is ADC3. It is also SPI1 CS, I2C0 SCL and PWM6 B.

Digital Pins

These are the digital I/O pins. They all have multiple capabilities.

- **D24/GPIO24** - Digital I/O pin 24. It is also UART1 TX, I2C0 SDA, and PWM4 A.
- **D25/GPIO25** - Digital I/O pin 25. It is also UART1 RX, I2C0 SCL, and PWM4 B.
- **SCK/GPIO14** - The main SPI1 SCK. It is also I2C1 SDA, and PWM7 A.
- **MO/GPIO15** - The main SPI1 MOSI. It is also I2C1 SCL, and PWM7 B.
- **MI/GPIO8** - The main SPI1 MISO. It is also UART1 TX, I2C0 SDA, and PWM4 A.
- **RX/GPIO1** - The main UART0 RX pin. It is also I2C0 SDA, SPI0 CS and PWM0 B.
- **TX/GPIO0** - The main UART0 TX pin. It is also I2C0 SCL, SPI0 MISO and PWM0 A.
- **D13/GPIO13** - Digital I/O pin 13. It is also SPI1 CS, UART0 RX, I2C0 SCL and PWM6 B.
- **D12/GPIO12** - Digital I/O pin 12. It is also SPI1 MISO, UART0 TX, I2C0 SDA and PWM6 A.
- **D11/GPIO11** - Digital I/O pin 11. It is also SPI1 MOSI, I2C1 SCL and PWM5 B.
- **D10/GPIO10** - Digital I/O pin 10. It is also SPI1 SCK, I2C1 SDA and PWM5 A.
- **D9/GPIO9** - Digital I/O pin 9. It is also SPI1 CS, UART1 RX, I2C0 SCL and PWM4 B.
- **D6/GPIO6** - Digital I/O pin 6. It is also SPI0 SCK, I2C1 SDA, and PWM3 A.
- **D5/GPIO5** - Digital I/O pin 5. It is also SPI0 CS, UART1 RX, I2C0 SCL, and PWM2 B.
- **D4/GPIO4** - Digital I/O pin 4. It is also RX0, TX1, SDA0 and PWM2 A.
- **SCL/GPIO3** - The main I2C1 clock pin. It is also SPI0 MOSI, I2C1 SCL and PWM1 B.
- **SDA/GPIO2** - The main I2C1 data pin. It is also SPI0 SCK, I2C1 SDA and PWM1 A.

CircuitPython I2C, SPI and UART

Note that in CircuitPython, there is a board object each for STEMMA QT, I2C, SPI and UART that use the connector and pins labeled on the Feather. You can use these objects to initialize these peripherals in your code.

- `board.STEMMA_I2C()` uses the STEMMA QT connector (in this case, SCL/SDA pins)
- `board.I2C()` uses SCL/SDA pins (GPIO2 and GPIO3)
- `board.SPI()` uses SCK/MO/MI pins (GPIO14, GPIO15 and GPIO8)
- `board.UART()` uses RX/TX pins (GPIO0 and GPIO1)

Arduino I2C, SPI and UART

I2C, SPI and UART can be accessed with these objects in Arduino:

- **Wire** is used for the default I2C and STEMMA QT connector (GPIO2 and GPIO3).
- **SPI** is used for the default SPI pins (GPIO14, GPIO15 and GPIO8).
- **Serial1** is used for the default UART pins (GPIO0 and GPIO1).

The peripheral order is defined in the board support definition for Arduino. For example, you'll notice that even though the default I2C (GPIO2 and GPIO3) is located on I2C1, it is defined as **Wire** rather than **Wire1**.

GPIO Pins by Pin Functionality

Primary pins based on the silkscreen pin labels are bold.

I2C Pins

- I2C0 SCL: A3, D25, RX, D13, D9, D5
- I2C0 SDA: A2, D24, MISO, TX, D12, D4
- I2C1 SCL: **SCL**, A1, MOSI, D11
- I2C1 SDA: **SDA**, A0, SCK, D10, D6

SPI Pins

- SPI0 SCK: D6, SDA
- SPI0 MOSI: SCL
- SPI0 MISO: TX
- SPI0 CS: RX, D5
- SPI1 SCK: **SCK**, A0, D10
- SPI1 MOSI: **MOSI**, A1, D11
- SPI1 MISO: **MISO**, A2, D24, D12
- SPI1 CS: A3, D25, D13, D9

UART Pins

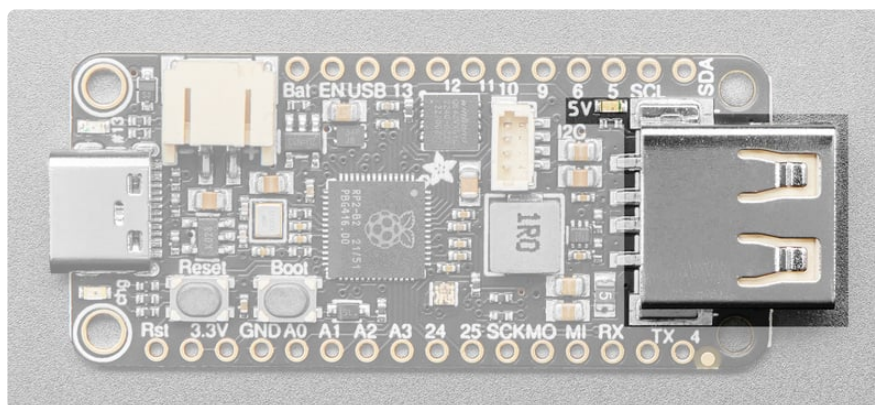
- UART0 TX: **TX**, A2, D12
- UART0 RX: **RX**, A3, D13, D4
- UART1 TX: D24, MISO, D4
- UART1 RX: D25, D9, D5

PWM Pins

- PWM0 A: TX
- PWM0 B: RX

- PWM1 A: SDA
- PWM1 B: SCL
- PWM2 A: D4
- PWM2 B: D5
- PWM3 A: D6
- PWM3 B: (none)
- PWM4 A: D24, MISO
- PWM4 B: D25, D9
- PWM5 A: A0, D10
- PWM5 B: A1, D11
- PWM6 A: A2, D12
- PWM6 B: A3, D13
- PWM7 A: SCK
- PWM7 B: MOSI

USB Type A Input and 5V LED



On the right end of the board is a USB type A port for connecting USB devices to. The RP2040 has a PIO peripheral that can be used to emulate a USB host. This USB-A port can be used for sending or receiving data from an attached USB device.

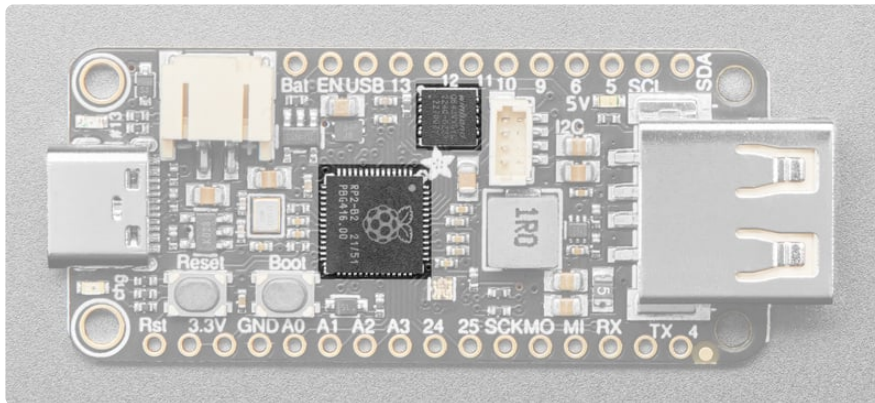
The USB host capability is controlled with the following pins:

- **USB Host Data Plus (D+)** - GPIO 16. In Arduino, this pin can be accessed with `PIN_USB_HOST_DP`. In CircuitPython as `USB_HOST_DATA_PLUS`.
- **USB Host Data Minus (D-)** - GPIO 17. In Arduino, this pin can be accessed with `PIN_USB_HOST_DM`. In CircuitPython as `USB_HOST_DATA_MINUS`.
- **USB Host 5V Power** - GPIO 18. In Arduino, this pin can be accessed with `PIN_5V_EN`. In CircuitPython as `USB_HOST_5V_POWER`.

The 5V LED is a green LED, located directly below **D6/GPIO6** next to the USB-A port. It indicates whether the USB host 5V power is enabled. The 5V output is provided by

a 5V boost converter, [based on the TPS61023](http://adafru.it/4654) (<http://adafru.it/4654>), with up to 1 Amp peak output for USB peripheral power.

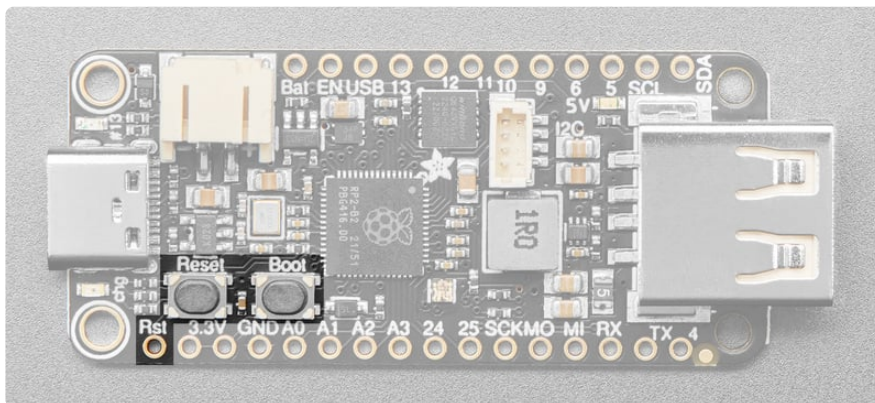
Microcontroller and Flash



The large square towards the middle is the **RP2040 microcontroller**, the "brains" of this Feather board.

The square towards the top-middle is the **QSPI Flash**. It is connected to 6 pins that are not brought out on the GPIO pads. It is used for program and data storage in Arduino and CircuitPython.

Buttons and RST Pin

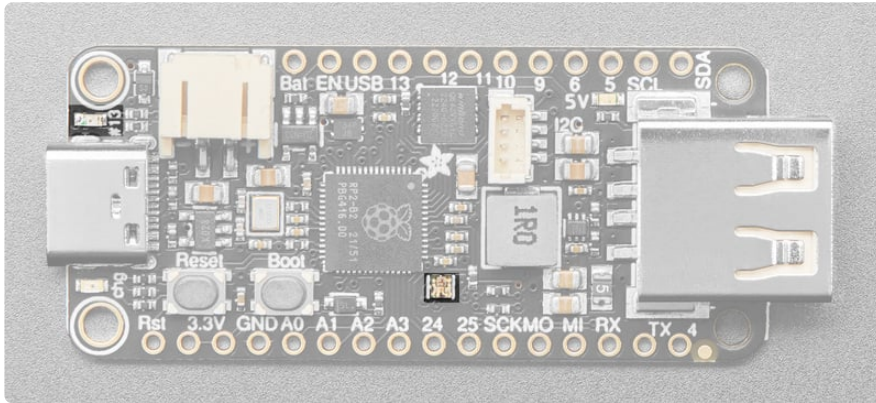


The **Boot button** is the button on the right, located on **GPIO7**. It is available as `board.BUTTON` in CircuitPython, and is available for use in Arduino as `PIN_BUTTON`. It is also used to enter the bootloader. To enter the bootloader, press and hold Boot and then power up the board (either by plugging it into USB or pressing Reset). The bootloader is used to install/update CircuitPython.

The **Reset button** is on the left. It restarts the board and helps enter the bootloader. You can click it to reset the board without unplugging the USB cable or battery.

The **RST pin** can be used to reset the board. Tie to ground manually to reset the board.

NeoPixel and Red LED

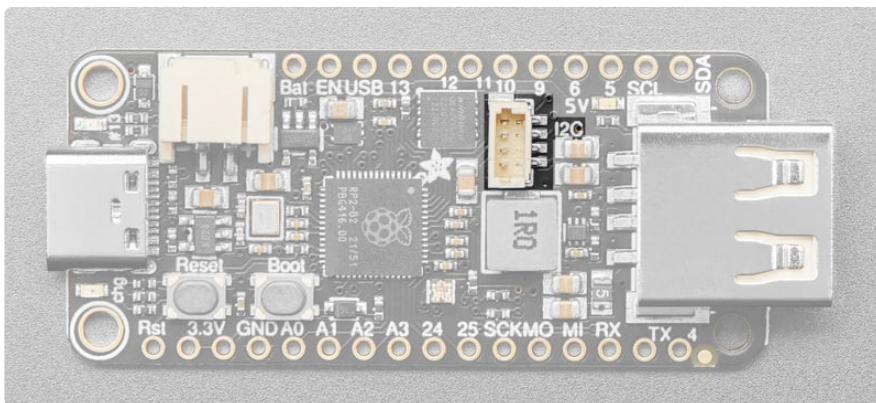


Above the pin labels for D24 and D25 is the status **NeoPixel LED**. It is connected to **GPIO21**. In CircuitPython, the NeoPixel is available at `board.NEOPIXEL` and the library for it is available in [the bundle \(https://adafru.it/ENC\)](https://adafru.it/ENC). In Arduino, it is accessible at `PIN_NEOPIXEL`. The NeoPixel is powered by the 3.3V power supply but that hasn't shown to make a big difference in brightness or color. In CircuitPython, the LED is used to indicate the runtime status.

Additionally, there is a NeoPixel power pin on **GPIO20**. It needs to be set High for the NeoPixel to be powered. This pin is available as `board.NEOPIXEL_POWER` in CircuitPython and `NEOPIXEL_POWER` in Arduino.

Above the USB C connector is the **D13 LED**. This little red LED is controllable in CircuitPython code using `board.LED`, and in Arduino as `PIN_LED`.

STEMMA QT



In the middle of the board, to the left of the USB A connector, is the **STEMMA QT connector**! This means you can connect up [all sorts of I2C sensors and breakouts \(https://adafru.it/18fV\)](https://adafru.it/18fV), no soldering required! This connector uses the SCL and SDA pins for I2C, which end up being the RP2040's I2C1 peripheral. In CircuitPython, you can initialise the STEMMA connector with `board.STEMMA_I2C()` (as well as with `board.SCL` `board.SDA`). In Arduino it is `Wire`.



STEMMA QT / Qwiic JST SH 4-pin Cable - 100mm Long

This 4-wire cable is a little over 100mm / 4" long and fitted with JST-SH female 4-pin connectors on both ends. Compared with the chunkier JST-PH these are 1mm pitch instead of...

<https://www.adafruit.com/product/4210>

Power Management



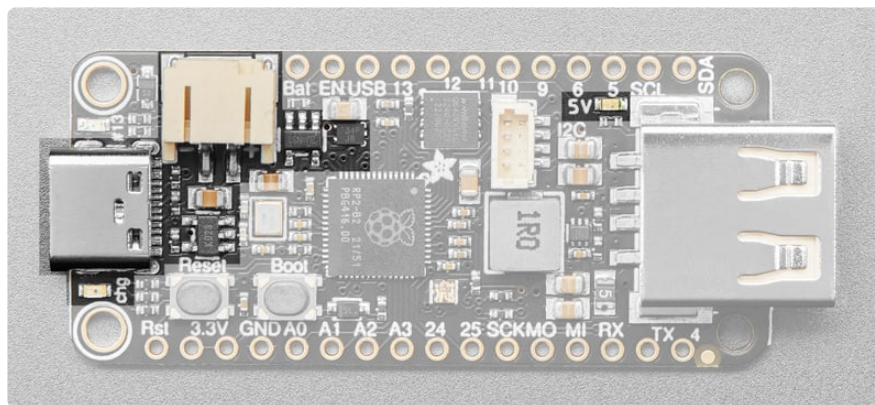
Battery + USB Power

We wanted to make our Feather boards easy to power both when connected to a computer as well as via battery.

There's **two ways to power** a Feather:

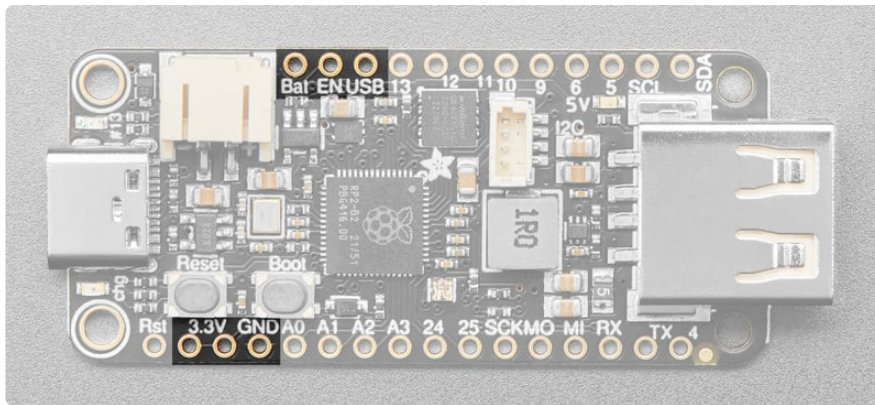
1. You can connect with a USB cable (just plug into the jack) and the Feather will regulate the 5V USB down to 3.3V.
2. You can also connect a 4.2/3.7V Lithium Polymer (LiPo/LiPoly) or Lithium Ion (Lilon) battery to the JST jack. This will let the Feather run on a rechargeable battery.

The JST connector polarity is matched to Adafruit LiPoly batteries. Using wrong polarity batteries can destroy your Feather. Many customers try to save money by purchasing Lipoly batteries from Amazon only to find that they plug them in and the Feather is destroyed!



There's also a **CHG** LED next to the USB jack, which will light up while the battery is charging. This LED might also flicker if the battery is not connected, it's normal.

You have a lot of power supply options here! We bring out the **BAT** pin, which is tied to the LiPoly JST connector, as well as **USB** which is the +5V from USB if connected. We also have the **3V** pin which has the output from the 3.3V regulator. We use a 500mA peak regulator. While you can get 500mA from it, you can't do it continuously from 5V as it will overheat the regulator.



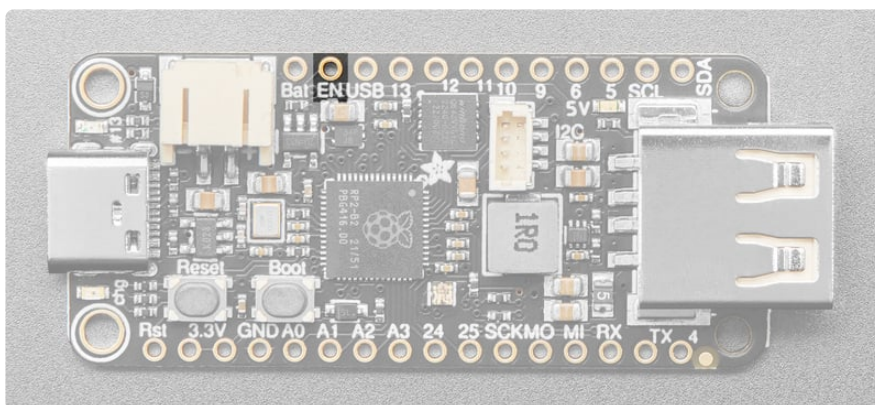
Measuring Battery

If you're running off of a battery, chances are you wanna know what the voltage is at! That way you can tell when the battery needs recharging. LiPoly batteries are 'maxed out' at 4.2V and stick around 3.7V for much of the battery life, then slowly sink down to 3.2V or so before the protection circuitry cuts it off. By measuring the voltage you can quickly tell when you're heading below 3.7V.

Note that unlike other Feathers, we do not have an ADC connected to a battery monitor. Reason being there's only 4 ADCs and we didn't want to use one precious ADC for a battery monitor. You can create a resistor divider from BAT to GND with two 10K resistors and connect the middle to one of the ADC pins on a breadboard.

ENable pin

If you'd like to turn off the 3.3V regulator, you can do that with the **EN**(able) pin. Simply tie this pin to **Ground** and it will disable the 3V regulator. The **BAT** and **USB** pins will still be powered.



NeoPixel Power

The onboard NeoPixel's power is controlled by its own GPIO pin at **GPIO20**. It enabled by default in CircuitPython and Arduino. You can disable it manually for low

power usage. The NeoPixel pin is available in CircuitPython and Arduino as `NEOPIXEL_POWER`.

Alternative Power Options

The two primary ways for powering a feather are a 3.7/4.2V LiPo battery plugged into the JST port or a USB power cable.

If you need other ways to power the Feather, here's what we recommend:

- For permanent installations, a [5V 1A USB wall adapter \(http://adafru.it/501\)](http://adafru.it/501) will let you plug in a USB cable for reliable power
- For mobile use, where you don't want a LiPoly, [use a USB battery pack! \(http://adafru.it/1959\)](http://adafru.it/1959)
- If you have a higher voltage power supply, [use a 5V buck converter \(https://adafru.it/DHs\)](https://adafru.it/DHs) and wire it to a [USB cable's 5V and GND input \(http://adafru.it/3972\)](http://adafru.it/3972)

Here's what you cannot do:

- **Do not use alkaline or NiMH batteries** and connect to the battery port - this will destroy the LiPoly charger
- **Do not use 7.4V RC batteries on the battery port** - this will destroy the board

The Feather is not designed for external power supplies - this is a design decision to make the board compact and low cost. It is not recommended, but technically possible:

- **Connect an external 3.3V power supply to the 3V and GND pins.** Not recommended, this may cause unexpected behavior and the **EN** pin will no longer work. Also this doesn't provide power on **BAT** or **USB** and some Feathers/Wings use those pins for high current usages. You may end up damaging your Feather.
- **Connect an external 5V power supply to the USB and GND pins.** Not recommended, this may cause unexpected behavior when plugging in the USB port because you will be back-powering the USB port, which could confuse or damage your computer.

CircuitPython

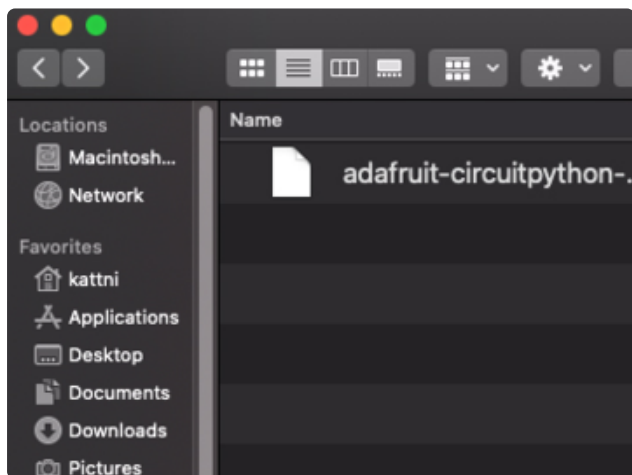
[CircuitPython](https://adafru.it/tB7) (<https://adafru.it/tB7>) is a derivative of [MicroPython](https://adafru.it/BeZ) (<https://adafru.it/BeZ>) designed to simplify experimentation and education on low-cost microcontrollers. It makes it easier than ever to get prototyping by requiring no upfront desktop software downloads. Simply copy and edit files on the **CIRCUITPY** drive to iterate.

CircuitPython Quickstart

Follow this step-by-step to quickly get CircuitPython running on your board.

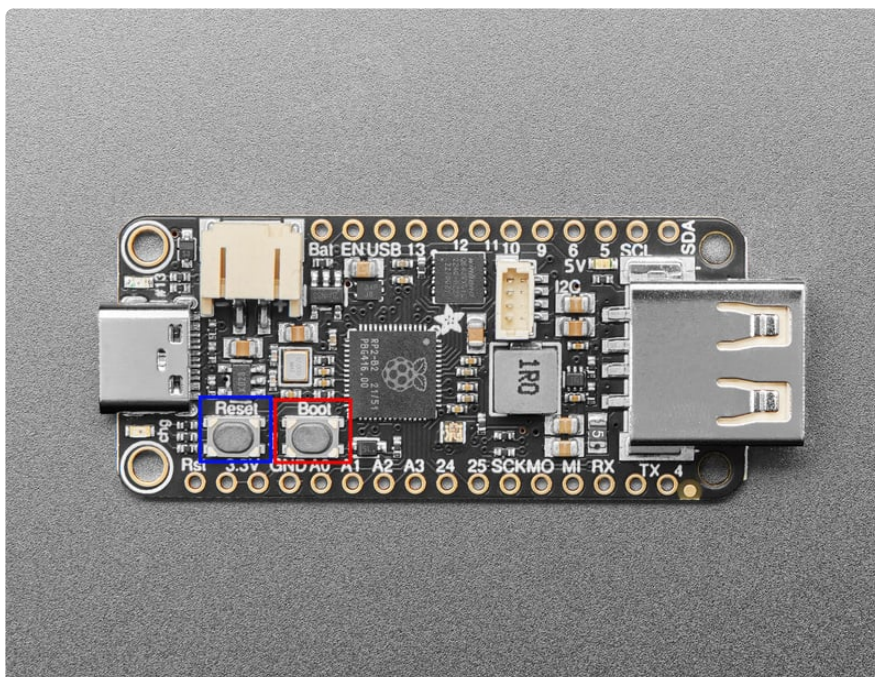
Download the latest version of
CircuitPython for this board via
circuitpython.org

<https://adafru.it/18Du>



Click the link above to download the
latest CircuitPython UF2 file.

Save it wherever is convenient for you.

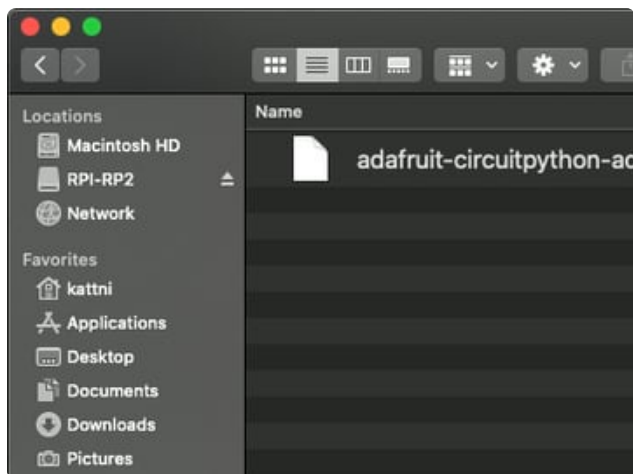


To enter the bootloader, hold down the **BOOT/BOOTSEL button** (highlighted in red above), and while continuing to hold it (don't let go!), press and release the **reset button** (highlighted in red or blue above). **Continue to hold the BOOT/BOOTSEL button until the RPI-RP2 drive appears!**

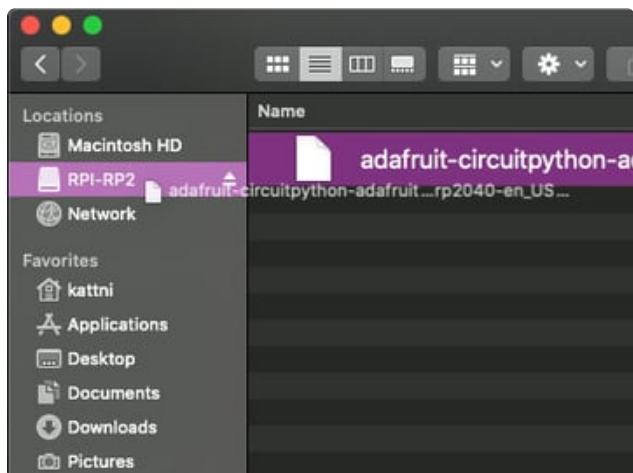
If the drive does not appear, release all the buttons, and then repeat the process above.

You can also start with your board unplugged from USB, press and hold the BOOTSEL button (highlighted in red above), continue to hold it while plugging it into USB, and wait for the drive to appear before releasing the button.

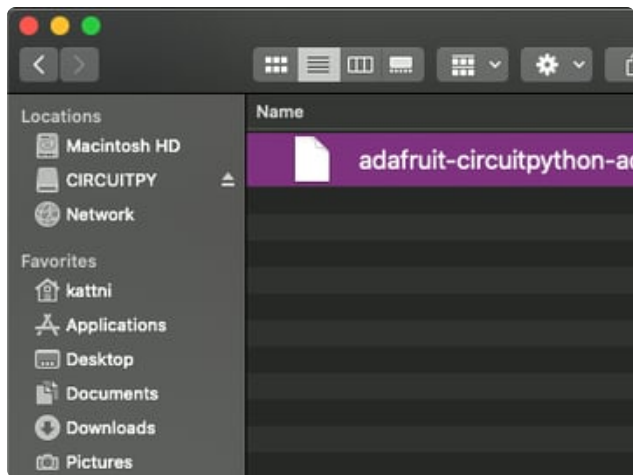
A lot of people end up using charge-only USB cables and it is very frustrating! **Make sure you have a USB cable you know is good for data sync.**



You will see a new disk drive appear called **RPI-RP2**.



Drag the **adafruit_circuitpython_etc.uf2** file to **RPI-RP2**.



The **RPI-RP2** drive will disappear and a new disk drive called **CIRCUITPY** will appear.

That's it, you're done! :)

Safe Mode

You want to edit your `code.py` or modify the files on your **CIRCUITPY** drive, but find that you can't. Perhaps your board has gotten into a state where **CIRCUITPY** is read-only. You may have turned off the **CIRCUITPY** drive altogether. Whatever the reason, safe mode can help.

Safe mode in CircuitPython does not run any user code on startup, and disables auto-reload. This means a few things. First, safe mode bypasses any code in `boot.py` (where you can set **CIRCUITPY** read-only or turn it off completely). Second, it does not run the code in `code.py`. And finally, it does not automatically soft-reload when data is written to the **CIRCUITPY** drive.

Therefore, whatever you may have done to put your board in a non-interactive state, safe mode gives you the opportunity to correct it without losing all of the data on the **CIRCUITPY** drive.

Entering Safe Mode

To enter safe mode when using CircuitPython, plug in your board or hit reset (highlighted in red above). Immediately after the board starts up or resets, it waits 1000ms. On some boards, the onboard status LED (highlighted in green above) will blink yellow during that time. If you press reset during that 1000ms, the board will start up in safe mode. It can be difficult to react to the yellow LED, so you may want to think of it simply as a slow double click of the reset button. (Remember, a fast double click of reset enters the bootloader.)

In Safe Mode

If you successfully enter safe mode on CircuitPython, the LED will intermittently blink yellow three times.

If you connect to the serial console, you'll find the following message.

```
Auto-reload is off.  
Running in safe mode! Not running saved code.  
  
CircuitPython is in safe mode because you pressed the reset button during boot.  
Press again to exit safe mode.  
  
Press any key to enter the REPL. Use CTRL-D to reload.
```

You can now edit the contents of the **CIRCUITPY** drive. Remember, your code will not run until you press the reset button, or unplug and plug in your board, to get out of safe mode.

Flash Resetting UF2

If your board ever gets into a really weird state and CIRCUITPY doesn't show up as a disk drive after installing CircuitPython, try loading this 'nuke' UF2 to RPI-RP2. which will do a 'deep clean' on your Flash Memory. **You will lose all the files on the board**, but at least you'll be able to revive it! After loading this UF2, follow the steps above to re-install CircuitPython.

[Download flash erasing "nuke" UF2](https://adafru.it/RLE)

<https://adafru.it/RLE>

Installing the Mu Editor

Mu is a simple code editor that works with the Adafruit CircuitPython boards. It's written in Python and works on Windows, MacOS, Linux and Raspberry Pi. The serial console is built right in so you get immediate feedback from your board's serial output!

Mu is our recommended editor - please use it (unless you are an experienced coder with a favorite editor already!).

Download and Install Mu



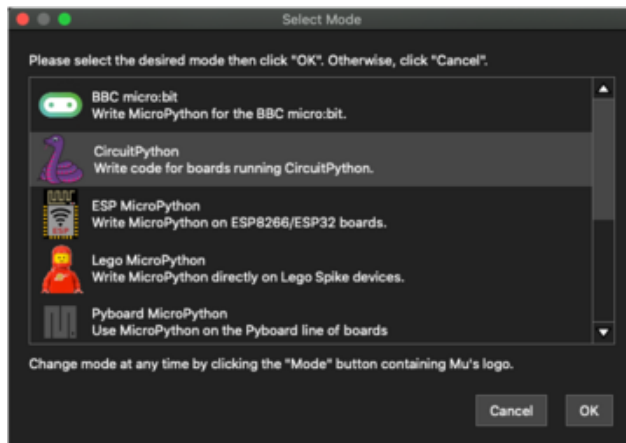
Download Mu from <https://codewith.mu> (<https://adafru.it/Be6>).

Click the **Download** link for downloads and installation instructions.

Click **Start Here** to find a wealth of other information, including extensive tutorials and and how-to's.

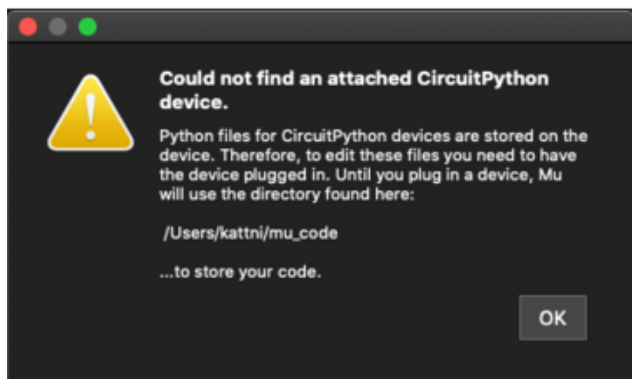
Windows users: due to the nature of MSI installers, please remove old versions of Mu before installing the latest version.

Starting Up Mu



The first time you start Mu, you will be prompted to select your 'mode' - you can always change your mind later. For now please select **CircuitPython**!

The current mode is displayed in the lower right corner of the window, next to the "gear" icon. If the mode says "Microbit" or something else, click the **Mode** button in the upper left, and then choose "CircuitPython" in the dialog box that appears.



Mu attempts to auto-detect your board on startup, so if you do not have a CircuitPython board plugged in with a **CIRCUITPY** drive available, Mu will inform you where it will store any code you save until you plug in a board.

To avoid this warning, plug in a board and ensure that the **CIRCUITPY** drive is mounted before starting Mu.

Using Mu

You can now explore Mu! The three main sections of the window are labeled below; the button bar, the text editor, and the serial console / REPL.



Now you're ready to code! Let's keep going...

The CIRCUITPY Drive

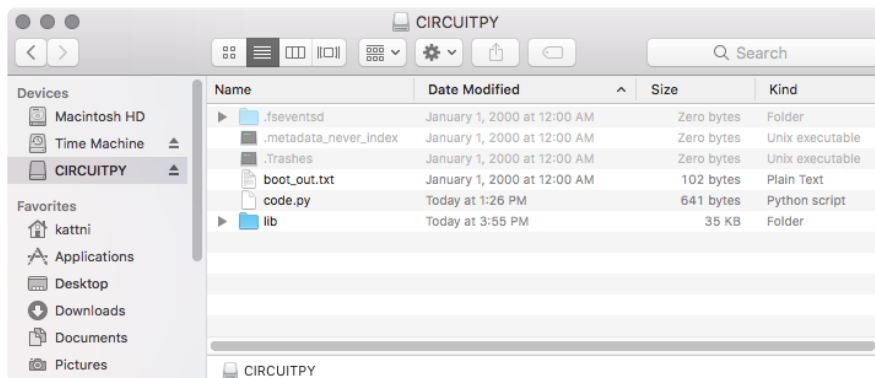
When CircuitPython finishes installing, or you plug a CircuitPython board into your computer with CircuitPython already installed, the board shows up on your computer as a USB drive called **CIRCUITPY**.

The **CIRCUITPY** drive is where your code and the necessary libraries and files will live. You can edit your code directly on this drive and when you save, it will run automatically. When you create and edit code, you'll save your code in a **code.py** file located on the **CIRCUITPY** drive. If you're following along with a Learn guide, you can

paste the contents of the tutorial example into **code.py** on the **CIRCUITPY** drive and save it to run the example.

With a fresh CircuitPython install, on your **CIRCUITPY** drive, you'll find a **code.py** file containing `print("Hello World!")` and an empty **lib** folder. If your **CIRCUITPY** drive does not contain a **code.py** file, you can easily create one and save it to the drive. CircuitPython looks for **code.py** and executes the code within the file automatically when the board starts up or resets. Following a change to the contents of **CIRCUITPY**, such as making a change to the **code.py** file, the board will reset, and the code will be run. You do not need to manually run the code. This is what makes it so easy to get started with your project and update your code!

Note that all changes to the contents of **CIRCUITPY**, such as saving a new file, renaming a current file, or deleting an existing file will trigger a reset of the board.



Boards Without CIRCUITPY

CircuitPython is available for some microcontrollers that do not support native USB. Those boards cannot present a **CIRCUITPY** drive. This includes boards using ESP32 or ESP32-C3 microcontrollers.

On these boards, there are alternative ways to transfer and edit files. You can use the [Thonny editor \(https://adafru.it/18e7\)](https://adafru.it/18e7), which uses hidden commands sent to the REPL to read and write files. Or you can use the CircuitPython web workflow, introduced in Circuitpython 8. The web workflow provides browser-based WiFi access to the CircuitPython filesystem. These guides will help you with the web workflow:

- [CircuitPython on ESP32 Quick Start \(https://adafru.it/10JF\)](https://adafru.it/10JF)
- [CircuitPython Web Workflow Code Editor Quick Start \(https://adafru.it/18e8\)](https://adafru.it/18e8)

Creating and Editing Code

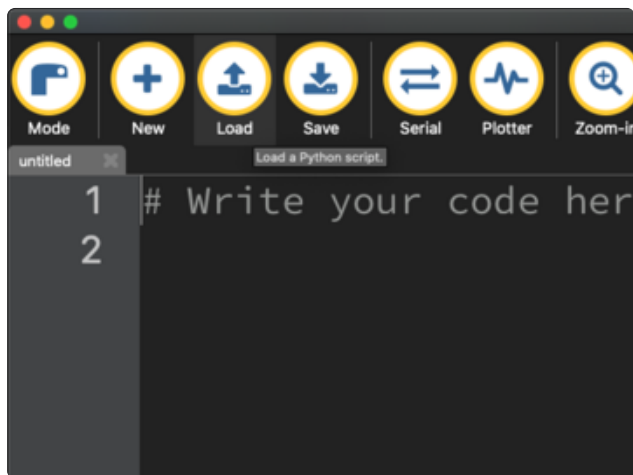
One of the best things about CircuitPython is how simple it is to get code up and running. This section covers how to create and edit your first CircuitPython program.

To create and edit code, all you'll need is an editor. There are many options. **Adafruit strongly recommends using Mu!** It's designed for CircuitPython, and it's really simple and easy to use, with a built in serial console!

If you don't or can't use Mu, there are a number of other editors that work quite well. The [Recommended Editors page \(https://adafru.it/Vue\)](https://adafru.it/Vue) has more details. Otherwise, make sure you do "Eject" or "Safe Remove" on Windows or "sync" on Linux after writing a file if you aren't using Mu. (This was formerly not a problem on macOS, but see the warning below.)

macOS Sonoma 14.1 introduced a bug that delays writes to small drives such as CIRCUITPY drives. This caused errors when saving files to CIRCUITPY. There is a [workaround](#). The bug was fixed in Sonoma 14.4, but at the cost of greatly slowed writes to drives 1GB or smaller.

Creating Code



Installing CircuitPython generates a **code.py** file on your **CIRCUITPY** drive. To begin your own program, open your editor, and load the **code.py** file from the **CIRCUITPY** drive.

If you are using Mu, click the **Load** button in the button bar, navigate to the **CIRCUITPY** drive, and choose **code.py**.

Copy and paste the following code into your editor:

```
import board
import digitalio
import time

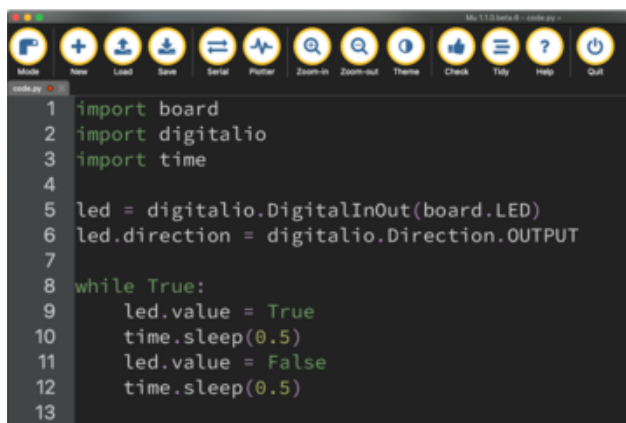
led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

The KB2040, QT Py , Qualia, and the Trinkeys do not have a built-in little red LED! There is an addressable RGB NeoPixel LED. The above example will NOT work on the KB2040, QT Py, Qualia, or the Trinkeys!

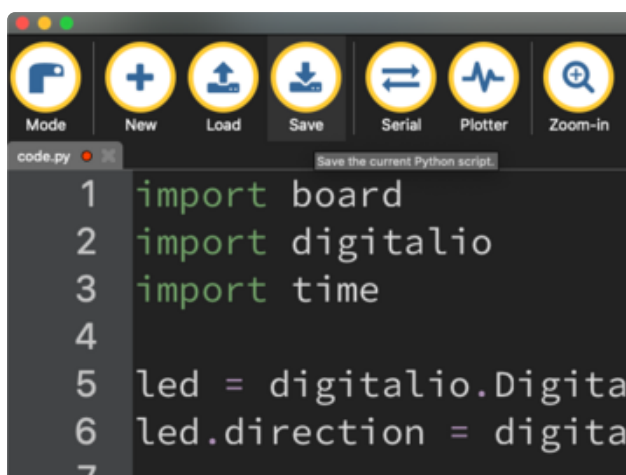
If you're using a KB2040, QT Py, Quaila, or a Trinkey, or any other board without a single-color LED that can blink, please download the [NeoPixel blink example](https://adafru.it/UDU) (<https://adafru.it/UDU>).

The NeoPixel blink example uses the onboard NeoPixel, but the time code is the same. You can use the linked NeoPixel Blink example to follow along with this guide page.



```
1 import board
2 import digitalio
3 import time
4
5 led = digitalio.DigitalInOut(board.LED)
6 led.direction = digitalio.Direction.OUTPUT
7
8 while True:
9     led.value = True
10    time.sleep(0.5)
11    led.value = False
12    time.sleep(0.5)
13
```

It will look like this. Note that under the `while True:` line, the next four lines begin with four spaces to indent them, and they're indented exactly the same amount. All the lines before that have no spaces before the text.



```
1 import board
2 import digitalio
3 import time
4
5 led = digitalio.Digital
6 led.direction = digit
7
```

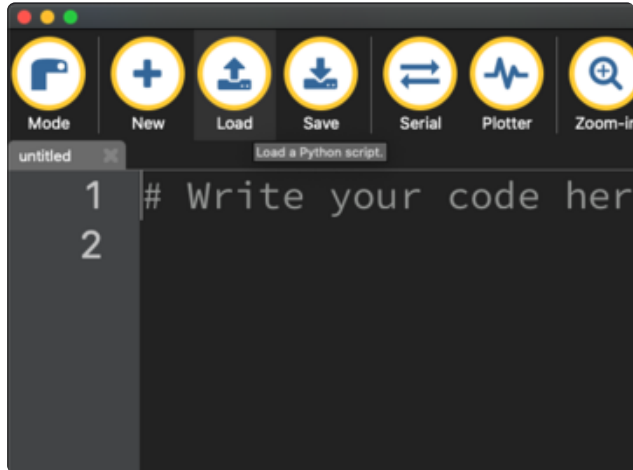
Save the `code.py` file on your **CIRCUITPY** drive.

The little LED should now be blinking. Once per half-second.

Congratulations, you've just run your first CircuitPython program!

On most boards you'll find a tiny red LED. On the ItsyBitsy nRF52840, you'll find a tiny blue LED. On QT Py M0, QT Py RP2040, Qualia, and the Trinky series, you will find only an RGB NeoPixel LED.

Editing Code



To edit code, open the **code.py** file on your **CIRCUITPY** drive into your editor.

Make the desired changes to your code.
Save the file. That's it!

Your code changes are run as soon as the file is done saving.

There's one warning before you continue...

Don't click reset or unplug your board!

The CircuitPython code on your board detects when the files are changed or written and will automatically re-start your code. This makes coding very fast because you save, and it re-runs. If you unplug or reset the board before your computer finishes writing the file to your board, you can corrupt the drive. If this happens, you may lose the code you've written, so it's important to backup your code to your computer regularly.

There are a couple of ways to avoid filesystem corruption.

1. Use an editor that writes out the file completely when you save it.

Check out the [Recommended Editors page \(https://adafru.it/Vue\)](https://adafru.it/Vue) for details on different editing options.

If you are dragging a file from your host computer onto the CIRCUITPY drive, you still need to do step 2. Eject or Sync (below) to make sure the file is completely written.

2. Eject or Sync the Drive After Writing

If you are using one of our not-recommended-editors, not all is lost! You can still make it work.

On Windows, you can Eject or Safe Remove the **CIRCUITPY** drive. It won't actually eject, but it will force the operating system to save your file to disk. On Linux, use the **sync** command in a terminal to force the write to disk.

You also need to do this if you use Windows Explorer or a Linux graphical file manager to drag a file onto **CIRCUITPY**.



Oh No I Did Something Wrong and Now The CIRCUITPY Drive Doesn't Show Up!!!

Don't worry! Corrupting the drive isn't the end of the world (or your board!). If this happens, follow the steps found on the [Troubleshooting \(https://adafru.it/Den\)](https://adafru.it/Den) page of every board guide to get your board up and running again.

If you are having trouble saving code on Windows 10, try including this code snippet at the top of code.py:

```
import supervisor
supervisor.runtime.autoreload = False
```

Back to Editing Code...

Now! Let's try editing the program you added to your board. Open your **code.py** file into your editor. You'll make a simple change. Change the first **0.5** to **0.1**. The code should look like this:

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.1)
    led.value = False
    time.sleep(0.5)
```

Leave the rest of the code as-is. Save your file. See what happens to the LED on your board? Something changed! Do you know why?

You don't have to stop there! Let's keep going. Change the second `0.5` to `0.1` so it looks like this:

```
while True:
    led.value = True
    time.sleep(0.1)
    led.value = False
    time.sleep(0.1)
```

Now it blinks really fast! You decreased the both time that the code leaves the LED on and off!

Now try increasing both of the `0.1` to `1`. Your LED will blink much more slowly because you've increased the amount of time that the LED is turned on and off.

Well done! You're doing great! You're ready to start into new examples and edit them to see what happens! These were simple changes, but major changes are done using the same process. Make your desired change, save it, and get the results. That's really all there is to it!

Naming Your Program File

CircuitPython looks for a code file on the board to run. There are four options: **code.txt**, **code.py**, **main.txt** and **main.py**. CircuitPython looks for those files, in that order, and then runs the first one it finds. While **code.py** is the recommended name for your code file, it is important to know that the other options exist. If your program doesn't seem to be updating as you work, make sure you haven't created another code file that's being read instead of the one you're working on.

Exploring Your First CircuitPython Program

First, you'll take a look at the code you're editing.

Here is the original code again for the LED blink example (if your board doesn't have a single-color LED to blink, look instead at the NeoPixel blink example):

```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

Imports & Libraries

Each CircuitPython program you run needs to have a lot of information to work. The reason CircuitPython is so simple to use is that most of that information is stored in other files and works in the background. The files built into CircuitPython are called **modules**, and the files you load separately are called **libraries**. Modules are built into CircuitPython. Libraries are stored on your **CIRCUITPY** drive in a folder called **lib**.

```
import board
import digitalio
import time
```

The `import` statements tell the board that you're going to use a particular library or module in your code. In this example, you imported three modules: `board`, `digitalio`, and `time`. All three of these modules are built into CircuitPython, so no separate library files are needed. That's one of the things that makes this an excellent first example. You don't need anything extra to make it work!

These three modules each have a purpose. The first one, `board`, gives you access to the hardware on your board. The second, `digitalio`, lets you access that hardware as inputs/outputs. The third, `time`, lets you control the flow of your code in multiple ways, including passing time by 'sleeping'.

Setting Up The LED

The next two lines setup the code to use the LED.

```
led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT
```

Your board knows the red LED as `LED`. So, you initialise that pin, and you set it to output. You set `led` to equal the rest of that information so you don't have to type it all out again later in our code.

Loop-de-loops

The third section starts with a `while` statement. `while True:` essentially means, "forever do the following:". `while True:` creates a loop. Code will loop "while" the condition is "true" (vs. false), and as `True` is never False, the code will loop forever. All code that is indented under `while True:` is "inside" the loop.

Inside our loop, you have four items:

```
while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

First, you have `led.value = True`. This line tells the LED to turn on. On the next line, you have `time.sleep(0.5)`. This line is telling CircuitPython to pause running code for 0.5 seconds. Since this is between turning the led on and off, the led will be on for 0.5 seconds.

The next two lines are similar. `led.value = False` tells the LED to turn off, and `time.sleep(0.5)` tells CircuitPython to pause for another 0.5 seconds. This occurs between turning the led off and back on so the LED will be off for 0.5 seconds too.

Then the loop will begin again, and continue to do so as long as the code is running!

So, when you changed the first `0.5` to `0.1`, you decreased the amount of time that the code leaves the LED on. So it blinks on really quickly before turning off!

Great job! You've edited code in a CircuitPython program!

What Happens When My Code Finishes Running?

When your code finishes running, CircuitPython resets your microcontroller board to prepare it for the next run of code. That means any set up you did earlier no longer applies, and the pin states are reset.

For example, try reducing the code snippet above by eliminating the loop entirely, and replacing it with `led.value = True`. The LED will flash almost too quickly to see, and turn off. This is because the code finishes running and resets the pin state, and the LED is no longer receiving a signal.

To that end, most CircuitPython programs involve some kind of loop, infinite or otherwise.

What if I Don't Have the Loop?

If you don't have the loop, the code will run to the end and exit. This can lead to some unexpected behavior in simple programs like this since the "exit" also resets the state of the hardware. This is a different behavior than running commands via REPL. So if you are writing a simple program that doesn't seem to work, you may need to add a loop to the end so the program doesn't exit.

The simplest loop would be:

```
while True:
    pass
```

And remember - you can press CTRL+C to exit the loop.

See also the [Behavior section in the docs \(https://adafru.it/Bvz\)](https://adafru.it/Bvz).

Connecting to the Serial Console

One of the staples of CircuitPython (and programming in general!) is something called a "print statement". This is a line you include in your code that causes your code to output text. A print statement in CircuitPython (and Python) looks like this:

```
print("Hello, world!")
```

This line in your code.py would result in:

```
Hello, world!
```

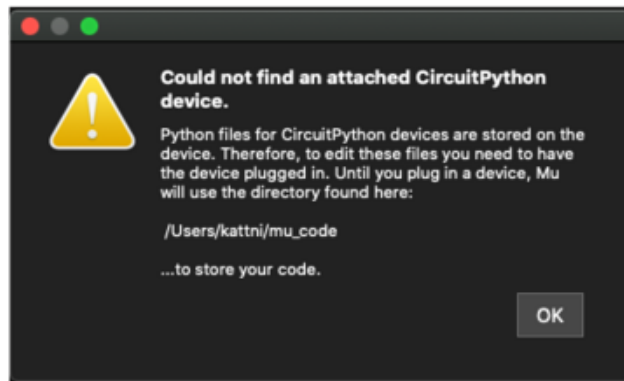
However, these print statements need somewhere to display. That's where the serial console comes in!

The serial console receives output from your CircuitPython board sent over USB and displays it so you can see it. This is necessary when you've included a print statement in your code and you'd like to see what you printed. It is also helpful for troubleshooting errors, because your board will send errors and the serial console will display those too.

The serial console requires an editor that has a built in terminal, or a separate terminal program. A terminal is a program that gives you a text-based interface to perform various tasks.

Are you using Mu?

If so, good news! The serial console is **built into Mu** and will **autodetect your board** making using the serial console really really easy.

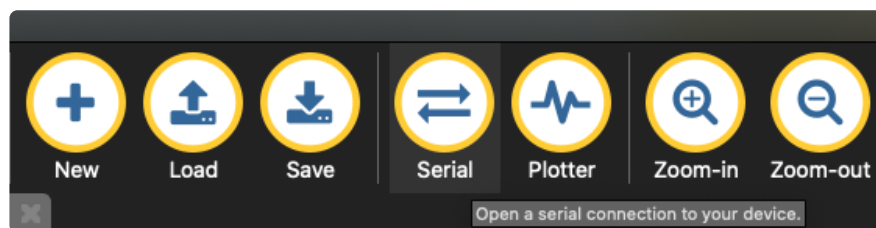


First, make sure your CircuitPython board is plugged in.

If you open Mu without a board plugged in, you may encounter the error seen here, letting you know no CircuitPython board was found and indicating where your code will be stored until you plug in a board.

[If you are using Windows 7, make sure you installed the drivers \(https://adafru.it/VuB\).](https://adafru.it/VuB)

Once you've opened Mu with your board plugged in, look for the **Serial** button in the button bar and click it.



The Mu window will split in two, horizontally, and display the serial console at the bottom.



If nothing appears in the serial console, it may mean your code is done running or has no print statements in it. Click into the serial console part of Mu, and press CTRL+D to reload.

Serial Console Issues or Delays on Linux

If you're on Linux, and are seeing multi-second delays connecting to the serial console, or are seeing "AT" and other gibberish when you connect, then the

`modemmanager` service might be interfering. Just remove it; it doesn't have much use unless you're still using dial-up modems.

To remove `modemmanager`, type the following command at a shell:

```
sudo apt purge modemmanager
```

Setting Permissions on Linux

On Linux, if you see an error box something like the one below when you press the **Serial** button, you need to add yourself to a user group to have permission to connect to the serial console.



On Ubuntu and Debian, add yourself to the **dialout** group by doing:

```
sudo adduser $USER dialout
```

After running the command above, reboot your machine to gain access to the group. On other Linux distributions, the group you need may be different. See the [Advanced Serial Console on Linux \(https://adafru.it/VAO\)](https://adafru.it/VAO) for details on how to add yourself to the right group.

Using Something Else?

If you're not using Mu to edit, are using or if for some reason you are not a fan of its built in serial console, you can run the serial console from a separate program.

Windows requires you to download a terminal program. [Check out the Advanced Serial Console on Windows page for more details. \(https://adafru.it/AAH\)](https://adafru.it/AAH)

MacOS has Terminal built in, though there are other options available for download. [Check the Advanced Serial Console on Mac page for more details. \(https://adafru.it/AAI\)](https://adafru.it/AAI)

Linux has a terminal program built in, though other options are available for download. [Check the Advanced Serial Console on Linux page for more details. \(https://adafru.it/VAO\)](https://adafru.it/VAO)

Once connected, you'll see something like the following.

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.  
code.py output:  
Hello, world!  
  
Code done running.  
  
Press any key to enter the REPL. Use CTRL-D to reload.
```

Interacting with the Serial Console

Once you've successfully connected to the serial console, it's time to start using it.

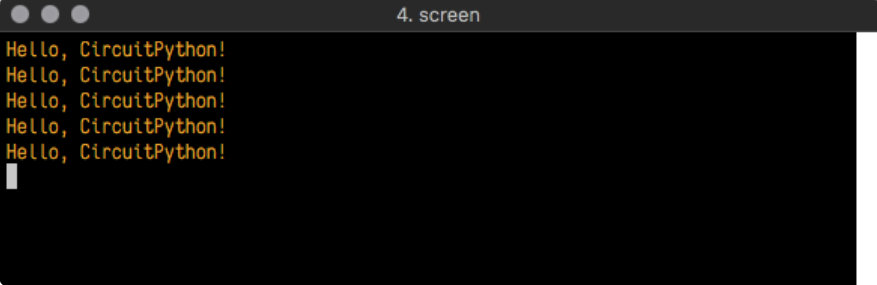
The code you wrote earlier has no output to the serial console. So, you're going to edit it to create some output.

Open your code.py file into your editor, and include a `print` statement. You can print anything you like! Just include your phrase between the quotation marks inside the parentheses. For example:

```
import board  
import digitalio  
import time  
  
led = digitalio.DigitalInOut(board.LED)  
led.direction = digitalio.Direction.OUTPUT  
  
while True:  
    print("Hello, CircuitPython!")  
    led.value = True  
    time.sleep(1)  
    led.value = False  
    time.sleep(1)
```

Save your file.

Now, let's go take a look at the window with our connection to the serial console.



```
4. screen  
Hello, CircuitPython!  
Hello, CircuitPython!  
Hello, CircuitPython!  
Hello, CircuitPython!  
Hello, CircuitPython!  
|
```

Excellent! Our print statement is showing up in our console! Try changing the printed text to something else.

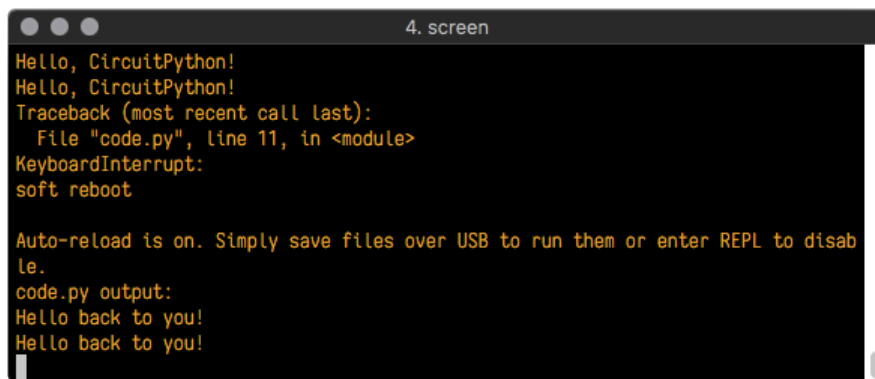
```
import board  
import digitalio
```

```
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    print("Hello back to you!")
    led.value = True
    time.sleep(1)
    led.value = False
    time.sleep(1)
```

Keep your serial console window where you can see it. Save your file. You'll see what the serial console displays when the board reboots. Then you'll see your new change!



```
4. screen
Hello, CircuitPython!
Hello, CircuitPython!
Traceback (most recent call last):
  File "code.py", line 11, in <module>
KeyboardInterrupt:
soft reboot

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Hello back to you!
Hello back to you!
```

The **Traceback (most recent call last):** is telling you the last thing your board was doing before you saved your file. This is normal behavior and will happen every time the board resets. This is really handy for troubleshooting. Let's introduce an error so you can see how it is used.

Delete the **e** at the end of **True** from the line **led.value = True** so that it says **led.value = Tru**

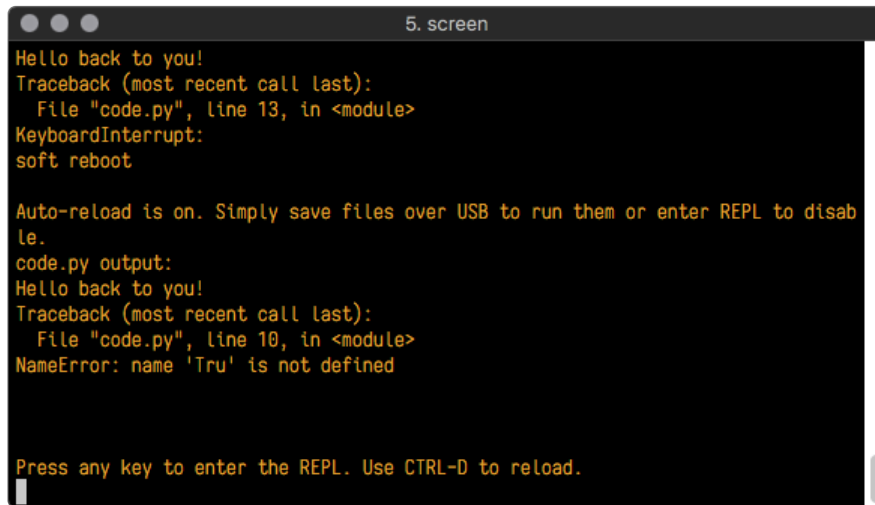
```
import board
import digitalio
import time

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
    print("Hello back to you!")
    led.value = Tru
    time.sleep(1)
    led.value = False
    time.sleep(1)
```

Save your file. You will notice that your red LED will stop blinking, and you may have a colored status LED blinking at you. This is because the code is no longer correct and can no longer run properly. You need to fix it!

Usually when you run into errors, it's not because you introduced them on purpose. You may have 200 lines of code, and have no idea where your error could be hiding. This is where the serial console can help. Let's take a look!



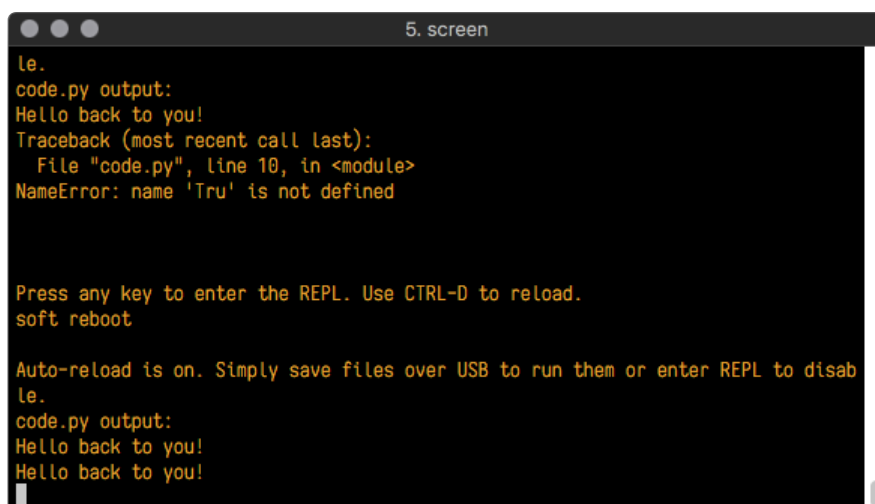
```
5. screen
Hello back to you!
Traceback (most recent call last):
  File "code.py", line 13, in <module>
KeyboardInterrupt:
soft reboot

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Hello back to you!
Traceback (most recent call last):
  File "code.py", line 10, in <module>
NameError: name 'Tru' is not defined

Press any key to enter the REPL. Use CTRL-D to reload.
```

The **Traceback (most recent call last):** is telling you that the last thing it was able to run was **line 10** in your code. The next line is your error: **NameError: name 'Tru' is not defined**. This error might not mean a lot to you, but combined with knowing the issue is on line 10, it gives you a great place to start!

Go back to your code, and take a look at line 10. Obviously, you know what the problem is already. But if you didn't, you'd want to look at line 10 and see if you could figure it out. If you're still unsure, try googling the error to get some help. In this case, you know what to look for. You spelled True wrong. Fix the typo and save your file.



```
5. screen
le.
code.py output:
Hello back to you!
Traceback (most recent call last):
  File "code.py", line 10, in <module>
NameError: name 'Tru' is not defined

Press any key to enter the REPL. Use CTRL-D to reload.
soft reboot

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Hello back to you!
Hello back to you!
```

Nice job fixing the error! Your serial console is streaming and your red LED is blinking again.

The serial console will display any output generated by your code. Some sensors, such as a humidity sensor or a thermistor, receive data and you can use print statements to display that information. You can also use print statements for

troubleshooting, which is called "print debugging". Essentially, if your code isn't working, and you want to know where it's failing, you can put print statements in various places to see where it stops printing.

The serial console has many uses, and is an amazing tool overall for learning and programming!

The REPL

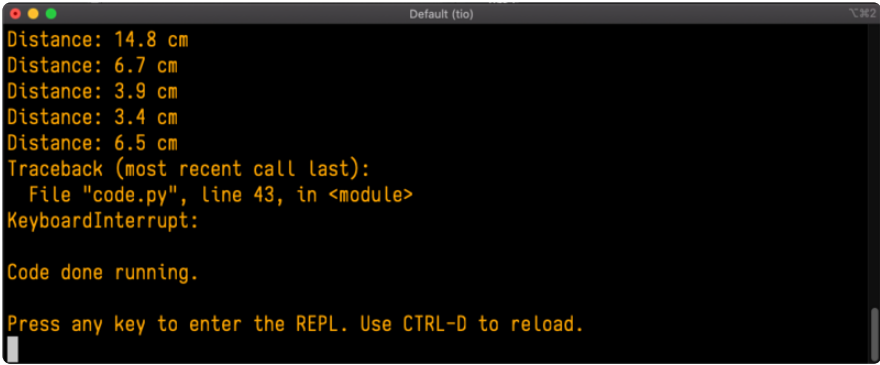
The other feature of the serial connection is the **Read-Evaluate-Print-Loop**, or REPL. The REPL allows you to enter individual lines of code and have them run immediately. It's really handy if you're running into trouble with a particular program and can't figure out why. It's interactive so it's great for testing new ideas.

Entering the REPL

To use the REPL, you first need to be connected to the serial console. Once that connection has been established, you'll want to press **CTRL+C**.

If there is code running, in this case code measuring distance, it will stop and you'll see **Press any key to enter the REPL. Use CTRL-D to reload.** Follow those instructions, and press any key on your keyboard.

The **Traceback (most recent call last):** is telling you the last thing your board was doing before you pressed Ctrl + C and interrupted it. The **KeyboardInterrupt** is you pressing CTRL+C. This information can be handy when troubleshooting, but for now, don't worry about it. Just note that it is expected behavior.



```
Distance: 14.8 cm
Distance: 6.7 cm
Distance: 3.9 cm
Distance: 3.4 cm
Distance: 6.5 cm
Traceback (most recent call last):
  File "code.py", line 43, in <module>
KeyboardInterrupt:

Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.
```

If your **code.py** file is empty or does not contain a loop, it will show an empty output and **Code done running.** There is no information about what your board was doing before you interrupted it because there is no code running.


```
Default (tio)
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:

Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.
```

If you have no `code.py` on your **CIRCUITPY** drive, you will enter the REPL immediately after pressing CTRL+C. Again, there is no information about what your board was doing before you interrupted it because there is no code running.

```
Default (tio)
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.

Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.
```

Regardless, once you press a key you'll see a `>>>` prompt welcoming you to the REPL!

```
Default (tio)
Adafruit CircuitPython 7.0.0 on 2021-10-26; Adafruit Feather RP2040 with rp2040
>>> |
```

If you have trouble getting to the `>>>` prompt, try pressing Ctrl + C a few more times. The first thing you get from the REPL is information about your board.

```
Adafruit CircuitPython 7.0.0 on 2021-10-26; Adafruit Feather RP2040 with rp2040
```

This line tells you the version of CircuitPython you're using and when it was released. Next, it gives you the type of board you're using and the type of microcontroller the board uses. Each part of this may be different for your board depending on the versions you're working with.

This is followed by the CircuitPython prompt.

```
>>> |
```

Interacting with the REPL

From this prompt you can run all sorts of commands and code. The first thing you'll do is run `help()`. This will tell you where to start exploring the REPL. To run code in the REPL, type it in next to the REPL prompt.

Type `help()` next to the prompt in the REPL.

```

Adafruit CircuitPython 7.0.0 on 2021-10-26; Adafruit Feather RP2040 with rp2040
>>> help()

```

Then press enter. You should then see a message.

```

Adafruit CircuitPython 7.0.0 on 2021-10-26; Adafruit Feather RP2040 with rp2040
>>> help()
Welcome to Adafruit CircuitPython 7.0.0!

Visit circuitpython.org for more information.

To list built-in modules type `help("modules")`.
>>>

```

First part of the message is another reference to the version of CircuitPython you're using. Second, a URL for the CircuitPython related project guides. Then... wait. What's this? To list built-in modules type `help("modules")`. Remember the modules you learned about while going through creating code? That's exactly what this is talking about! This is a perfect place to start. Let's take a look!

Type `help("modules")` into the REPL next to the prompt, and press enter.

```

>>> help("modules")
__main__      board          micropython    storage
_bleio        builtins       msgpack         struct
adafruit_bus_device collections busio           neopixel_write supervisor
adafruit_pixelbuf countio        onewireio       synthio
aesio          digitalio     os              sys
alarm          displayio    paralleldisplay terminalio
analogio       errno        pwmio           time
array          fontio       qrio            traceback
atexit         framebufferio rainbowlow       ulab
audiobusio     gc           random          usb_cdc
audiocore      getpass      re              usb_hid
audiomixer     imagecapture rgbmatrix       usb_midi
audiopwmio     io           rotaryio        vectorio
binascii       json         rp2pio          watchdog
bitbangio      keypad       rtc
bitmaptools    math         sdcardio
bitops         microcontroller sharpdisplay
Plus any modules on the filesystem
>>>

```

This is a list of all the core modules built into CircuitPython, including `board`. Remember, `board` contains all of the pins on the board that you can use in your code. From the REPL, you are able to see that list!

Type `import board` into the REPL and press enter. It'll go to a new prompt. It might look like nothing happened, but that's not the case! If you recall, the `import` statement simply tells the code to expect to do something with that module. In this case, it's telling the REPL that you plan to do something with that module.

```

>>> import board
>>>

```

Next, type `dir(board)` into the REPL and press enter.

```
>>> dir(board)
['_class_', '_name_', 'A0', 'A1', 'A2', 'A3', 'D0', 'D1', 'D10', 'D11', 'D12', 'D13',
'D24', 'D25', 'D4', 'D5', 'D6', 'D9', 'I2C', 'LED', 'MISO', 'MOSI', 'NEOPIXEL', 'RX', 'SCK',
'SCL', 'SDA', 'SPI', 'TX', 'UART', 'board_id']
>>>
```

This is a list of all of the pins on your board that are available for you to use in your code. Each board's list will differ slightly depending on the number of pins available. Do you see **LED** ? That's the pin you used to blink the red LED!

The REPL can also be used to run code. Be aware that **any code you enter into the REPL isn't saved** anywhere. If you're testing something new that you'd like to keep, make sure you have it saved somewhere on your computer as well!

Every programmer in every programming language starts with a piece of code that says, "Hello, World." You're going to say hello to something else. Type into the REPL:

```
print("Hello, CircuitPython!")
```

Then press enter.

```
>>> print("Hello, CircuitPython")
Hello, CircuitPython
>>>
```

That's all there is to running code in the REPL! Nice job!

You can write single lines of code that run stand-alone. You can also write entire programs into the REPL to test them. Remember that nothing typed into the REPL is saved.

There's a lot the REPL can do for you. It's great for testing new ideas if you want to see if a few new lines of code will work. It's fantastic for troubleshooting code by entering it one line at a time and finding out where it fails. It lets you see what modules are available and explore those modules.

Try typing more into the REPL to see what happens!

Everything typed into the REPL is ephemeral. Once you reload the REPL or return to the serial console, nothing you typed will be retained in any memory space. So be sure to save any desired code you wrote somewhere else, or you'll lose it when you leave the current REPL instance!

Returning to the Serial Console

When you're ready to leave the REPL and return to the serial console, simply press **CTRL+D**. This will reload your board and reenter the serial console. You will restart the program you had running before entering the REPL. In the console window, you'll see

any output from the program you had running. And if your program was affecting anything visual on the board, you'll see that start up again as well.

You can return to the REPL at any time!

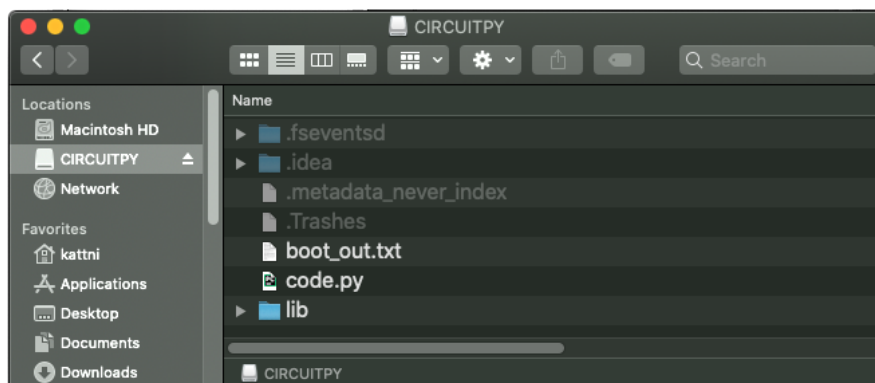
A terminal window titled "Default (tio)" with a black background and yellow text. The text inside the window reads: "Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.", "Code done running.", and "Press any key to enter the REPL. Use CTRL-D to reload."

CircuitPython Libraries

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. Visit <https://circuitpython.org/downloads> to download the latest version of CircuitPython for your board. You must download the CircuitPython Library Bundle that matches your version of CircuitPython. Please update CircuitPython and then visit <https://circuitpython.org/libraries> to download the latest Library Bundle.

Each CircuitPython program you run needs to have a lot of information to work. The reason CircuitPython is so simple to use is that most of that information is stored in other files and works in the background. These files are called libraries. Some of them are built into CircuitPython. Others are stored on your **CIRCUITPY** drive in a folder called **lib**. Part of what makes CircuitPython so great is its ability to store code separately from the firmware itself. Storing code separately from the firmware makes it easier to update both the code you write and the libraries you depend.

Your board may ship with a **lib** folder already, it's in the base directory of the drive. If not, simply create the folder yourself. When you first install CircuitPython, an empty **lib** directory will be created for you.



CircuitPython libraries work in the same way as regular Python modules so the [Python docs \(https://adafru.it/rar\)](https://adafru.it/rar) are an excellent reference for how it all should work. In

Python terms, you can place our library files in the **lib** directory because it's part of the Python path by default.

One downside of this approach of separate libraries is that they are not built in. To use them, one needs to copy them to the **CIRCUITPY** drive before they can be used. Fortunately, there is a library bundle.

The bundle and the library releases on GitHub also feature optimized versions of the libraries with the **.mpy** file extension. These files take less space on the drive and have a smaller memory footprint as they are loaded.

Due to the regular updates and space constraints, Adafruit does not ship boards with the entire bundle. Therefore, you will need to load the libraries you need when you begin working with your board. You can find example code in the guides for your board that depends on external libraries.

Either way, as you start to explore CircuitPython, you'll want to know how to get libraries on board.

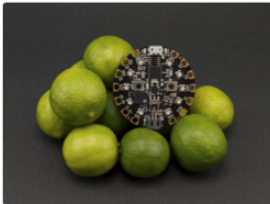
The Adafruit Learn Guide Project Bundle

The quickest and easiest way to get going with a project from the Adafruit Learn System is by utilising the Project Bundle. Most guides now have a **Download Project Bundle** button available at the top of the full code example embed. This button downloads all the necessary files, including images, etc., to get the guide project up and running. Simply click, open the resulting zip, copy over the right files, and you're good to go!

The first step is to find the Download Project Bundle button in the guide you're working on.

The Download Project Bundle button is only available on full demo code embedded from GitHub in a Learn guide. Code snippets will NOT have the button available.

🏠 > Circuit Playground Express: Piano in the Key of Lime > Piano in the Key of Lime



Piano in the Key of Lime

Now we'll take everything we learned and put it together!

Be sure to save your current code.py if you've changed anything you'd like to keep. Download the following file. Rename it to code.py and save it to your Circuit Playground Express.

[Download Project Bundle](#) [Copy Code](#)

```
# SPDX-FileCopyrightText: 2017 Kattni Rembor for Adafruit Industries
#
# SPDX-License-Identifier: MIT

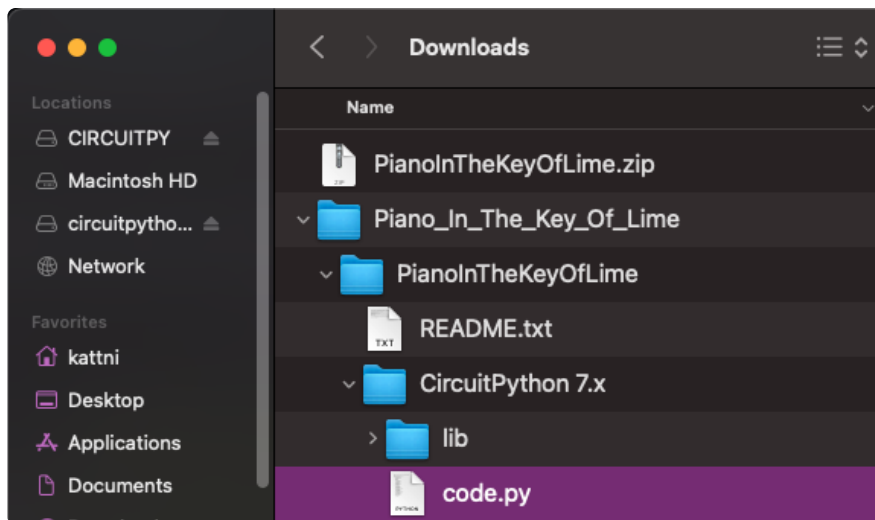
from adafruit_circuitplayground import cp

while True:
    if cp.switch:
        print("Slide switch off!")
        cp.pixels.fill((0, 0, 0))
```

Circuit Playground Express: Piano in the Key of Lime
By [Kattni Rembor](#)
Create a full scale tone piano using CircuitPython, capacitive touch and some cute little fruits.

When you copy the contents of the Project Bundle to your CIRCUITPY drive, it will replace all the existing content! If you don't want to lose anything, ensure you copy your current code to your computer before you copy over the new Project Bundle content!

The Download Project Bundle button downloads a zip file. This zip contains a series of directories, nested within which is the **code.py**, any applicable assets like images or audio, and the **lib/** folder containing all the necessary libraries. The following zip was downloaded from the Piano in the Key of Lime guide.



The Piano in the Key of Lime guide was chosen as an example. That guide is specific to Circuit Playground Express, and cannot be used on all boards. Do not expect to download that exact bundle and have it work on your non-CPX microcontroller.

When you open the zip, you'll find some nested directories. Navigate through them until you find what you need. You'll eventually find a directory for your CircuitPython version (in this case, 7.x). In the version directory, you'll find the file and directory you need: **code.py** and **lib/**. Once you find the content you need, you can copy it all over to your **CIRCUITPY** drive, replacing any files already on the drive with the files from the freshly downloaded zip.

In some cases, there will be other files such as audio or images in the same directory as **code.py** and **lib/**. Make sure you include all the files when you copy things over!

Once you copy over all the relevant files, the project should begin running! If you find that the project is not running as expected, make sure you've copied ALL of the project files onto your microcontroller board.

That's all there is to using the Project Bundle!

The Adafruit CircuitPython Library Bundle

Adafruit provides CircuitPython libraries for much of the hardware they provide, including sensors, breakouts and more. To eliminate the need for searching for each library individually, the libraries are available together in the Adafruit CircuitPython Library Bundle. The bundle contains all the files needed to use each library.

Downloading the Adafruit CircuitPython Library Bundle

You can download the latest Adafruit CircuitPython Library Bundle release by clicking the button below. The libraries are being constantly updated and improved, so you'll always want to download the latest bundle.

Match up the bundle version with the version of CircuitPython you are running. For example, you would download the 6.x library bundle if you're running any version of CircuitPython 6, or the 7.x library bundle if you're running any version of CircuitPython 7, etc. If you mix libraries with major CircuitPython versions, you will get incompatible mpy errors due to changes in library interfaces possible during major version changes.

Click to visit circuitpython.org for the latest Adafruit CircuitPython Library Bundle

<https://adafru.it/ENC>

Download the bundle version that matches your CircuitPython firmware version. If you don't know the version, check the version info in `boot_out.txt` file on the **CIRCUITPY** drive, or the initial prompt in the CircuitPython REPL. For example, if you're running v7.0.0, download the 7.x library bundle.

There's also a `py` bundle which contains the uncompressed python files, you probably don't want that unless you are doing advanced work on libraries.

The CircuitPython Community Library Bundle

The CircuitPython Community Library Bundle is made up of libraries written and provided by members of the CircuitPython community. These libraries are often written when community members encountered hardware not supported in the Adafruit Bundle, or to support a personal project. The authors all chose to submit these libraries to the Community Bundle make them available to the community.

These libraries are maintained by their authors and are not supported by Adafruit.

As you would with any library, if you run into problems, feel free to file an issue on the GitHub repo for the library. Bear in mind, though, that most of these libraries are supported by a single person and you should be patient about receiving a response. Remember, these folks are not paid by Adafruit, and are volunteering their personal time when possible to provide support.

Downloading the CircuitPython Community Library Bundle

You can download the latest CircuitPython Community Library Bundle release by clicking the button below. The libraries are being constantly updated and improved, so you'll always want to download the latest bundle.

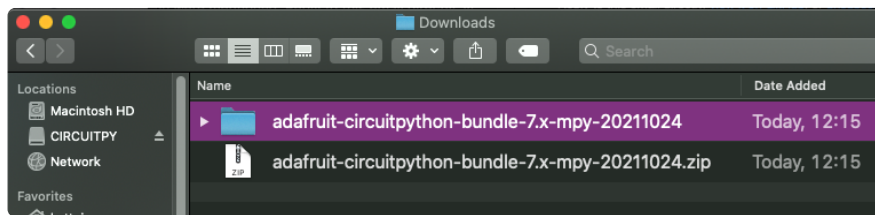
Click for the latest CircuitPython
Community Library Bundle release

<https://adafru.it/VCn>

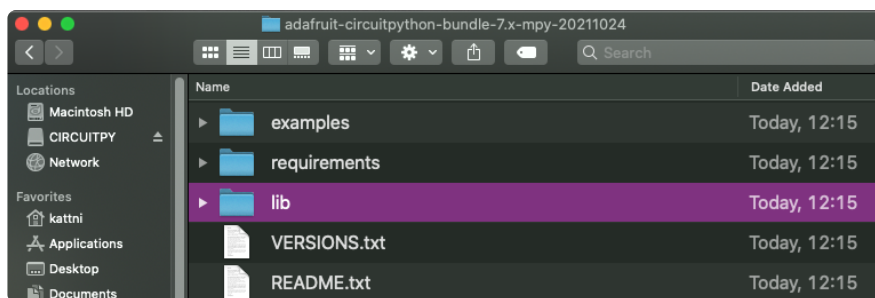
The link takes you to the latest release of the CircuitPython Community Library Bundle on GitHub. There are multiple versions of the bundle available. **Download the bundle version that matches your CircuitPython firmware version.** If you don't know the version, check the version info in `boot_out.txt` file on the **CIRCUITPY** drive, or the initial prompt in the CircuitPython REPL. For example, if you're running v7.0.0, download the 7.x library bundle.

Understanding the Bundle

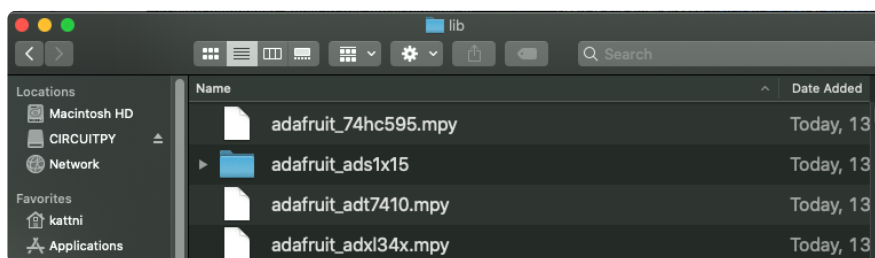
After downloading the zip, extract its contents. This is usually done by double clicking on the zip. On Mac OSX, it places the file in the same directory as the zip.



Open the bundle folder. Inside you'll find two information files, and two folders. One folder is the lib bundle, and the other folder is the examples bundle.



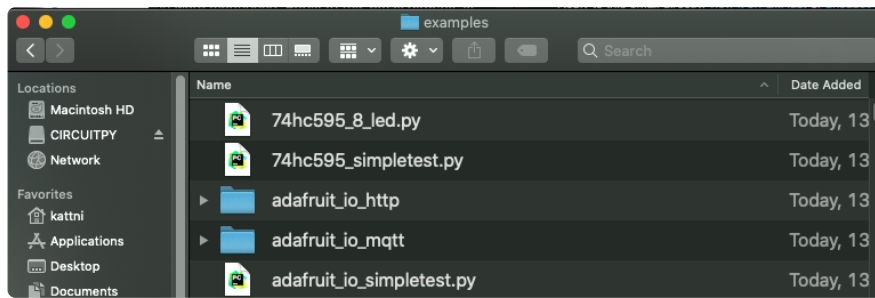
Now open the lib folder. When you open the folder, you'll see a large number of .mpy files, and folders.



Example Files

All example files from each library are now included in the bundles in an **examples** directory (as seen above), as well as an examples-only bundle. These are included for two main reasons:

- Allow for quick testing of devices.
- Provide an example base of code, that is easily built upon for individualized purposes.



Copying Libraries to Your Board

First open the **lib** folder on your **CIRCUITPY** drive. Then, open the **lib** folder you extracted from the downloaded zip. Inside you'll find a number of folders and **.mpy** files. Find the library you'd like to use, and copy it to the **lib** folder on **CIRCUITPY**.

If the library is a directory with multiple **.mpy** files in it, be sure to **copy the entire folder to CIRCUITPY/lib**.

This also applies to example files. Open the **examples** folder you extracted from the downloaded zip, and copy the applicable file to your **CIRCUITPY** drive. Then, rename it to **code.py** to run it.

If a library has multiple **.mpy** files contained in a folder, be sure to copy the entire folder to **CIRCUITPY/lib**.

Understanding Which Libraries to Install

You now know how to load libraries on to your CircuitPython-compatible microcontroller board. You may now be wondering, how do you know which libraries you need to install? Unfortunately, it's not always straightforward. Fortunately, there is an obvious place to start, and a relatively simple way to figure out the rest. First up: the best place to start.

When you look at most CircuitPython examples, you'll see they begin with one or more **import** statements. These typically look like the following:

- `import library_or_module`

However, **import** statements can also sometimes look like the following:

- `from library_or_module import name`
- `from library_or_module.subpackage import name`

- `from library_or_module import name as local_name`

They can also have more complicated formats, such as including a `try` / `except` block, etc.

The important thing to know is that an `import` statement will always include the name of the module or library that you're importing.

Therefore, the best place to start is by reading through the `import` statements.

Here is an example import list for you to work with in this section. There is no setup or other code shown here, as the purpose of this section involves only the import list.

```
import time
import board
import neopixel
import adafruit_lis3dh
import usb_hid
from adafruit_hid.consumer_control import ConsumerControl
from adafruit_hid.consumer_control_code import ConsumerControlCode
```

Keep in mind, not all imported items are libraries. Some of them are almost always built-in CircuitPython modules. How do you know the difference? Time to visit the REPL.

In the [Interacting with the REPL section \(https://adafru.it/Awz\)](https://adafru.it/Awz) on [The REPL page \(https://adafru.it/Awz\)](https://adafru.it/Awz) in this guide, the `help("modules")` command is discussed. This command provides a list of all of the built-in modules available in CircuitPython for your board. So, if you connect to the serial console on your board, and enter the REPL, you can run `help("modules")` to see what modules are available for your board. Then, as you read through the `import` statements, you can, for the purposes of figuring out which libraries to load, ignore the statement that import modules.

The following is the list of modules built into CircuitPython for the Feather RP2040. Your list may look similar or be anything down to a significant subset of this list for smaller boards.

```
>>> help("modules")
__main__      board          micropython    storage
_bleio        builtins       msgpack        struct
adafruit_bus_device collections busio          neopixel_write supervisor
adafruit_pixelbuf countio        onewireio      synthio
aesio         digitalio     os             sys
alarm         displayio    paralleldisplay terminalio
analogio      errno        pwmio          time
array         fontio       qrio           touchio
atexit        framebufferio rainbowio       traceback
audiobusio    gc           random         ulab
audiocore     getpass      re             usb_cdc
audiomixer    imagecapture rgbmatrix      usb_hid
audiomp3      io           rotaryio       usb_midi
audiopwmio    json         rp2pio         vectorio
binascii      keypad       rtc            watchdog
bitbangio     math         sdcardio
bitmaptools   microcontroller sharpdisplay
bitops
```

Now that you know what you're looking for, it's time to read through the import statements. The first two, `time` and `board`, are on the modules list above, so they're built-in.

The next one, `neopixel`, is not on the module list. That means it's your first library! So, you would head over to the bundle zip you downloaded, and search for `neopixel`. There is a `neopixel.mpy` file in the bundle zip. Copy it over to the `lib` folder on your **CIRCUITPY** drive. The following one, `adafruit_lis3dh`, is also not on the module list. Follow the same process for `adafruit_lis3dh`, where you'll find `adafruit_lis3dh.mpy`, and copy that over.

The fifth one is `usb_hid`, and it is in the modules list, so it is built in. Often all of the built-in modules come first in the import list, but sometimes they don't! Don't assume that everything after the first library is also a library, and verify each import with the modules list to be sure. Otherwise, you'll search the bundle and come up empty!

The final two imports are not as clear. Remember, when `import` statements are formatted like this, the first thing after the `from` is the library name. In this case, the library name is `adafruit_hid`. A search of the bundle will find an `adafruit_hid` folder. When a library is a folder, you must copy the **entire folder and its contents as it is in the bundle** to the `lib` folder on your **CIRCUITPY** drive. In this case, you would copy the entire `adafruit_hid` folder to your **CIRCUITPY/lib** folder.

Notice that there are two imports that begin with `adafruit_hid`. Sometimes you will need to import more than one thing from the same library. Regardless of how many times you import the same library, you only need to load the library by copying over the `adafruit_hid` folder once.

That is how you can use your example code to figure out what libraries to load on your CircuitPython-compatible board!

There are cases, however, where libraries require other libraries internally. The internally required library is called a dependency. In the event of library

dependencies, the easiest way to figure out what other libraries are required is to connect to the serial console and follow along with the `ImportError` printed there. The following is a very simple example of an `ImportError`, but the concept is the same for any missing library.

Example: `ImportError` Due to Missing Library

If you choose to load libraries as you need them, or you're starting fresh with an existing example, you may end up with code that tries to use a library you haven't yet loaded. This section will demonstrate what happens when you try to utilise a library that you don't have loaded on your board, and cover the steps required to resolve the issue.

This demonstration will only return an error if you do not have the required library loaded into the `lib` folder on your **CIRCUITPY** drive.

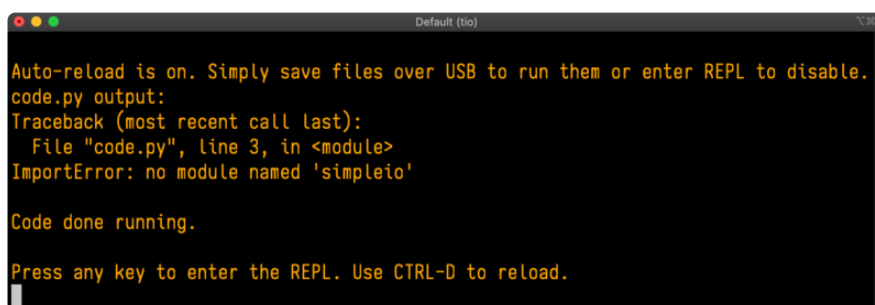
Let's use a modified version of the Blink example.

```
import board
import time
import simpleio

led = simpleio.DigitalOut(board.LED)

while True:
    led.value = True
    time.sleep(0.5)
    led.value = False
    time.sleep(0.5)
```

Save this file. Nothing happens to your board. Let's check the serial console to see what's going on.

A screenshot of a serial console window titled "Default (tio)". The text inside shows the output of a program: "Auto-reload is on. Simply save files over USB to run them or enter REPL to disable." followed by "code.py output:". Then a traceback is shown: "Traceback (most recent call last):", "File \"code.py\", line 3, in <module>", and "ImportError: no module named 'simpleio'". Below this, it says "Code done running." and "Press any key to enter the REPL. Use CTRL-D to reload.".

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Traceback (most recent call last):
  File "code.py", line 3, in <module>
ImportError: no module named 'simpleio'

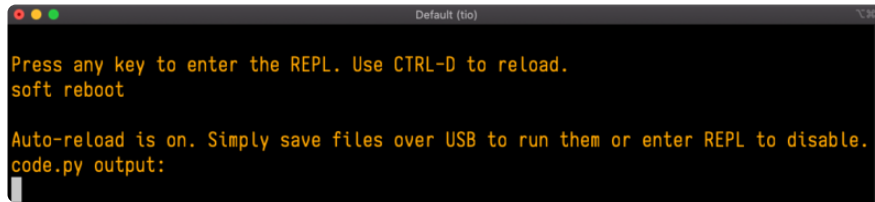
Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.
```

You have an `ImportError`. It says there is `no module named 'simpleio'`. That's the one you just included in your code!

Click the link above to download the correct bundle. Extract the `lib` folder from the downloaded bundle file. Scroll down to find `simpleio.mpy`. This is the library file you're looking for! Follow the steps above to load an individual library file.

The LED starts blinking again! Let's check the serial console.



```
Default (tio)
Press any key to enter the REPL. Use CTRL-D to reload.
soft reboot

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
```

No errors! Excellent. You've successfully resolved an `ImportError`!

If you run into this error in the future, follow along with the steps above and choose the library that matches the one you're missing.

Library Install on Non-Express Boards

If you have an M0 non-Express board such as Trinket M0, Gemma M0, QT Py M0, or one of the M0 Trinkeys, you'll want to follow the same steps in the example above to install libraries as you need them. Remember, you don't need to wait for an `ImportError` if you know what library you added to your code. Open the library bundle you downloaded, find the library you need, and drag it to the `lib` folder on your **CIRCUITPY** drive.

You can still end up running out of space on your M0 non-Express board even if you only load libraries as you need them. There are a number of steps you can use to try to resolve this issue. You'll find suggestions on the [Troubleshooting page \(https://adafru.it/Den\)](https://adafru.it/Den).

Updating CircuitPython Libraries and Examples

Libraries and examples are updated from time to time, and it's important to update the files you have on your **CIRCUITPY** drive.

To update a single library or example, follow the same steps above. When you drag the library file to your `lib` folder, it will ask if you want to replace it. Say yes. That's it!

A new library bundle is released every time there's an update to a library. Updates include things like bug fixes and new features. It's important to check in every so often to see if the libraries you're using have been updated.

CircUp CLI Tool

There is a command line interface (CLI) utility called [CircUp \(https://adafru.it/Tfi\)](https://adafru.it/Tfi) that can be used to easily install and update libraries on your device. Follow the directions on the [install page within the CircUp learn guide \(https://adafru.it/-Ad\)](https://adafru.it/-Ad). Once you've got it installed you run the command `circup update` in a terminal to interactively

update all libraries on the connected CircuitPython device. See the [usage page in the CircUp guide](https://adafru.it/-Ah) (<https://adafru.it/-Ah>) for a full list of functionality

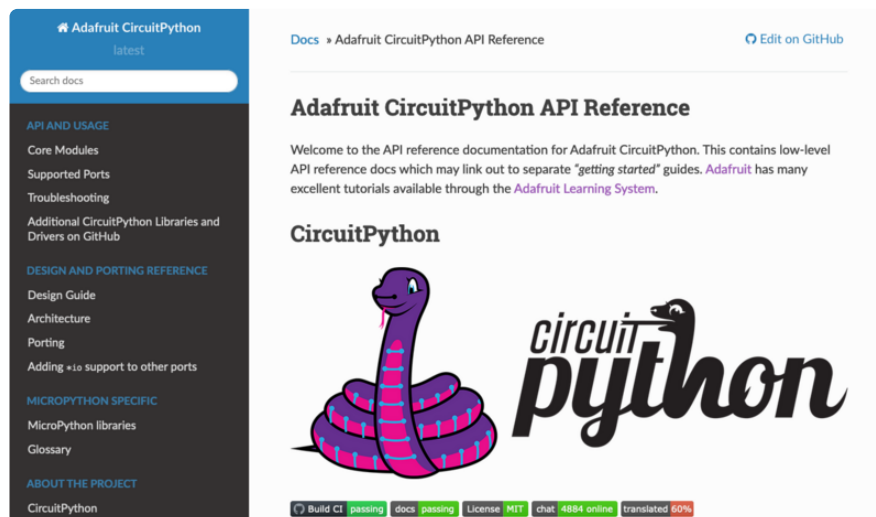
CircuitPython Documentation

You've learned about the CircuitPython built-in modules and external libraries. You know that you can find the modules in CircuitPython, and the libraries in the Library Bundles. There are guides available that explain the basics of many of the modules and libraries. However, there's sometimes more capabilities than are necessarily showcased in the guides, and often more to learn about a module or library. So, where can you find more detailed information? That's when you want to look at the API documentation.

The entire CircuitPython project comes with extensive documentation available on Read the Docs. This includes both the [CircuitPython core](https://adafru.it/Beg) (<https://adafru.it/Beg>) and the [Adafruit CircuitPython libraries](https://adafru.it/Tra) (<https://adafru.it/Tra>).

CircuitPython Core Documentation

The [CircuitPython core documentation](https://adafru.it/Beg) (<https://adafru.it/Beg>) covers many of the details you might want to know about the CircuitPython core and related topics. It includes API and usage info, a design guide and information about porting CircuitPython to new boards, MicroPython info with relation to CircuitPython, and general information about the project.



The main page covers the basics including where to **download CircuitPython**, how to **contribute**, **differences from MicroPython**, information about the **project structure**, and a **full table of contents** for the rest of the documentation.

The list along the left side leads to more information about specific topics.

The first section is **API and Usage**. This is where you can find information about how to use individual built-in **core modules**, such as `time` and `digitalio`, details about the **supported ports**, suggestions for **troubleshooting**, and basic info and links to the **library bundles**. The **Core Modules** section also includes the **Support Matrix**, which is a table of which core modules are available on which boards.

The second section is **Design and Porting Reference**. It includes a **design guide**, **architecture** information, details on **porting**, and **adding module support** to other ports.

The third section is **MicroPython Specific**. It includes information on **MicroPython and related libraries**, and a **glossary** of terms.

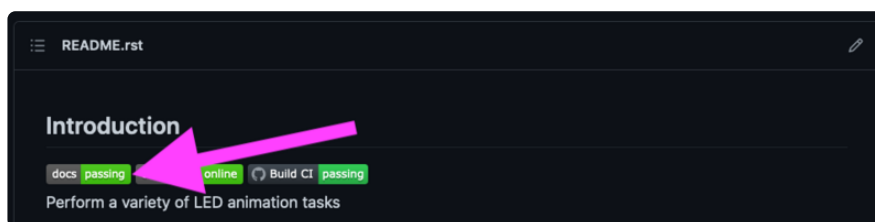
The fourth and final section is **About the Project**. It includes further information including details on **building, testing, and debugging CircuitPython**, along with various other useful links including the **Adafruit Community Code of Conduct**.

Whether you're a seasoned pro or new to electronics and programming, you'll find a wealth of information to help you along your CircuitPython journey in the documentation!

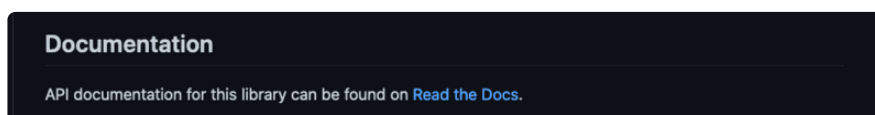
CircuitPython Library Documentation

The Adafruit CircuitPython libraries are documented in a very similar fashion. Each library has its own page on Read the Docs. There is a comprehensive list available [here](https://adafru.it/Tra) (<https://adafru.it/Tra>). Otherwise, to view the documentation for a specific library, you can visit the GitHub repository for the library, and find the link in the README.

For the purposes of this page, the [LED Animation library](https://adafru.it/O2d) (<https://adafru.it/O2d>) documentation will be featured. There are two links to the documentation in each library GitHub repo. The first one is the **docs badge** near the top of the README.



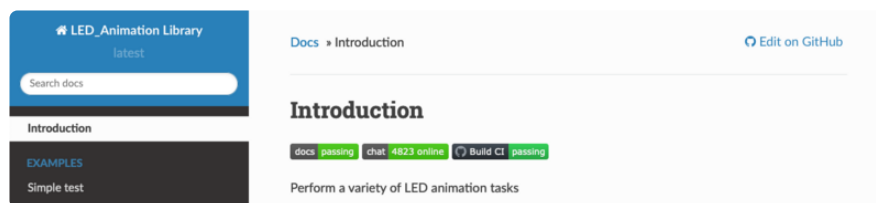
The second place is the **Documentation** section of the README. Scroll down to find it, and click on Read the Docs to get to the documentation.



Now that you know how to find it, it's time to take a look at what to expect.

Not all library documentation will look exactly the same, but this will give you some idea of what to expect from library docs.

The **Introduction** page is generated from the README, so it includes all the same info, such as PyPI installation instructions, a quick demo, and some build details. It also includes a full table of contents for the rest of the documentation (which is not part of the GitHub README). The page should look something like the following.



The left side contains links to the rest of the documentation, divided into three separate sections: **Examples**, **API Reference**, and **Other Links**.

Examples

The [Examples section \(https://adafru.it/VFD\)](https://adafru.it/VFD) is a list of library examples. This list contains anywhere from a small selection to the full list of the examples available for the library.

This section will always contain at least one example - the **simple test** example.



The simple test example is usually a basic example designed to show your setup is working. It may require other libraries to run. Keep in mind, it's simple - it won't showcase a comprehensive use of all the library features.

The LED Animation simple test demonstrates the Blink animation.

Simple test

Ensure your device works with this simple test.

`examples/led_animation_simpletest.py`

```
1 # SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
2 # SPDX-License-Identifier: MIT
3
4 """
5 This simpletest example displays the Blink animation.
6
7 For NeoPixel FeatherWing. Update pixel_pin and pixel_num to match your wiring if using
8 a different form of NeoPixels.
9 """
10 import board
11 import neopixel
12 from adafruit_led_animation.animation.blink import Blink
13 from adafruit_led_animation.color import RED
14
15 # Update to match the pin connected to your NeoPixels
16 pixel_pin = board.D6
17 # Update to match the number of NeoPixels you have connected
18 pixel_num = 32
19
20 pixels = neopixel.NeoPixel(pixel_pin, pixel_num, brightness=0.5, auto_write=False)
21
22 blink = Blink(pixels, speed=0.5, color=RED)
23
24 while True:
25     blink.animate()
```

In some cases, you'll find a longer list, that may include examples that explore other features in the library. The LED Animation documentation includes a series of examples, all of which are available in the library. These examples include demonstrations of both basic and more complex features. Simply click on the example that interests you to view the associated code.

EXAMPLES

Simple test

Basic Animations

All Animations

Pixel Map

Animation Sequence

Animation Group

Blink

Basic Animations

Demonstrates the basic animations.

`examples/led_animation_basic_animations.py`

```
1 # SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
2 # SPDX-License-Identifier: MIT
3
4 """
5 This example displays the basic animations in sequence, at a five second interval.
6 """
```

When there are multiple links in the Examples section, all of the example content is, in actuality, on the same page. Each link after the first is an anchor link to the specified section of the page. Therefore, you can also view all the available examples by scrolling down the page.

You can view the rest of the examples by clicking through the list or scrolling down the page. These examples are fully working code. Which is to say, while they may rely on other libraries as well as the library for which you are viewing the documentation, they should not require modification to otherwise work.

API Reference

The [API Reference section \(https://adafru.it/Rqa\)](https://adafru.it/Rqa) includes a list of the library functions and classes. The API (Application Programming Interface) of a library is the set of functions and classes the library provides. Essentially, the API defines how your program interfaces with the functions and classes that you call in your code to use the library.

There is always at least one list item included. Libraries for which the code is included in a single Python (.py) file, will only have one item. Libraries for which the code is multiple Python files in a directory (called subpackages) will have multiple items in this list. The LED Animation library has a series of subpackages, and therefore, multiple items in this list.

Click on the first item in the list to begin viewing the API Reference section.



As with the Examples section, all of the API Reference content is on a single page, and the links under API Reference are anchor links to the specified section of the page.

When you click on an item in the API Reference section, you'll find details about the classes and functions in the library. In the case of only one item in this section, all the available functionality of the library will be contained within that first and only subsection. However, in the case of a library that has subpackages, each item will contain the features of the particular subpackage indicated by the link. The documentation will cover all of the available functions of the library, including more complex ones that may not interest you.

The first list item is the animation subpackage. If you scroll down, you'll begin to see the available features of animation. They are listed alphabetically. Each of these things can be called in your code. It includes the name and a description of the specific function you would call, and if any parameters are necessary, lists those with a description as well.

```
class adafruit_led_animation.Animation(pixel_object, speed, color, peers=None, paused=False, name=None)
```

Base class for animations.

`add_cycle_complete_receiver(callback)`

Adds an additional callback when the cycle completes.

Parameters

callback – Additional callback to trigger when a cycle completes. The callback is passed the animation object instance.

`after_draw()`

Animation subclasses may implement `after_draw()` to do operations after the main `draw()` is called.

You can view the other subpackages by clicking the link on the left or scrolling down the page. You may be interested in something a little more practical. Here is an example. To use the LED Animation library Comet animation, you would run the following example.

```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT

"""
This example animates a jade comet that bounces from end to end of the strip.

For QT Py Haxpress and a NeoPixel strip. Update pixel_pin and pixel_num to match
your wiring if
using a different board or form of NeoPixels.

This example will run on SAMD21 (M0) Express boards (such as Circuit Playground
Express or QT Py
Haxpress), but not on SAMD21 non-Express boards (such as QT Py or Trinket).
"""
import board
import neopixel

from adafruit_led_animation.animation.comet import Comet
from adafruit_led_animation.color import JADE

# Update to match the pin connected to your NeoPixels
pixel_pin = board.A3
# Update to match the number of NeoPixels you have connected
pixel_num = 30

pixels = neopixel.NeoPixel(pixel_pin, pixel_num, brightness=0.5, auto_write=False)
comet = Comet(pixels, speed=0.02, color=JADE, tail_length=10, bounce=True)

while True:
    comet.animate()
```

Note the line where you create the `comet` object. There are a number of items inside the parentheses. In this case, you're provided with a fully working example. But what if you want to change how the comet works? The code alone does not explain what the options mean.

So, in the API Reference documentation list, click the `adafruit_led_animation.animation.comet` link and scroll down a bit until you see the following.

```
class adafruit_led_animation.animation.comet.Comet(pixel_object, speed, color, tail_length=0, reverse=False, bounce=False, name=None, ring=False)
```

A comet animation.

Parameters

- pixel_object** – The initialised LED object.
- speed** (*float*) – Animation speed in seconds, e.g. `0.1`.
- color** – Animation color in `(r, g, b)` tuple, or `0x000000` hex format.
- tail_length** (*int*) – The length of the comet. Defaults to 25% of the length of the `pixel_object`. Automatically compensates for a minimum of 2 and a maximum of the length of the `pixel_object`.
- reverse** (*bool*) – Animates the comet in the reverse order. Defaults to `False`.
- bounce** (*bool*) – Comet will bounce back and forth. Defaults to `True`.
- ring** (*bool*) – Ring mode. Defaults to `False`.

Look familiar? It is! This is the documentation for setting up the comet object. It explains what each argument provided in the comet setup in the code meant, as well as the other available features. For example, the code includes `speed=0.02`. The documentation clarifies that this is the "Animation speed in seconds". The code doesn't include `ring`. The documentation indicates this is an available setting that enables "Ring mode".

This type of information is available for any function you would set up in your code. If you need clarification on something, wonder whether there's more options available, or are simply interested in the details involved in the code you're writing, check out the documentation for the CircuitPython libraries!

Other Links

This section is the same for every library. It includes a list of links to external sites, which you can visit for more information about the CircuitPython Project and Adafruit.

That covers the CircuitPython library documentation! When you are ready to go beyond the basic library features covered in a guide, or you're interested in understanding those features better, the library documentation on Read the Docs has you covered!

Recommended Editors

The CircuitPython code on your board detects when the files are changed or written and will automatically re-start your code. This makes coding very fast because you save, and it re-runs.

However, you must wait until the file is done being saved before unplugging or resetting your board! On Windows using some editors this can sometimes take up to 90 seconds, on Linux it can take 30 seconds to complete because the text editor does not save the file completely. Mac OS does not seem to have this delay, which is nice!

This is really important to be aware of. If you unplug or reset the board before your computer finishes writing the file to your board, you can corrupt the drive. If this happens, you may lose the code you've written, so it's important to backup your code to your computer regularly.

To avoid the likelihood of filesystem corruption, use an editor that writes out the file completely when you save it. Check out the list of recommended editors below.

Recommended editors

- [mu](https://adafru.it/ANO) (<https://adafru.it/ANO>) is an editor that safely writes all changes (it's also our recommended editor!)
- [emacs](https://adafru.it/xNA) (<https://adafru.it/xNA>) is also an editor that will [fully write files on save](https://adafru.it/Be7) (<https://adafru.it/Be7>)
- [Sublime Text](https://adafru.it/xNB) (<https://adafru.it/xNB>) safely writes all changes
- [Visual Studio Code](https://adafru.it/Be9) (<https://adafru.it/Be9>) appears to safely write all changes
- [gedit](#) on Linux appears to safely write all changes
- [IDLE](https://adafru.it/IWB) (<https://adafru.it/IWB>), in Python 3.8.1 or later, [was fixed](https://adafru.it/IWD) (<https://adafru.it/IWD>) to write all changes immediately
- [Thonny](https://adafru.it/Qb6) (<https://adafru.it/Qb6>) fully writes files on save
- [Notepad++](https://adafru.it/xNf) (<https://adafru.it/xNf>) flushes files after writes, as of several years ago. In addition, you can change the path used for "Enable session snapshot and periodic backup" to write somewhere else than the CIRCUITPY drive. This will save space on CIRCUITPY and reduce writes to the drive.

Recommended only with particular settings or add-ons

- [vim](https://adafru.it/ek9) (<https://adafru.it/ek9>) / [vi](#) safely writes all changes. But set up [vim](#) to not write [swapfiles](https://adafru.it/ELO) (<https://adafru.it/ELO>) (.swp files: temporary records of your edits) to CIRCUITPY. Run vim with `vim -n`, set the `no swapfile` option, or set the `directory` option to write swapfiles elsewhere. Otherwise the swapfile writes trigger restarts of your program.
- The [PyCharm IDE](https://adafru.it/xNC) (<https://adafru.it/xNC>) is safe if "Safe Write" is turned on in Settings->System Settings->Synchronization (true by default).
- If you are using [Atom](https://adafru.it/fMG) (<https://adafru.it/fMG>), install the [fsync-on-save package](https://adafru.it/E9m) (<https://adafru.it/E9m>) or the [language-circuitpython package](https://adafru.it/Vuf) (<https://adafru.it/Vuf>) so that it will always write out all changes to files on CIRCUITPY.

- **SlickEdit** (<https://adafru.it/DdP>) works only if you [add a macro to flush the disk](https://adafru.it/ven) (<https://adafru.it/ven>).

The editors listed below are specifically NOT recommended!

Editors that are NOT recommended

- **notepad** (the default Windows editor) can be slow to write, so the editors above are recommended! If you are using notepad, be sure to eject the drive.
- **IDLE** in Python 3.8.0 or earlier does not force out changes immediately. Later versions do force out changes.
- **nano** (on Linux) does not force out changes.
- **geany** (on Linux) does not force out changes.
- **Anything else** - Other editors have not been tested so please use a recommended one!

Advanced Serial Console on Windows Windows 7 and 8.1

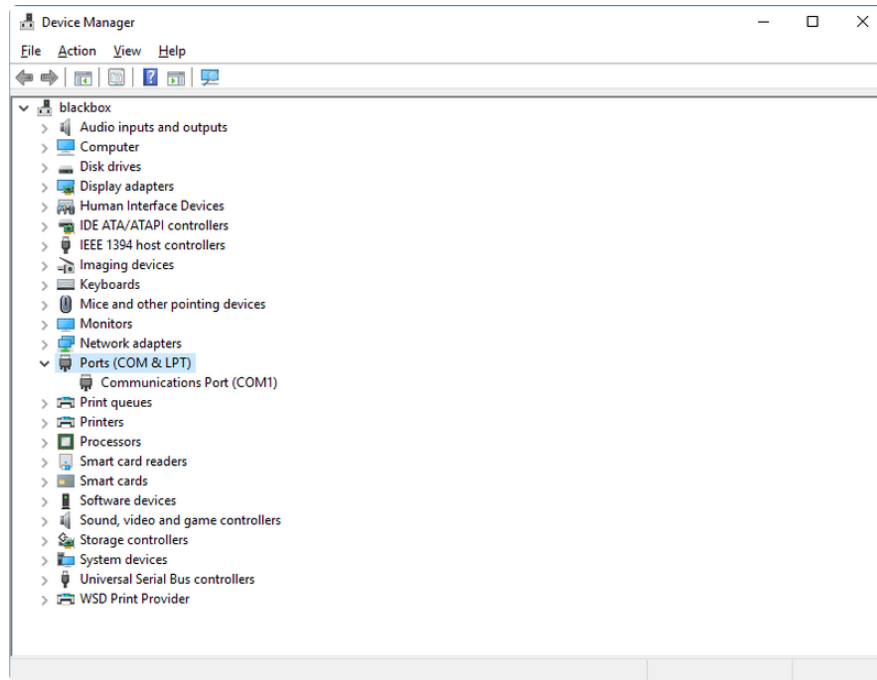
If you're using Windows 7 (or 8 or 8.1), you'll need to install drivers. See the [Windows 7 and 8.1 Drivers page](https://adafru.it/VuB) (<https://adafru.it/VuB>) for details. You will not need to install drivers on Mac, Linux or Windows 10.

You are strongly encouraged to upgrade to Windows 10 if you are still using Windows 7 or Windows 8 or 8.1. Windows 7 has reached end-of-life and no longer receives security updates. A free upgrade to Windows 10 is [still available](https://adafru.it/RWc) (<https://adafru.it/RWc>).

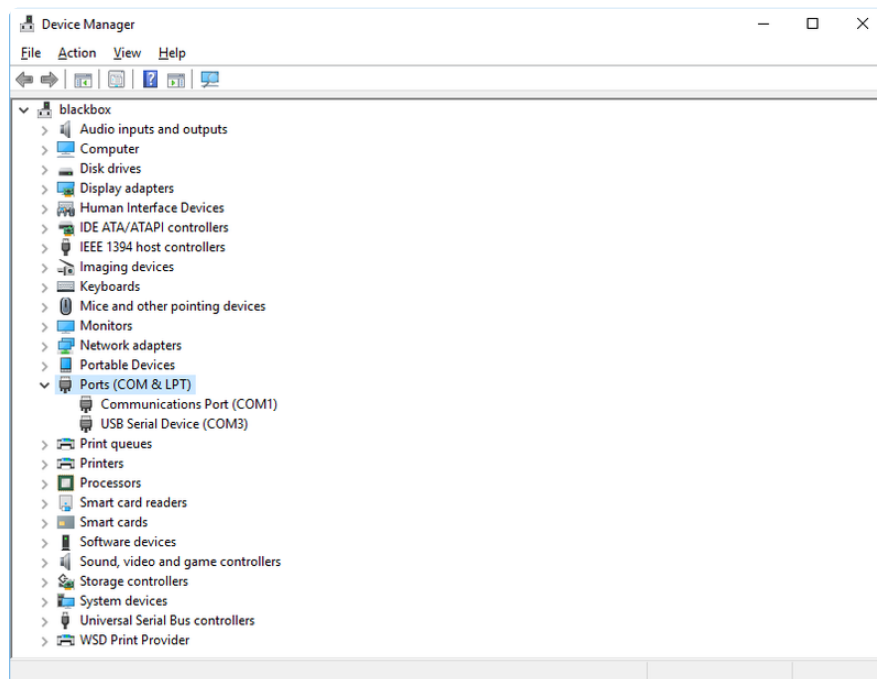
What's the COM?

First, you'll want to find out which serial port your board is using. When you plug your board in to USB on your computer, it connects to a serial port. The port is like a door through which your board can communicate with your computer using USB.

You'll use Windows Device Manager to determine which port the board is using. The easiest way to determine which port the board is using is to first check **without** the board plugged in. Open Device Manager. Click on Ports (COM & LPT). You should find something already in that list with (COM#) after it where # is a number.



Now plug in your board. The Device Manager list will refresh and a new item will appear under Ports (COM & LPT). You'll find a different (COM#) after this item in the list.



Sometimes the item will refer to the name of the board. Other times it may be called something like USB Serial Device, as seen in the image above. Either way, there is a new (COM#) following the name. This is the port your board is using.

Install Putty

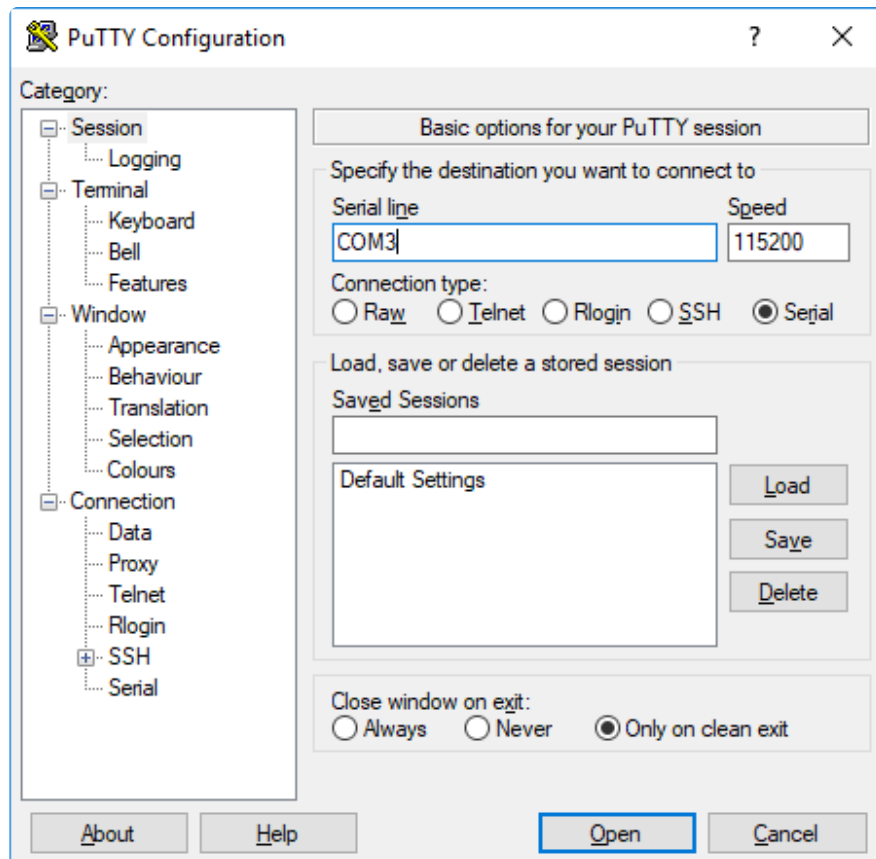
If you're using Windows, you'll need to download a terminal program. You're going to use PuTTY.

The first thing to do is download the [latest version of PuTTY \(https://adafru.it/Bf1\)](https://adafru.it/Bf1). You'll want to download the Windows installer file. It is most likely that you'll need the 64-bit version. Download the file and install the program on your machine. If you run into issues, you can try downloading the 32-bit version instead. However, the 64-bit version will work on most PCs.

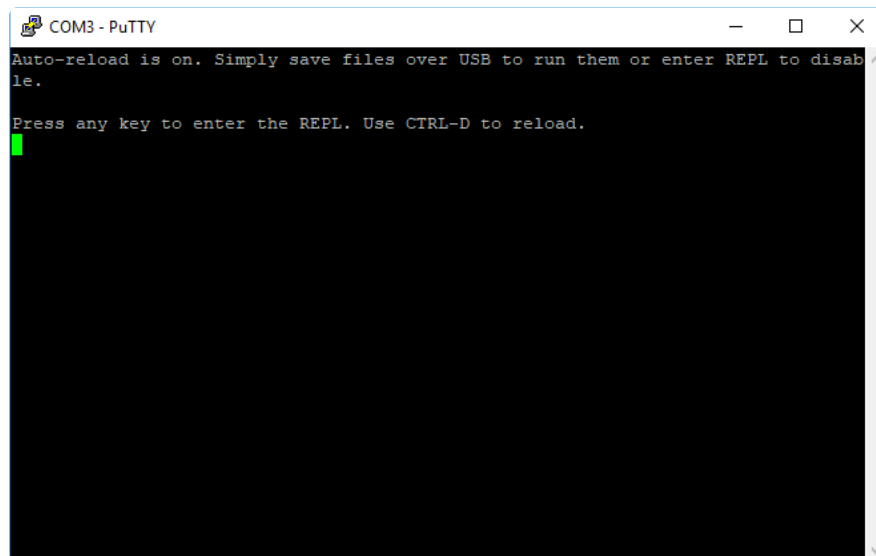
Now you need to open PuTTY.

- Under **Connection type**: choose the button next to **Serial**.
- In the box under **Serial line**, enter the serial port you found that your board is using.
- In the box under **Speed**, enter 115200. This called the baud rate, which is the speed in bits per second that data is sent over the serial connection. For boards with built in USB it doesn't matter so much but for ESP8266 and other board with a separate chip, the speed required by the board is 115200 bits per second. So you might as well just use 115200!

If you want to save those settings for later, use the options under **Load, save or delete a stored session**. Enter a name in the box under **Saved Sessions**, and click the **Save** button on the right.



Once your settings are entered, you're ready to connect to the serial console. Click "Open" at the bottom of the window. A new window will open.



If no code is running, the window will either be blank or will look like the window above. Now you're ready to see the results of your code.

Great job! You've connected to the serial console!

Advanced Serial Console on Mac

Connecting to the serial console on Mac does not require installing any drivers or extra software. You'll use a terminal program to find your board, and `screen` to connect to it. Terminal and `screen` both come installed by default.

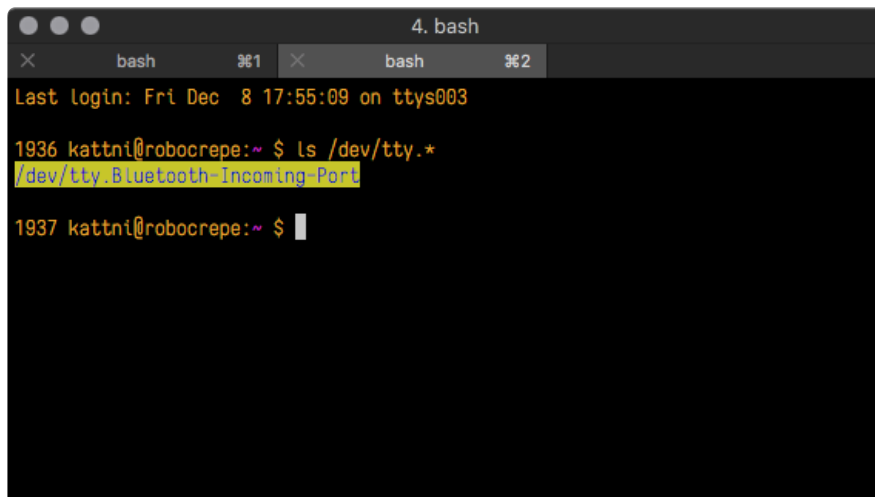
What's the Port?

First you'll want to find out which serial port your board is using. When you plug your board in to USB on your computer, it connects to a serial port. The port is like a door through which your board can communicate with your computer using USB.

The easiest way to determine which port the board is using is to first check **without** the board plugged in. Open Terminal and type the following:

```
ls /dev/tty.*
```

Each serial connection shows up in the `/dev/` directory. It has a name that starts with `tty.`. The command `ls` shows you a list of items in a directory. You can use `*` as a wildcard, to search for files that start with the same letters but end in something different. In this case, you're asking to see all of the listings in `/dev/` that start with `tty.` and end in anything. This will show us the current serial connections.

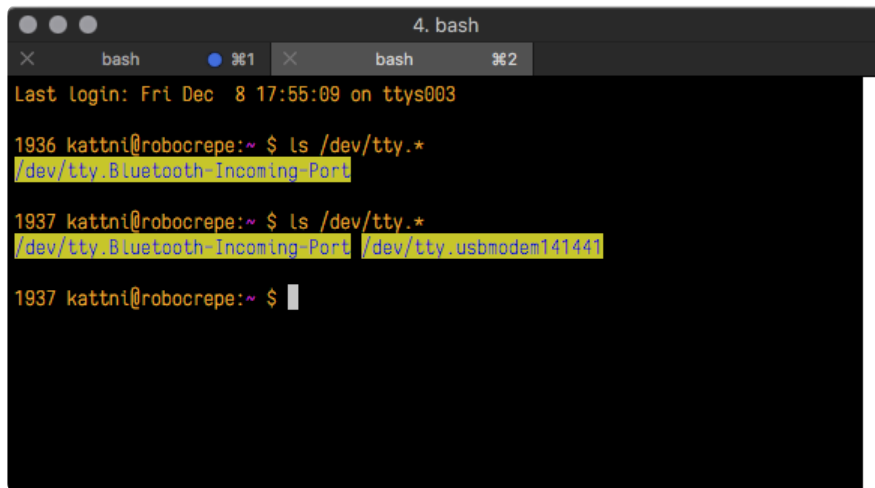


```
4. bash
bash  %1  bash  %2
Last login: Fri Dec 8 17:55:09 on ttys003
1936 kattni@robocrepe:~ $ ls /dev/tty.*
/dev/tty.Bluetooth-Incoming-Port
1937 kattni@robocrepe:~ $
```

Now, plug your board. In Terminal, type:

```
ls /dev/tty.*
```

This will show you the current serial connections, which will now include your board.

A screenshot of a macOS Terminal window titled "4. bash". It shows two tabs, both labeled "bash". The first tab shows the command `ls /dev/tty.*` being executed, resulting in the output `/dev/tty.Bluetooth-Incoming-Port`. The second tab shows the same command being executed again, resulting in the output `/dev/tty.Bluetooth-Incoming-Port /dev/tty.usbmodem141441`. The prompt is `kattni@robocrepe:~`.

```
4. bash
bash
Last login: Fri Dec 8 17:55:09 on ttys003

1936 kattni@robocrepe:~ $ ls /dev/tty.*
/dev/tty.Bluetooth-Incoming-Port

1937 kattni@robocrepe:~ $ ls /dev/tty.*
/dev/tty.Bluetooth-Incoming-Port /dev/tty.usbmodem141441

1937 kattni@robocrepe:~ $
```

A new listing has appeared called `/dev/tty.usbmodem141441`. The `tty.usbmodem141441` part of this listing is the name the example board is using. Yours will be called something similar.

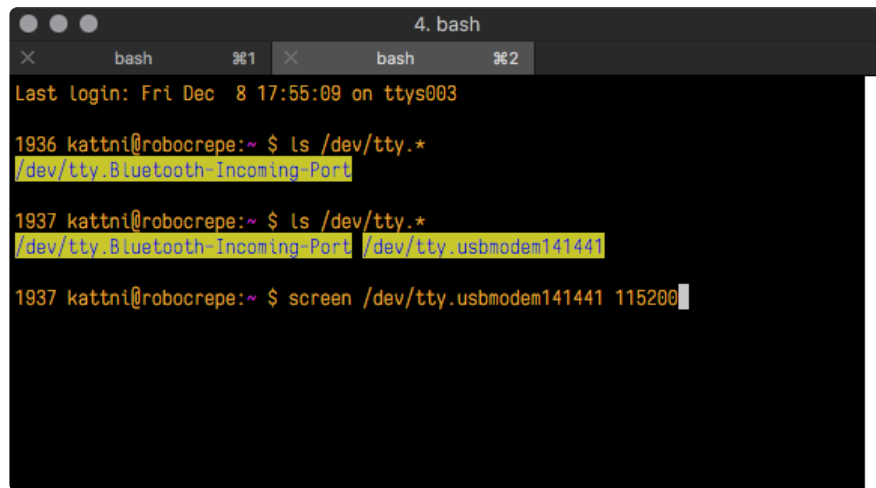
Using Linux, a new listing has appeared called `/dev/ttyACM0`. The `ttyACM0` part of this listing is the name the example board is using. Yours will be called something similar.

Connect with screen

Now that you know the name your board is using, you're ready connect to the serial console. You're going to use a command called `screen`. The `screen` command is included with MacOS. To connect to the serial console, use Terminal. Type the following command, replacing `board_name` with the name you found your board is using:

```
screen /dev/tty.board_name 115200
```

The first part of this establishes using the `screen` command. The second part tells screen the name of the board you're trying to use. The third part tells screen what baud rate to use for the serial connection. The baud rate is the speed in bits per second that data is sent over the serial connection. In this case, the speed required by the board is 115200 bits per second.



```
4. bash
bash %1 bash %2
Last login: Fri Dec 8 17:55:09 on ttys003
1936 kattni@robocrepe:~ $ ls /dev/tty.*
/dev/tty.Bluetooth-Incoming-Port
1937 kattni@robocrepe:~ $ ls /dev/tty.*
/dev/tty.Bluetooth-Incoming-Port /dev/tty.usbmodem141441
1937 kattni@robocrepe:~ $ screen /dev/tty.usbmodem141441 115200
```

Press enter to run the command. It will open in the same window. If no code is running, the window will be blank. Otherwise, you'll see the output of your code.

Great job! You've connected to the serial console!

Advanced Serial Console on Linux

Connecting to the serial console on Linux does not require installing any drivers, but you may need to install `screen` using your package manager. You'll use a terminal program to find your board, and `screen` to connect to it. There are a variety of terminal programs such as `gnome-terminal` (called Terminal) or Konsole on KDE.

The `tio` program works as well to connect to your board, and has the benefit of automatically reconnecting. You would need to install it using your package manager.

What's the Port?

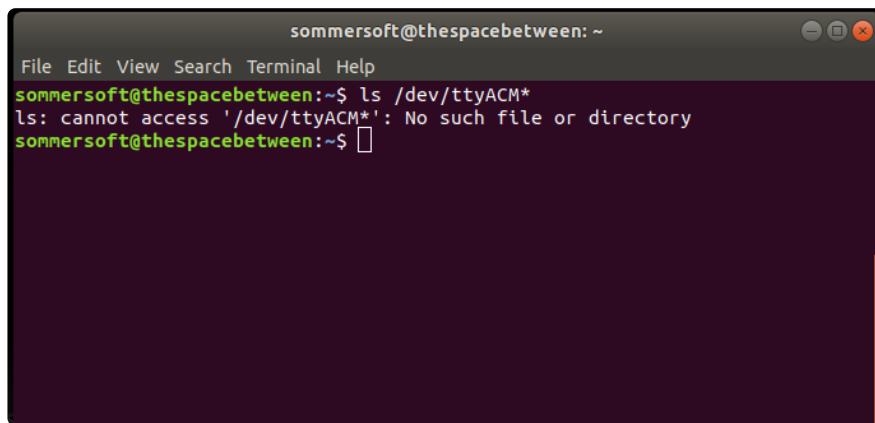
First you'll want to find out which serial port your board is using. When you plug your board in to USB on your computer, it connects to a serial port. The port is like a door through which your board can communicate with your computer using USB.

The easiest way to determine which port the board is using is to first check **without** the board plugged in. Open your terminal program and type the following:

```
ls /dev/ttyACM*
```

Each serial connection shows up in the `/dev/` directory. It has a name that starts with `ttyACM`. The command `ls` shows you a list of items in a directory. You can use `*` as a wildcard, to search for files that start with the same letters but end in something different. In this case, You're asking to see all of the listings in `/dev/` that start with `ttyACM` and end in anything. This will show us the current serial connections.

In the example below, the error is indicating that there are no current serial connections starting with **ttyACM**.

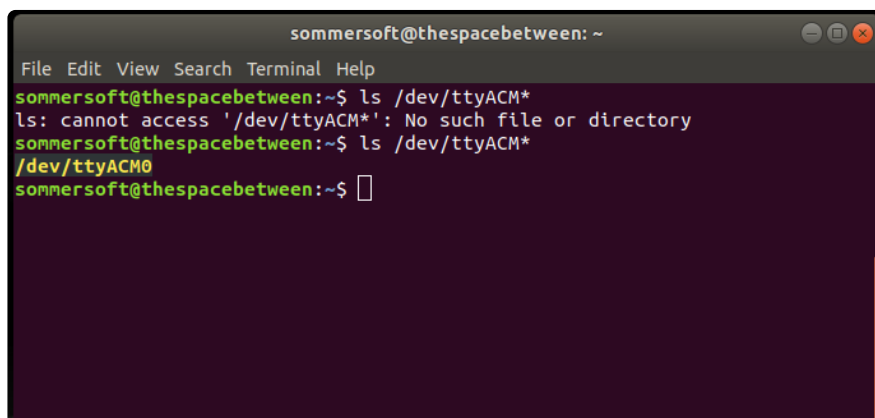
A terminal window titled 'sommersoft@thespacebetween: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows the command 'ls /dev/ttyACM*' being executed, which results in the error message 'ls: cannot access '/dev/ttyACM*': No such file or directory'. The prompt 'sommersoft@thespacebetween:~\$' is shown again.

```
sommersoft@thespacebetween: ~  
File Edit View Search Terminal Help  
sommersoft@thespacebetween:~$ ls /dev/ttyACM*  
ls: cannot access '/dev/ttyACM*': No such file or directory  
sommersoft@thespacebetween:~$
```

Now plug in your board. In your terminal program, type:

```
ls /dev/ttyACM*
```

This will show you the current serial connections, which will now include your board.

A terminal window titled 'sommersoft@thespacebetween: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows the command 'ls /dev/ttyACM*' being executed, which results in the error message 'ls: cannot access '/dev/ttyACM*': No such file or directory'. The command is then repeated, and the output shows '/dev/ttyACM0'. The prompt 'sommersoft@thespacebetween:~\$' is shown again.

```
sommersoft@thespacebetween: ~  
File Edit View Search Terminal Help  
sommersoft@thespacebetween:~$ ls /dev/ttyACM*  
ls: cannot access '/dev/ttyACM*': No such file or directory  
sommersoft@thespacebetween:~$ ls /dev/ttyACM*  
/dev/ttyACM0  
sommersoft@thespacebetween:~$
```

A new listing has appeared called **/dev/ttyACM0**. The **ttyACM0** part of this listing is the name the example board is using. Yours will be called something similar.

Connect with screen

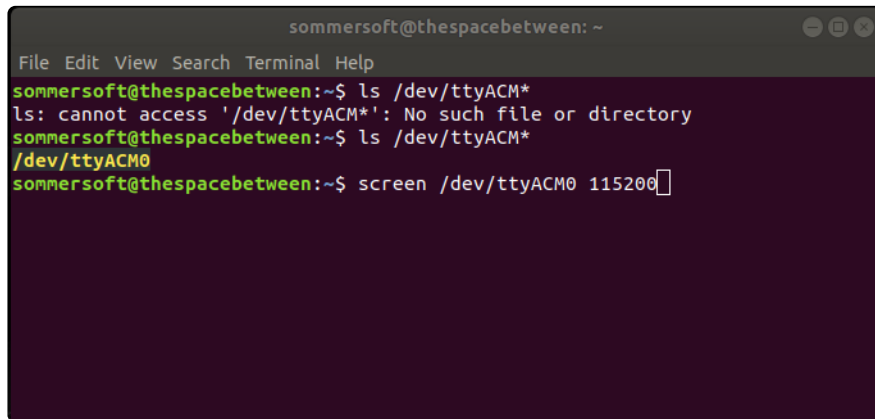
Now that you know the name your board is using, you're ready to connect to the serial console. You'll use a command called **screen**. You may need to install it using the package manager.

To connect to the serial console, use your terminal program. Type the following command, replacing **board_name** with the name you found your board is using:

```
screen /dev/tty.board_name 115200
```

The first part of this establishes using the **screen** command. The second part tells screen the name of the board you're trying to use. The third part tells screen what

baud rate to use for the serial connection. The baud rate is the speed in bits per second that data is sent over the serial connection. In this case, the speed required by the board is 115200 bits per second.



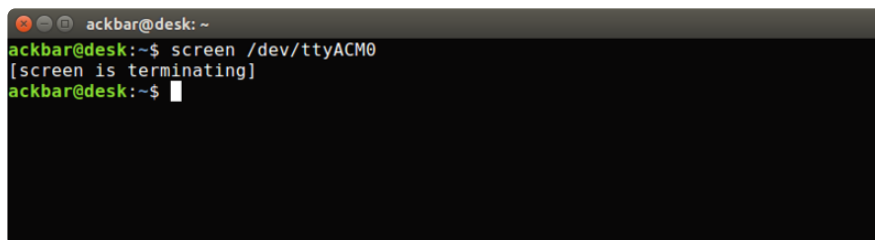
```
sommersoft@thespacebetween: ~  
File Edit View Search Terminal Help  
sommersoft@thespacebetween:~$ ls /dev/ttyACM*  
ls: cannot access '/dev/ttyACM*': No such file or directory  
sommersoft@thespacebetween:~$ ls /dev/ttyACM*  
/dev/ttyACM0  
sommersoft@thespacebetween:~$ screen /dev/ttyACM0 115200
```

Press enter to run the command. It will open in the same window. If no code is running, the window will be blank. Otherwise, you'll see the output of your code.

Great job! You've connected to the serial console!

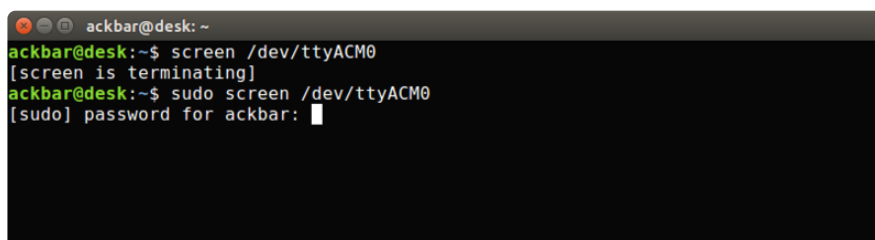
Permissions on Linux

If you try to run `screen` and it doesn't work, then you may be running into an issue with permissions. Linux keeps track of users and groups and what they are allowed to do and not do, like access the hardware associated with the serial connection for running `screen`. So if you see something like this:



```
ackbar@desk: ~  
ackbar@desk:~$ screen /dev/ttyACM0  
[screen is terminating]  
ackbar@desk:~$
```

then you may need to grant yourself access. There are generally two ways you can do this. The first is to just run `screen` using the `sudo` command, which temporarily gives you elevated privileges.



```
ackbar@desk: ~  
ackbar@desk:~$ screen /dev/ttyACM0  
[screen is terminating]  
ackbar@desk:~$ sudo screen /dev/ttyACM0  
[sudo] password for ackbar:   
[screen is terminating]
```

Once you enter your password, you should be in:

```
ackbar@desk: ~  
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.  
  
Press any key to enter the REPL. Use CTRL-D to reload.  
  
Adafruit CircuitPython 2.1.0 on 2017-10-17; Adafruit Trinket M0 with samd21e18  
>>> |
```

The second way is to add yourself to the group associated with the hardware. To figure out what that group is, use the command `ls -l` as shown below. The group name is circled in red.

Then use the command `adduser` to add yourself to that group. You need elevated privileges to do this, so you'll need to use `sudo`. In the example below, the group is `adm` and the user is `ackbar`.

```
ackbar@desk: ~  
ackbar@desk:~$ ls -l /dev/ttyACM0  
crw-rw---- 1 root adm 166, 0 Dec 21 08:29 /dev/ttyACM0  
ackbar@desk:~$ sudo adduser ackbar adm  
Adding user `ackbar' to group `adm' ...  
Adding user ackbar to group adm  
Done.  
ackbar@desk:~$ |
```

After you add yourself to the group, you'll need to logout and log back in, or in some cases, reboot your machine. After you log in again, verify that you have been added to the group using the command `groups`. If you are still not in the group, reboot and check again.

```
ackbar@desk: ~  
ackbar@desk:~$ groups  
ackbar adm sudo  
ackbar@desk:~$ |
```

And now you should be able to run `screen` without using `sudo`.

```
ackbar@desk: ~  
ackbar@desk:~$ groups  
ackbar adm sudo  
ackbar@desk:~$ screen /dev/ttyACM0 115200 |
```

And you're in:


```
ackbar@desk: ~  
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.  
  
Press any key to enter the REPL. Use CTRL-D to reload.  
  
Adafruit CircuitPython 2.1.0 on 2017-10-17; Adafruit Trinket M0 with samd21e18  
>>> 
```

The examples above use `screen`, but you can also use other programs, such as `putty` or `picocom`, if you prefer.

Troubleshooting

From time to time, you will run into issues when working with CircuitPython. Here are a few things you may encounter and how to resolve them.

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. Visit <https://circuitpython.org/downloads> to download the latest version of CircuitPython for your board. You must download the CircuitPython Library Bundle that matches your version of CircuitPython. Please update CircuitPython and then visit <https://circuitpython.org/libraries> to download the latest Library Bundle.

Always Run the Latest Version of CircuitPython and Libraries

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. **You need to [update to the latest CircuitPython](https://adafru.it/Em8)**. (<https://adafru.it/Em8>).

You need to download the CircuitPython Library Bundle that matches your version of CircuitPython. **Please update CircuitPython and then [download the latest bundle](https://adafru.it/ENC)** (<https://adafru.it/ENC>).

As new versions of CircuitPython are released, Adafruit will stop providing the previous bundles as automatically created downloads on the Adafruit CircuitPython Library Bundle repo. If you must continue to use an earlier version, you can still download the appropriate version of `mpy-cross` from the particular release of CircuitPython on the CircuitPython repo and create your own compatible .mpy library files. **However, it is best to update to the latest for both CircuitPython and the library bundle.**

I have to continue using CircuitPython 7.x or earlier.
Where can I find compatible libraries?

Adafruit is no longer building or supporting the CircuitPython 7.x or earlier library bundles. You are highly encouraged to [update CircuitPython to the latest version \(https://adafru.it/Em8\)](https://adafru.it/Em8) and use [the current version of the libraries \(https://adafru.it/ENC\)](https://adafru.it/ENC).

However, if for some reason you cannot update, links to the previous bundles are available in the [FAQ \(https://adafru.it/FwY\)](https://adafru.it/FwY).

macOS Sonoma before 14.4: Errors Writing to CIRCUITPY

macOS 14.4 - 15.1: Slow Writes to CIRCUITPY

macOS Sonoma before 14.4 took many seconds to complete writes to small FAT drives, 8MB or smaller. This causes errors when writing to CIRCUITPY. The best solution was to remount the CIRCUITPY drive after it is automatically mounted. Or consider downgrading back to Ventura if that works for you. This problem was tracked in [CircuitPython GitHub issue 8449 \(https://adafru.it/18ea\)](https://adafru.it/18ea).

Below is a shell script to do this remount conveniently (courtesy [@czei in GitHub \(https://adafru.it/18ea\)](https://adafru.it/18ea)). Copy the code here into a file named, say, **remount-CIRCUITPY.sh**. Place the file in a directory on your PATH, or in some other convenient place.

macOS Sonoma 14.4 and versions of macOS before Sequoia 15.2 did not have the problem above, but did take an inordinately long time to write to FAT drives of size 1GB or less (40 times longer than 2GB drives). As of macOS 15.2, writes are no longer very slow. This problem was tracked in [CircuitPython GitHub issue 8918 \(https://adafru.it/19iD\)](https://adafru.it/19iD).

```
#!/bin/sh
#
# This works around bug where, by default,
# macOS 14.x before 14.4 writes part of a file immediately,
# and then doesn't update the directory for 20-60 seconds, causing
# the file system to be corrupted.
#
disky=`df | grep CIRCUITPY | cut -d" " -f1`
sudo umount /Volumes/CIRCUITPY
sudo mkdir /Volumes/CIRCUITPY
sleep 2
sudo mount -v -o noasync -t msdos $disky /Volumes/CIRCUITPY
```

Then in a Terminal window, do this to make this script executable:

```
chmod +x remount-CIRCUITPY.sh
```

Place the file in a directory on your **PATH**, or in some other convenient place.

Now, each time you plug in or reset your CIRCUITPY board, run the file **remount-CIRCUITPY.sh**. You can run it in a Terminal window or you may be able to place it on the desktop or in your dock to run it just by double-clicking.

This will be something of a nuisance but it is the safest solution.

This problem is being tracked in [this CircuitPython issue \(https://adafru.it/18ea\)](https://adafru.it/18ea).

Bootloader (boardnameBOOT) Drive Not Present

You may have a different board.

Only Adafruit Express boards and the SAMD21 non-Express boards ship with the [UF2 bootloader \(https://adafru.it/zbX\)](https://adafru.it/zbX) installed. The Feather M0 Basic, Feather M0 Adalogger, and similar boards use a regular Arduino-compatible bootloader, which does not show a **boardnameBOOT** drive.

MakeCode

If you are running a [MakeCode \(https://adafru.it/zbY\)](https://adafru.it/zbY) program on Circuit Playground Express, press the reset button just once to get the **CPLAYBOOT** drive to show up. Pressing it twice will not work.

macOS

DriveDx and its accompanying **SAT SMART Driver** can interfere with seeing the BOOT drive. [See this forum post \(https://adafru.it/sTc\)](https://adafru.it/sTc) for how to fix the problem.

Windows 10 or later

Did you install the Adafruit Windows Drivers package by mistake, or did you upgrade to Windows 10 or later with the driver package installed? You don't need to install this package on Windows 10 or 11 for most Adafruit boards. The old version (v1.5) can interfere with recognizing your device. Go to **Settings -> Apps** and uninstall all the "Adafruit" driver programs.

Windows 7 or 8.1

Windows 7 and 8.1 have reached end of life. It is [recommended \(https://adafru.it/Amd\)](https://adafru.it/Amd) that you upgrade to Windows 10 or 11 if possible. Drivers are available for some older CircuitPython boards, but there are no plans to release drivers for newer boards.

The Windows Drivers installer was last updated in November 2020 (v2.5.0.0). Windows 7 drivers for CircuitPython boards released since then, including RP2040 boards, are not available. There are no plans to release drivers for newer boards. The boards work fine on Windows 10 and later.

You should now be done! Test by unplugging and replugging the board. You should see the **CIRCUITPY** drive, and when you double-click the reset button (single click on Circuit Playground Express running MakeCode), you should see the appropriate **boardnameBOOT** drive.

Let us know in the [Adafruit support forums \(https://adafru.it/jlf\)](https://adafru.it/jlf) or on the [Adafruit Discord \(\)](#) if this does not work for you!

Windows Explorer Locks Up When Accessing **boardnameBOOT** Drive

On Windows, several third-party programs that can cause issues. The symptom is that you try to access the **boardnameBOOT** drive, and Windows or Windows Explorer seems to lock up. These programs are known to cause trouble:

- **AIDA64**: to fix, stop the program. This problem has been reported to AIDA64. They acquired hardware to test, and released a beta version that fixes the problem. This may have been incorporated into the latest release. Please let us know in the forums if you test this.
- **Hard Disk Sentinel**
- **Kaspersky anti-virus**: To fix, you may need to disable Kaspersky completely. Disabling some aspects of Kaspersky does not always solve the problem. This problem has been reported to Kaspersky.
- **ESET NOD32 anti-virus**: There have been problems with at least version 9.0.386.0, solved by uninstallation.

Copying UF2 to **boardnameBOOT** Drive Hangs at 0% Copied

On Windows, a **Western Digital (WD) utility** that comes with their external USB drives can interfere with copying UF2 files to the **boardnameBOOT** drive. Uninstall that utility to fix the problem.

CIRCUITPY Drive Does Not Appear or Disappears Quickly

Kaspersky anti-virus can block the appearance of the **CIRCUITPY** drive. There has not yet been settings change discovered that prevents this. Complete uninstallation of Kaspersky fixes the problem.

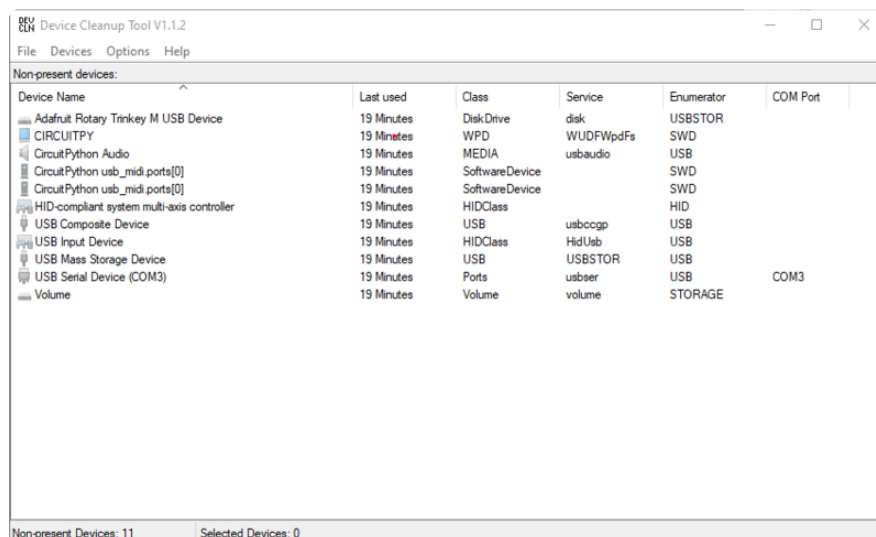
Norton anti-virus can interfere with **CIRCUITPY**. A user has reported this problem on Windows 7. The user turned off both Smart Firewall and Auto Protect, and **CIRCUITPY** then appeared.

Sophos Endpoint security software [can cause CIRCUITPY to disappear \(https://adafru.it/ELr\)](https://adafru.it/ELr) and the BOOT drive to reappear. It is not clear what causes this behavior.

Samsung Magician can cause CIRCUITPY to disappear (reported [here \(https://adafru.it/18eb\)](https://adafru.it/18eb) and [here \(https://adafru.it/18ec\)](https://adafru.it/18ec)).

Device Errors or Problems on Windows

Windows can become confused about USB device installations. Try cleaning up your USB devices. Use [Uwe Sieber's Device Cleanup Tool \(https://adafru.it/RWd\)](https://adafru.it/RWd) (on that page, scroll down to "Device Cleanup Tool"). Download and unzip the tool. Unplug all the boards and other USB devices you want to clean up. Run the tool as Administrator. You will see a listing like this, probably with many more devices. It is listing all the USB devices that are not currently attached.



Select all the devices you want to remove, and then press Delete. It is usually safe just to select everything. Any device that is removed will get a fresh install when you plug it in. Using the Device Cleanup Tool also discards all the COM port assignments for the unplugged boards. If you have used many Arduino and CircuitPython boards,

you have probably seen higher and higher COM port numbers used, seemingly without end. This will fix that problem.

Serial Console in Mu Not Displaying Anything

There are times when the serial console will accurately not display anything, such as, when no code is currently running, or when code with no serial output is already running before you open the console. However, if you find yourself in a situation where you feel it should be displaying something like an error, consider the following.

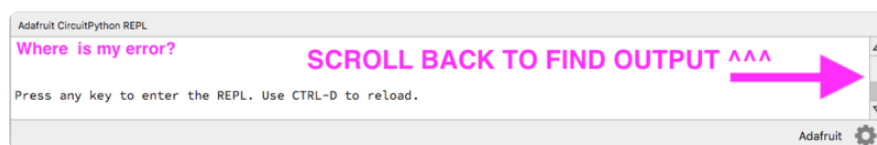
Depending on the size of your screen or Mu window, when you open the serial console, the serial console panel may be very small. This can be a problem. A basic CircuitPython error takes 10 lines to display!

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.  
code.py output:  
Traceback (most recent call last):  
  File "code.py", line 7  
SyntaxError: invalid syntax
```

Press any key to enter the REPL. Use CTRL-D to reload.

More complex errors take even more lines!

Therefore, if your serial console panel is five lines tall or less, you may only see blank lines or blank lines followed by **Press any key to enter the REPL. Use CTRL-D to reload.** . If this is the case, you need to either mouse over the top of the panel to utilise the option to resize the serial panel, or use the scrollbar on the right side to scroll up and find your message.



This applies to any kind of serial output whether it be error messages or print statements. So before you start trying to debug your problem on the hardware side, be sure to check that you haven't simply missed the serial messages due to serial output panel height.

code.py Restarts Constantly

CircuitPython will restart **code.py** if you or your computer writes to something on the CIRCUITPY drive. This feature is called auto-reload, and lets you test a change to your program immediately.

Some utility programs, such as backup, anti-virus, or disk-checking apps, will write to the CIRCUITPY as part of their operation. Sometimes they do this very frequently, causing constant restarts.

Acronis True Image and related Acronis programs on Windows are known to cause this problem. It is possible to prevent this by [disabling the " \(https://adafru.it/XDZ\)Acronis Managed Machine Service Mini" \(https://adafru.it/XDZ\)](https://adafru.it/XDZ).

If you cannot stop whatever is causing the writes, you can disable auto-reload by putting this code in **boot.py** or **code.py**:

```
import supervisor
supervisor.runtime.autoreload = False
```

CircuitPython RGB Status Light

Nearly all CircuitPython-capable boards have a single NeoPixel or DotStar RGB LED on the board that indicates the status of CircuitPython. A few boards designed before CircuitPython existed, such as the Feather M0 Basic, do not.

Circuit Playground Express and Circuit Playground Bluefruit have multiple RGB LEDs, but do NOT have a status LED. The LEDs are all green when in the bootloader. In versions before 7.0.0, they do NOT indicate any status while running CircuitPython.

CircuitPython 7.0.0 and Later

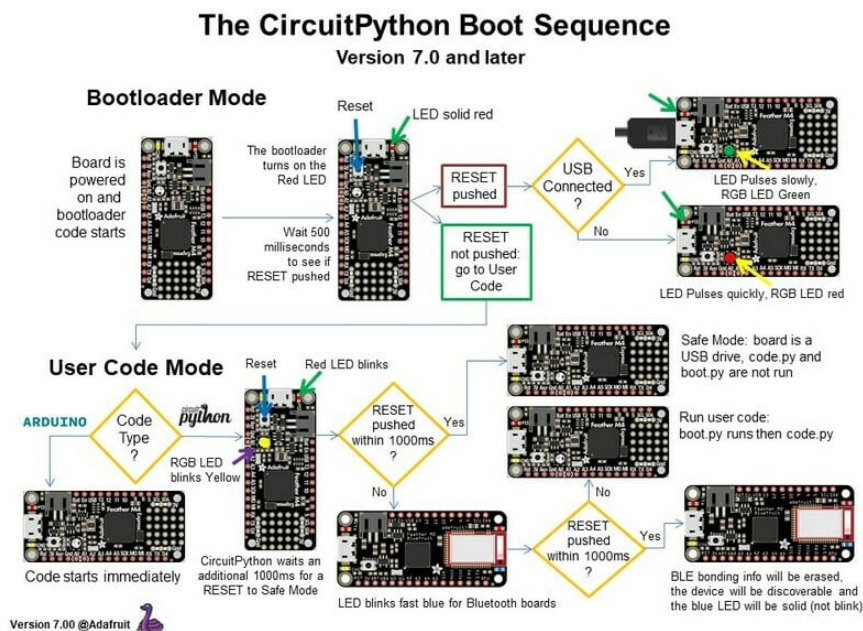
The status LED blinks were changed in CircuitPython 7.0.0 in order to save battery power and simplify the blinks. These blink patterns will occur on single color LEDs when the board does not have any RGB LEDs. Speed and blink count also vary for this reason.

On start up, the LED will blink **YELLOW** multiple times for 1 second. Pressing the RESET button (or on Espressif, the BOOT button) during this time will restart the board and then enter safe mode. On Bluetooth capable boards, after the yellow blinks, there will be a set of faster blue blinks. Pressing reset during the **BLUE** blinks will clear Bluetooth information and start the device in discoverable mode, so it can be used with a BLE code editor.

Once started, CircuitPython will blink a pattern every 5 seconds when no user code is running to indicate why the code stopped:

- 1 **GREEN** blink: Code finished without error.
- 2 **RED** blinks: Code ended due to an exception. Check the serial console for details.
- 3 **YELLOW** blinks: CircuitPython is in safe mode. No user code was run. Check the serial console for safe mode reason.

When in the REPL, CircuitPython will set the status LED to **WHITE**. You can change the LED color from the REPL. The status indicator will not persist on non-NeoPixel or DotStar LEDs.



CircuitPython 6.3.0 and earlier

Here's what the colors and blinking mean:

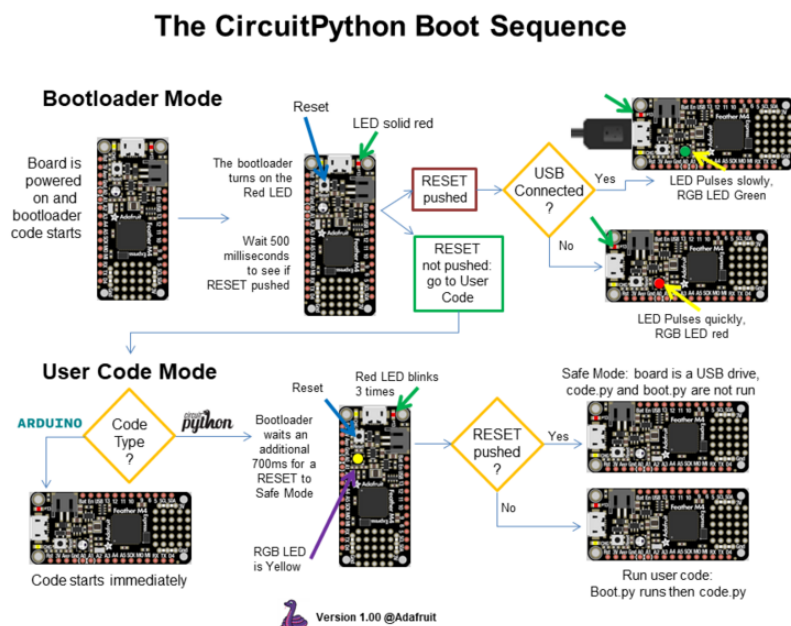
- steady **GREEN**: `code.py` (or `code.txt`, `main.py`, or `main.txt`) is running
- pulsing **GREEN**: `code.py` (etc.) has finished or does not exist
- steady **YELLOW** at start up: (4.0.0-alpha.5 and newer) CircuitPython is waiting for a reset to indicate that it should start in safe mode
- pulsing **YELLOW**: Circuit Python is in safe mode: it crashed and restarted
- steady **WHITE**: REPL is running
- steady **BLUE**: `boot.py` is running

Colors with multiple flashes following indicate a Python exception and then indicate the line number of the error. The color of the first flash indicates the type of error:

- **GREEN:** IndentationError
- **CYAN:** SyntaxError
- **WHITE:** NameError
- **ORANGE:** OSError
- **PURPLE:** ValueError
- **YELLOW:** other error

These are followed by flashes indicating the line number, including place value.

WHITE flashes are thousands' place, **BLUE** are hundreds' place, **YELLOW** are tens' place, and **CYAN** are one's place. So for example, an error on line 32 would flash **YELLOW** three times and then **CYAN** two times. Zeroes are indicated by an extra-long dark gap.



Serial console showing **ValueError: Incompatible .mpy file**

This error occurs when importing a module that is stored as a **.mpy** binary file that was generated by a different version of CircuitPython than the one its being loaded into. In particular, the mpy binary format changed between CircuitPython versions 6.x and 7.x, 2.x and 3.x, and 1.x and 2.x.

So, for instance, if you upgraded to CircuitPython 7.x from 6.x you'll need to download a newer version of the library that triggered the error on **import**. All libraries are available in the [Adafruit bundle \(https://adafru.it/y8E\)](https://adafru.it/y8E).

CIRCUITPY Drive Issues

You may find that you can no longer save files to your **CIRCUITPY** drive. You may find that your **CIRCUITPY** stops showing up in your file explorer, or shows up as **NO_NAME**. These are indicators that your filesystem has issues. When the **CIRCUITPY** disk is not safely ejected before being reset by the button or being disconnected from USB, it may corrupt the flash drive. It can happen on Windows, Mac or Linux, though it is more common on Windows.

Be aware, if you have used Arduino to program your board, CircuitPython is no longer able to provide the USB services. You will need to reload CircuitPython to resolve this situation.

The easiest first step is to reload CircuitPython. Double-tap reset on the board so you get a **boardnameBOOT** drive rather than a **CIRCUITPY** drive, and copy the latest version of CircuitPython (.uf2) back to the board. This may restore **CIRCUITPY** functionality.

If reloading CircuitPython does not resolve your issue, the next step is to try putting the board into safe mode.

Safe Mode

Whether you've run into a situation where you can no longer edit your **code.py** on your **CIRCUITPY** drive, your board has gotten into a state where **CIRCUITPY** is read-only, or you have turned off the **CIRCUITPY** drive altogether, safe mode can help.

Safe mode in CircuitPython does not run any user code on startup, and disables auto-reload. This means a few things. First, safe mode bypasses any code in **boot.py** (where you can set **CIRCUITPY** read-only or turn it off completely). Second, it does not run the code in **code.py**. And finally, it does not automatically soft-reload when data is written to the **CIRCUITPY** drive.

Therefore, whatever you may have done to put your board in a non-interactive state, safe mode gives you the opportunity to correct it without losing all of the data on the **CIRCUITPY** drive.

Entering Safe Mode in CircuitPython 7.x and Later

You can enter safe by pressing reset during the right time when the board boots. Immediately after the board starts up or resets, it waits one second. On some boards, the onboard status LED will blink yellow during that time. If you press reset during that one second period, the board will start up in safe mode. It can be difficult to react to the yellow LED, so you may want to think of it simply as a "slow" double click of the reset button. (Remember, a fast double click of reset enters the bootloader.)

Entering Safe Mode in CircuitPython 6.x

You can enter safe by pressing reset during the right time when the board boots.. Immediately after the board starts up or resets, it waits 0.7 seconds. On some boards, the onboard status LED (highlighted in green above) will turn solid yellow during this time. If you press reset during that 0.7 seconds, the board will start up in safe mode. It can be difficult to react to the yellow LED, so you may want to think of it simply as a slow double click of the reset button. (Remember, a fast double click of reset enters the bootloader.)

In Safe Mode

Once you've entered safe mode successfully in CircuitPython 6.x, the LED will pulse yellow.

If you successfully enter safe mode on CircuitPython 7.x, the LED will intermittently blink yellow three times.

If you connect to the serial console, you'll find the following message.

```
Auto-reload is off.  
Running in safe mode! Not running saved code.  
  
CircuitPython is in safe mode because you pressed the reset button during boot.  
Press again to exit safe mode.  
  
Press any key to enter the REPL. Use CTRL-D to reload.
```

You can now edit the contents of the **CIRCUITPY** drive. Remember, your code will not run until you press the reset button, or unplug and plug in your board, to get out of safe mode.

At this point, you'll want to remove any user code in **code.py** and, if present, the **boot.py** file from **CIRCUITPY**. Once removed, tap the reset button, or unplug and plug in your board, to restart CircuitPython. This will restart the board and may resolve your drive issues. If resolved, you can begin coding again as usual.

If safe mode does not resolve your issue, the board must be completely erased and CircuitPython must be reloaded onto the board.

You WILL lose everything on the board when you complete the following steps. If possible, make a copy of your code before continuing.

To erase CIRCUITPY: `storage.erase_filesystem()`

CircuitPython includes a built-in function to erase and reformat the filesystem. If you have a version of CircuitPython older than 2.3.0 on your board, you can [update to the newest version](https://adafru.it/Amd) (<https://adafru.it/Amd>) to do this.

1. [Connect to the CircuitPython REPL](https://adafru.it/Bec) (<https://adafru.it/Bec>) using Mu or a terminal program.
2. Type the following into the REPL:

```
>>> import storage
>>> storage.erase_filesystem()
```

CIRCUITPY will be erased and reformatted, and your board will restart. That's it!

Erase CIRCUITPY Without Access to the REPL

If you can't access the REPL, or you're running a version of CircuitPython previous to 2.3.0 and you don't want to upgrade, there are options available for some specific boards.

The options listed below are considered to be the "old way" of erasing your board. The method shown above using the REPL is highly recommended as the best method for erasing your board.

If at all possible, it is recommended to use the REPL to erase your CIRCUITPY drive. The REPL method is explained above.

For the specific boards listed below:

If the board you are trying to erase is listed below, follow the steps to use the file to erase your board.

1. Download the correct erase file:

Circuit Playground Express

<https://adafru.it/AdI>

Feather M0 Express

<https://adafru.it/AdJ>

Feather M4 Express

<https://adafru.it/EVK>

Metro M0 Express

<https://adafru.it/AdK>

Metro M4 Express QSPI Eraser

<https://adafru.it/EoM>

Trellis M4 Express (QSPI)

<https://adafru.it/DjD>

Grand Central M4 Express (QSPI)

<https://adafru.it/DBA>

PyPortal M4 Express (QSPI)

<https://adafru.it/Eca>

Circuit Playground Bluefruit (QSPI)

<https://adafru.it/Gnc>

Monster M4SK (QSPI)

<https://adafru.it/GAN>

PyBadge/PyGamer QSPI Eraser.UF2

<https://adafru.it/GAO>

CLUE_Flash_Erase.UF2

<https://adafru.it/Jat>

Matrix_Portal_M4_(QSPI).UF2

<https://adafru.it/Q5B>

RP2040 boards (flash_nuke.uf2)

<https://adafru.it/18ed>

2. Double-click the reset button on the board to bring up the **boardnameBOOT** drive.

3. Drag the erase **.uf2** file to the **boardnameBOOT** drive.

4. The status LED will turn yellow or blue, indicating the erase has started.

5. After approximately 15 seconds, the status LED will light up green. On the NeoTrellis M4 this is the first NeoPixel on the grid

6. Double-click the reset button on the board to bring up the **boardnameBOOT** drive.

7. [Drag the appropriate latest release of CircuitPython \(https://adafru.it/Em8\)](https://adafru.it/Em8) **.uf2** file to the **boardnameBOOT** drive.

It should reboot automatically and you should see **CIRCUITPY** in your file explorer again.

If the LED flashes red during step 5, it means the erase has failed. Repeat the steps starting with 2.

[If you haven't already downloaded the latest release of CircuitPython for your board, check out the installation page \(https://adafru.it/Amd\)](https://adafru.it/Amd). You'll also need to load your code and reinstall your libraries!

For SAMD21 non-Express boards that have a UF2 bootloader:

Any SAMD21-based microcontroller that does not have external flash available is considered a SAMD21 non-Express board. Non-Express boards that have a UF2 bootloader include Trinket M0, GEMMA M0, QT Py M0, and the SAMD21-based Trinkey boards.

If you are trying to erase a SAMD21 non-Express board, follow these steps to erase your board.

1. Download the erase file:

SAMD21 non-Express Boards

<https://adafru.it/VB->

2. Double-click the reset button on the board to bring up the **boardnameBOOT** drive.

3. Drag the erase **.uf2** file to the **boardnameBOOT** drive.

4. The boot LED will start flashing again, and the **boardnameBOOT** drive will reappear.

5. [Drag the appropriate latest release CircuitPython \(https://adafru.it/Em8\)](https://adafru.it/Em8) **.uf2** file to the **boardnameBOOT** drive.

It should reboot automatically and you should see **CIRCUITPY** in your file explorer again.

[If you haven't already downloaded the latest release of CircuitPython for your board, check out the installation page \(https://adafru.it/Amd\)](https://adafru.it/Amd) You'll also need to load your code and reinstall your libraries!

For SAMD21 non-Express boards that do not have a UF2 bootloader:

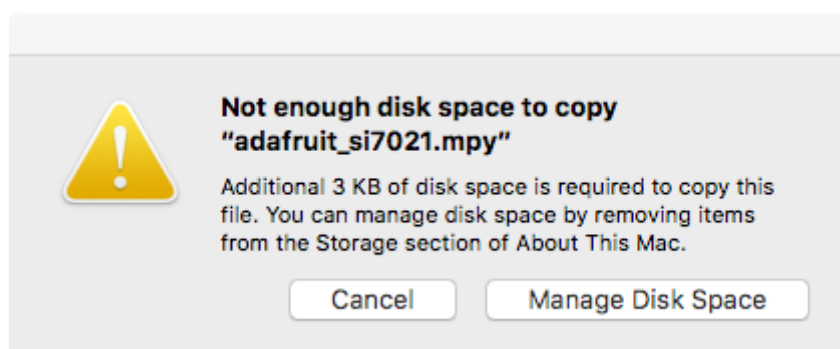
Any SAMD21-based microcontroller that does not have external flash available is considered a SAMD21 non-Express board. Non-Express boards that do **not** have a UF2 bootloader include the Feather M0 Basic Proto, Feather Adalogger, or the Arduino Zero.

If you are trying to erase a non-Express board that does not have a UF2 bootloader, [follow these directions to reload CircuitPython using bossac \(https://adafru.it/Bed\)](https://adafru.it/Bed), which will erase and re-create **CIRCUITPY**.

Running Out of File Space on SAMD21 Non-Express Boards

Any SAMD21-based microcontroller that does not have external flash available is considered a SAMD21 non-Express board. This includes boards like the Trinket M0, GEMMA M0, QT Py M0, and the SAMD21-based Trinkey boards.

The file system on the board is very tiny. (Smaller than an ancient floppy disk.) So, its likely you'll run out of space but don't panic! There are a number of ways to free up space.



Delete something!

The simplest way of freeing up space is to delete files from the drive. Perhaps there are libraries in the **lib** folder that you aren't using anymore or test code that isn't in use. Don't delete the **lib** folder completely, though, just remove what you don't need.

The board ships with the Windows 7 serial driver too! Feel free to delete that if you don't need it or have already installed it. It's ~12KiB or so.

Use tabs

One unique feature of Python is that the indentation of code matters. Usually the recommendation is to indent code with four spaces for every indent. In general, that is recommended too. **However**, one trick to storing more human-readable code is to use a single tab character for indentation. This approach uses 1/4 of the space for indentation and can be significant when you're counting bytes.

On macOS?

MacOS loves to generate hidden files. Luckily you can disable some of the extra hidden files that macOS adds by running a few commands to disable search indexing and create zero byte placeholders. Follow the steps below to maximize the amount of space available on macOS.

Prevent & Remove macOS Hidden Files

First find the volume name for your board. With the board plugged in run this command in a terminal to list all the volumes:

```
ls -l /Volumes
```

Look for a volume with a name like **CIRCUITPY** (the default for CircuitPython). The full path to the volume is the **/Volumes/CIRCUITPY** path.

Now follow the [steps from this question \(https://adafru.it/u1c\)](https://adafru.it/u1c) to run these terminal commands that stop hidden files from being created on the board:

```
mdutil -i off /Volumes/CIRCUITPY
cd /Volumes/CIRCUITPY
rm -rf .{,._}{fseventsd,Spotlight-V*,Trashes}
mkdir .fseventsd
touch .fseventsd/no_log .metadata_never_index .Trashes
cd -
```

Replace **/Volumes/CIRCUITPY** in the commands above with the full path to your board's volume if it's different. At this point all the hidden files should be cleared from the board and some hidden files will be prevented from being created.

Alternatively, with CircuitPython 4.x and above, the special files and folders mentioned above will be created automatically if you erase and reformat the filesystem. **WARNING: Save your files first!** Do this in the REPL:

```
>>> import storage
>>> storage.erase_filesystem()
```

However there are still some cases where hidden files will be created by MacOS. In particular if you copy a file that was downloaded from the internet it will have special metadata that MacOS stores as a hidden file. Luckily you can run a copy command from the terminal to copy files **without** this hidden metadata file. See the steps below.

Copy Files on macOS Without Creating Hidden Files

Once you've disabled and removed hidden files with the above commands on macOS you need to be careful to copy files to the board with a special command that prevents future hidden files from being created. Unfortunately you **cannot** use drag and drop copy in Finder because it will still create these hidden extended attribute files in some cases (for files downloaded from the internet, like Adafruit's modules).

To copy a file or folder use the **-X** option for the **cp** command in a terminal. For example to copy a **file_name.mpy** file to the board use a command like:

```
cp -X file_name.mpy /Volumes/CIRCUITPY
```

(Replace **file_name.mpy** with the name of the file you want to copy.)

Or to copy a folder and all of the files and folders contained within, use a command like:

```
cp -rX folder_to_copy /Volumes/CIRCUITPY
```

If you are copying to the **lib** folder, or another folder, make sure it exists before copying.

```
# if lib does not exist, you'll create a file named lib !
cp -X file_name.mpy /Volumes/CIRCUITPY/lib
# This is safer, and will complain if a lib folder does not exist.
cp -X file_name.mpy /Volumes/CIRCUITPY/lib/
```

Other macOS Space-Saving Tips

If you'd like to see the amount of space used on the drive and manually delete hidden files here's how to do so. First, move into the **Volumes/** directory with **cd / Volumes/**, and then list the amount of space used on the **CIRCUITPY** drive with the **df** command.

```

Last login: Thu Oct 28 17:19:15 on ttys008

7039 kattni@robocrepe:~ $ cd /Volumes/

7040 kattni@robocrepe:Volumes $ df -h CIRCUITPY/
Filesystem      Size  Used Avail Capacity iused ifree %used  Mounted on
/dev/disk2s1    47Ki  46Ki  1.0Ki   98%    512     0  100%  /Volumes/CIRCUITPY

7041 kattni@robocrepe:Volumes $

```

That's not very much space left! The next step is to show a list of the files currently on the **CIRCUITPY** drive, including the hidden files, using the `ls` command. You cannot use Finder to do this, you must do it via command line!

```

7041 kattni@robocrepe:Volumes $ ls -a CIRCUITPY/
.          .trinket_code.py  code.py
..         .fsevents         lib
.Trashes   .idea             original_code.py
..code.py  .metadata_never_index trinket_code.py
..original_code.py boot_out.txt

7042 kattni@robocrepe:Volumes $

```

There are a few of the hidden files that MacOS loves to generate, all of which begin with a `._` before the file name. Remove the `._` files using the `rm` command. You can remove them all once by running `rm CIRCUITPY/._*`. The `*` acts as a wildcard to apply the command to everything that begins with `._` at the same time.

```

7042 kattni@robocrepe:Volumes $ rm CIRCUITPY/._*

7043 kattni@robocrepe:Volumes $

```

Finally, you can run `df` again to see the current space used.

```

7043 kattni@robocrepe:Volumes $ df -h CIRCUITPY/
Filesystem      Size  Used Avail Capacity iused ifree %used  Mounted on
/dev/disk2s1    47Ki  34Ki  13Ki   73%    512     0  100%  /Volumes/CIRCUITPY

7044 kattni@robocrepe:Volumes $

```

Nice! You have 12Ki more than before! This space can now be used for libraries and code!

Device Locked Up or Boot Looping

In rare cases, it may happen that something in your `code.py` or `boot.py` files causes the device to get locked up, or even go into a boot loop. A boot loop occurs when the board reboots repeatedly and never fully loads. These are not caused by your everyday Python exceptions, typically it's the result of a deeper problem within CircuitPython. In this situation, it can be difficult to recover your device if **CIRCUITPY** is not allowing you to modify the `code.py` or `boot.py` files. Safe mode is one recovery option. When the device boots up in safe mode it will not run the `code.py` or `boot.py`

scripts, but will still connect the **CIRCUITPY** drive so that you can remove or modify those files as needed.

For more information on safe mode and how to enter safe mode, see the [Safe Mode section on this page \(https://adafru.it/Den\)](https://adafru.it/Den).

Frequently Asked Questions

These are some of the common questions regarding CircuitPython and CircuitPython microcontrollers.



What are some common acronyms to know?

CP or CPy = [CircuitPython \(https://adafru.it/KJD\)](https://adafru.it/KJD)

CPC = [Circuit Playground Classic \(http://adafru.it/3000\)](http://adafru.it/3000) (does not run CircuitPython)

CPX = [Circuit Playground Express \(http://adafru.it/3333\)](http://adafru.it/3333)

CPB = [Circuit Playground Bluefruit \(http://adafru.it/4333\)](http://adafru.it/4333)

Using Older Versions

As CircuitPython development continues and there are new releases, Adafruit will stop supporting older releases. Visit <https://circuitpython.org/downloads> to download the latest version of CircuitPython for your board. You must download the CircuitPython Library Bundle that matches your version of CircuitPython. Please update CircuitPython and then visit <https://circuitpython.org/libraries> to download the latest Library Bundle.



I have to continue using CircuitPython 8.x or earlier. Where can I find compatible libraries?

We are no longer building or supporting the CircuitPython 8.x or earlier library bundles. We highly encourage you to [update CircuitPython to the latest version \(https://adafru.it/Em8\)](https://adafru.it/Em8) and use [the current version of the libraries \(https://adafru.it/ENC\)](https://adafru.it/ENC). However, if for some reason you cannot update, here are the last available library bundles for older versions:

- [2.x bundle \(https://adafru.it/FJA\)](https://adafru.it/FJA)

- [3.x bundle \(https://adafru.it/FJB\)](https://adafru.it/FJB)
- [4.x bundle \(https://adafru.it/QDL\)](https://adafru.it/QDL)
- [5.x bundle \(https://adafru.it/QDJ\)](https://adafru.it/QDJ)
- [6.x bundle \(https://adafru.it/Xmf\)](https://adafru.it/Xmf)
- [7.x bundle \(https://adafru.it/18e9\)](https://adafru.it/18e9)
- [8.x bundle \(https://adafru.it/1af0\)](https://adafru.it/1af0)

Python Arithmetic



Does CircuitPython support floating-point numbers?

All CircuitPython boards support floating point arithmetic, even if the microcontroller chip does not support floating point in hardware. Floating point numbers are stored in 30 bits, with an 8-bit exponent and a 22-bit mantissa. Note that this is two bits less than standard 32-bit single-precision floats. You will get about 5-1/2 digits of decimal precision.

(The **broadcom** port may provide 64-bit floats in some cases.)



Does CircuitPython support long integers, like regular Python?

Python long integers (integers of arbitrary size) are available on most builds, except those on boards with the smallest available firmware size. On these boards, integers are stored in 31 bits.

Boards without long integer support are mostly SAMD21 ("M0") boards without an external flash chip, such as the Adafruit Gemma M0, Trinket M0, QT Py M0, and the Trinkey series. There are also a number of third-party boards in this category. There are also a few small STM third-party boards without long integer support.

`time.localtime()`, `time.mktime()`, `time.time()`, and `time.monotonic_ns()` are available only on builds with long integers.

Wireless Connectivity



How do I connect to the Internet with CircuitPython?

If you'd like to include WiFi in your project, your best bet is to use a board that is running natively on ESP32 chipsets - those have WiFi built in!

If your development board has an SPI port and at least 4 additional pins, you can check out [this guide \(https://adafru.it/F5X\)](https://adafru.it/F5X) on using AirLift with CircuitPython - extra wiring is required and some boards like the MacroPad or NeoTrellis do not have enough available pins to add the hardware support.

For further project examples, and guides about using AirLift with specific hardware, check out [the Adafruit Learn System \(https://adafru.it/VBr\)](https://adafru.it/VBr).



How do I do BLE (Bluetooth Low Energy) with CircuitPython?

nRF52840, nRF52833, and as of **CircuitPython 9.1.0**, ESP32, ESP32-C3, and ESP32-S3 boards (with 8MB) have the most complete BLE implementation. Your program can act as both a BLE central and peripheral. As a central, you can scan for advertisements, and connect to an advertising board. As a peripheral, you can advertise, and you can create services available to a central. Pairing and bonding are supported.

Most Espressif boards with only 4MB of flash do not have enough room to include BLE in CircuitPython 9. Check the [Module Support Matrix \(https://adafru.it/-Cy\)](https://adafru.it/-Cy) to see if your board has support for `_bleio`. CircuitPython 10 is planned to support `_bleio` on Espressif boards with 4MB flash.

Note that the ESP32-S2 does not have Bluetooth capability.

On most other boards with adequate firmware space, [BLE is available for use with AirLift \(https://adafru.it/11Av\)](https://adafru.it/11Av) or other NINA-FW-based coprocessors. Some boards have this coprocessor on board, such as the [PyPortal \(https://adafru.it/11Aw\)](https://adafru.it/11Aw). Currently, this implementation only supports acting as a BLE peripheral. Scanning and connecting as a central are not yet implemented. Bonding and pairing are not supported.



Are there other ways to communicate by radio with CircuitPython?

Check out [Adafruit's RFM boards \(https://adafru.it/11Ay\)](https://adafru.it/11Ay) for simple radio communication supported by CircuitPython, which can be used over distances of 100m to over a km, depending on the version. The RFM SAMD21 M0 boards can be used, but they were not designed for CircuitPython, and have limited RAM and flash space; using the RFM breakouts or FeatherWings with more capable boards will be easier.

Asyncio and Interrupts



Is there asyncio support in CircuitPython?

There is support for asyncio starting with CircuitPython 7.1.0, on all boards except the smallest SAMD21 builds. Read about using it in the [Cooperative Multitasking in CircuitPython \(https://adafru.it/XnA\)](https://adafru.it/XnA) Guide.



Does CircuitPython support interrupts?

No. CircuitPython does not currently support interrupts - please use asyncio for multitasking / 'threaded' control of your code

Status RGB LED



My RGB NeoPixel/DotStar LED is blinking funny colors - what does it mean?

The status LED can tell you what's going on with your CircuitPython board. [Read more here for what the colors mean! \(https://adafru.it/Den\)](https://adafru.it/Den)

Memory Issues



What is a MemoryError?

Memory allocation errors happen when you're trying to store too much on the board. The CircuitPython microcontroller boards have a limited amount of memory available. You can have about 250 lines of code on the M0 Express boards. If you try to `import` too many libraries, a combination of large libraries, or run a program with too many lines of code, your code will fail to run and you will receive a `MemoryError` in the serial console.



What do I do when I encounter a MemoryError?

Try resetting your board. Each time you reset the board, it reallocates the memory. While this is unlikely to resolve your issue, it's a simple step and is worth trying.

Make sure you are using `.mpy` versions of libraries. All of the CircuitPython libraries are available in the bundle in a `.mpy` format which takes up less memory than `.py` format. Be sure that you're using [the latest library bundle \(https://adafru.it/uap\)](https://adafru.it/uap) for your version of CircuitPython.

If that does not resolve your issue, try shortening your code. Shorten comments, remove extraneous or unneeded code, or any other clean up you can do to shorten your code. If you're using a lot of functions, you could try moving those into a separate library, creating a `.mpy` of that library, and importing it into your code.

You can turn your entire file into a `.mpy` and `import` that into `code.py`. This means you will be unable to edit your code live on the board, but it can save you space.



Can the order of my `import` statements affect memory?

It can because the memory gets fragmented differently depending on allocation order and the size of objects. Loading `.mpy` files uses less memory so its recommended to do that for files you aren't editing.



How can I create my own `.mpy` files?

You can make your own `.mpy` versions of files with `mpy-cross`.

You can download `mpy-cross` for your operating system from [here \(https://adafru.it/QDK\)](https://adafru.it/QDK). Builds are available for Windows, macOS, x64 Linux, and Raspberry Pi Linux. Choose the latest `mpy-cross` whose version matches the version of CircuitPython you are using.

On macOS and Linux, after you download `mpy-cross`, you must make the the file executable by doing `chmod +x name-of-the-mpy-cross-executable`.

To make a `.mpy` file, run `./mpy-cross path/to/yourfile.py` to create a `yourfile.mpy` in the same directory as the original file.



How do I check how much memory I have free?

Run the following to see the number of bytes available for use:

```
import gc
gc.mem_free()
```

Unsupported Hardware



Is ESP8266 or ESP32 supported in CircuitPython? Why not?

We dropped ESP8266 support as of 4.x - For more information please read about it [here \(https://adafru.it/CiG\)](https://adafru.it/CiG)!

As of CircuitPython 8.x we have started to support ESP32 and ESP32-C3 and have added a WiFi workflow for wireless coding! (<https://adafru.it/10JF>)

We also support ESP32-S2 & ESP32-S3, which have native USB.



Does Feather M0 support WINC1500?

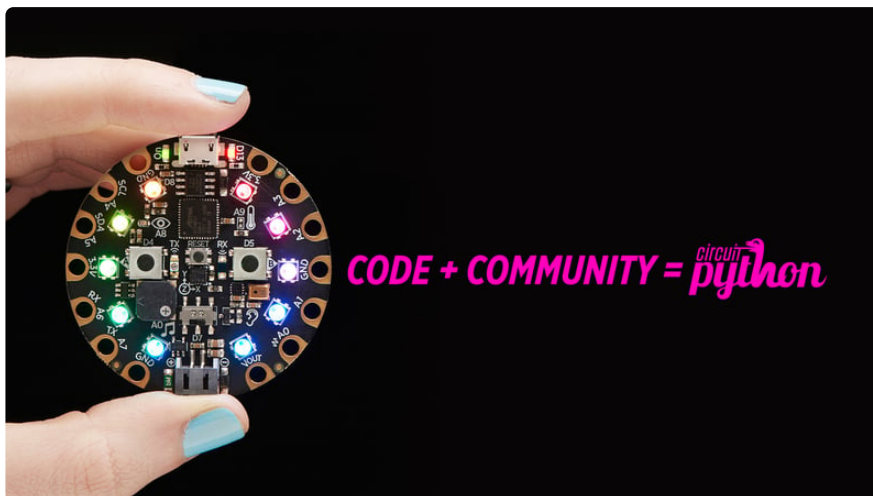
No, WINC1500 will not fit into the M0 flash space.



Can AVR's such as ATmega328 or ATmega2560 run CircuitPython?

No.

Welcome to the Community!

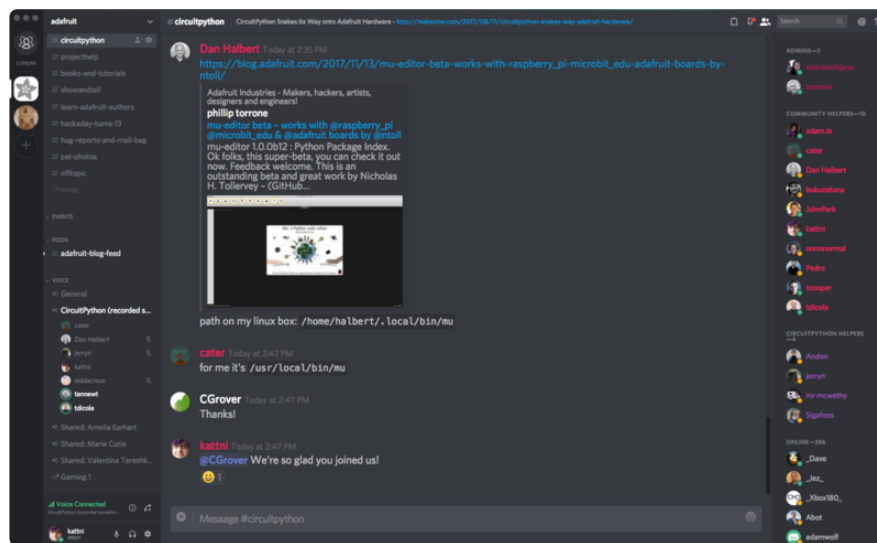


CircuitPython is a programming language that's super simple to get started with and great for learning. It runs on microcontrollers and works out of the box. You can plug it

in and get started with any text editor. The best part? CircuitPython comes with an amazing, supportive community.

Everyone is welcome! CircuitPython is Open Source. This means it's available for anyone to use, edit, copy and improve upon. This also means CircuitPython becomes better because of you being a part of it. Whether this is your first microcontroller board or you're a seasoned software engineer, you have something important to offer the Adafruit CircuitPython community. This page highlights some of the many ways you can be a part of it!

Adafruit Discord



The Adafruit Discord server is the best place to start. Discord is where the community comes together to volunteer and provide live support of all kinds. From general discussion to detailed problem solving, and everything in between, Discord is a digital maker space with makers from around the world.

There are many different channels so you can choose the one best suited to your needs. Each channel is shown on Discord as "#channelname". There's the #help-with-projects channel for assistance with your current project or help coming up with ideas for your next one. There's the #show-and-tell channel for showing off your newest creation. Don't be afraid to ask a question in any channel! If you're unsure, #general is a great place to start. If another channel is more likely to provide you with a better answer, someone will guide you.

The help with CircuitPython channel is where to go with your CircuitPython questions. #help-with-circuitpython is there for new users and developers alike so feel free to ask a question or post a comment! Everyone of any experience level is welcome to join in on the conversation. Your contributions are important! The #circuitpython-dev channel is available for development discussions as well.

The easiest way to contribute to the community is to assist others on Discord. Supporting others doesn't always mean answering questions. Join in celebrating successes! Celebrate your mistakes! Sometimes just hearing that someone else has gone through a similar struggle can be enough to keep a maker moving forward.

The Adafruit Discord is the 24x7x365 hackerspace that you can bring your granddaughter to.

Visit <https://adafru.it/discord> () to sign up for Discord. Everyone is looking forward to meeting you!

CircuitPython.org



Beyond the Adafruit Learn System, which you are viewing right now, the best place to find information about CircuitPython is [circuitpython.org](https://adafru.it/KJD) (<https://adafru.it/KJD>). Everything you need to get started with your new microcontroller and beyond is available. You can do things like [download CircuitPython for your microcontroller](https://adafru.it/Em8) (<https://adafru.it/Em8>) or [download the latest CircuitPython Library bundle](https://adafru.it/ENC) (<https://adafru.it/ENC>), or check out [which single board computers support Blinks](https://adafru.it/EA8) (<https://adafru.it/EA8>). You can also get to various other CircuitPython related things like Awesome CircuitPython or the Python for Microcontrollers newsletter. This is all incredibly useful, but it isn't necessarily community related. So why is it included here? The [Contributing page](https://adafru.it/VD7) (<https://adafru.it/VD7>).

Contributing

If you'd like to contribute to the CircuitPython project, the CircuitPython libraries are a great way to begin. This page is updated with daily status information from the CircuitPython libraries, including open pull requests, open issues and library infrastructure issues.

Do you write a language other than English? Another great way to contribute to the project is to contribute new localizations (translations) of CircuitPython, or update current localizations, using [Weblate](#).

If this is your first time contributing, or you'd like to see our recommended contribution workflow, we have a guide on [Contributing to CircuitPython with Git and Github](#). You can also find us in the #circuitpython channel on the [Adafruit Discord](#).

Have an idea for a new driver or library? [File an issue on the CircuitPython repo!](#)

CircuitPython itself is written in C. However, all of the Adafruit CircuitPython libraries are written in Python. If you're interested in contributing to CircuitPython on the

Python side of things, check out [circuitpython.org/contributing](https://adafru.it/VD7) (<https://adafru.it/VD7>). You'll find information pertaining to every Adafruit CircuitPython library GitHub repository, giving you the opportunity to join the community by finding a contributing option that works for you.

Note the date on the page next to **Current Status** for:

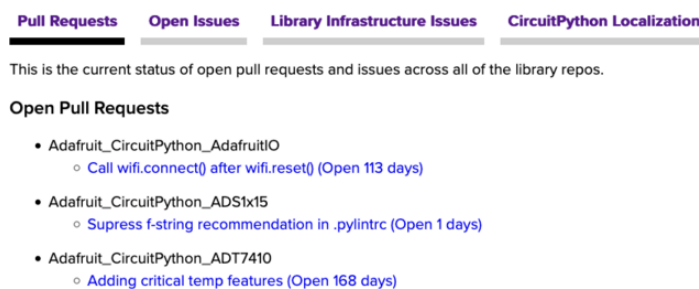
Current Status for Tue, Nov 02, 2021

If you submit any contributions to the libraries, and do not see them reflected on the Contributing page, it could be that the job that checks for new updates hasn't yet run for today. Simply check back tomorrow!

Now, a look at the different options.

Pull Requests

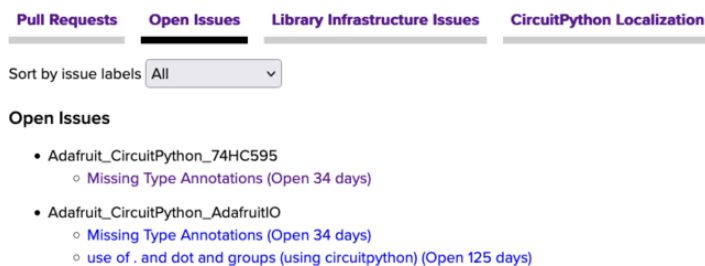
The first tab you'll find is a list of **open pull requests**.



GitHub pull requests, or PRs, are opened when folks have added something to an Adafruit CircuitPython library GitHub repo, and are asking for Adafruit to add, or merge, their changes into the main library code. For PRs to be merged, they must first be reviewed. Reviewing is a great way to contribute! Take a look at the list of open pull requests, and pick one that interests you. If you have the hardware, you can test code changes. If you don't, you can still check the code updates for syntax. In the case of documentation updates, you can verify the information, or check it for spelling and grammar. Once you've checked out the update, you can leave a comment letting us know that you took a look. Once you've done that for a while, and you're more comfortable with it, you can consider joining the CircuitPythonLibrarians review team. The more reviewers we have, the more authors we can support. Reviewing is a crucial part of an open source ecosystem, CircuitPython included.

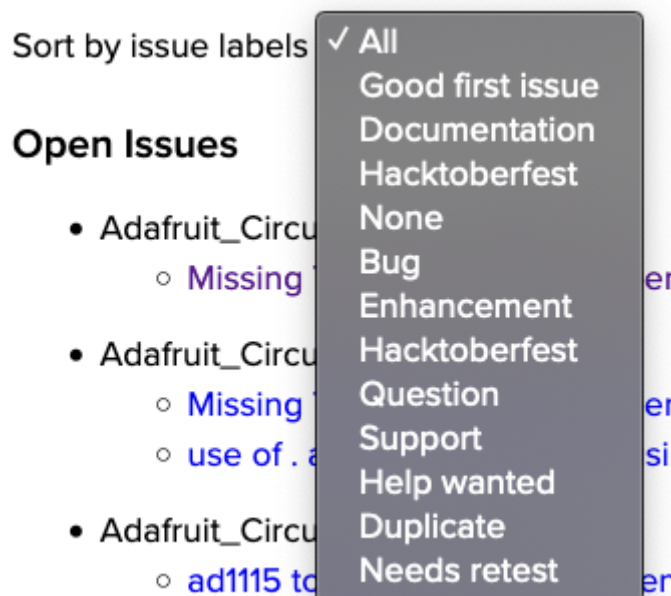
Open Issues

The second tab you'll find is a list of **open issues**.



GitHub issues are filed for a number of reasons, including when there is a bug in the library or example code, or when someone wants to make a feature request. Issues are a great way to find an opportunity to contribute directly to the libraries by updating code or documentation. If you're interested in contributing code or documentation, take a look at the open issues and find one that interests you.

If you're not sure where to start, you can search the issues by label. Labels are applied to issues to make the goal easier to identify at a first glance, or to indicate the difficulty level of the issue. Click on the dropdown next to "Sort by issue labels" to see the list of available labels, and click on one to choose it.



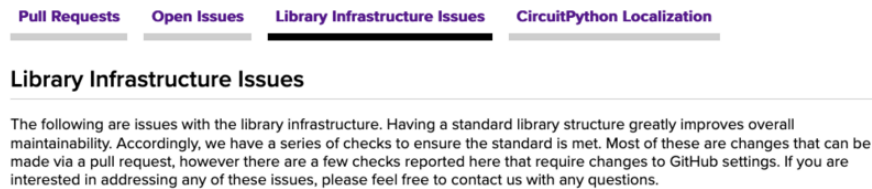
If you're new to everything, new to contributing to open source, or new to contributing to the CircuitPython project, you can choose "Good first issue". Issues with that label are well defined, with a finite scope, and are intended to be easy for someone new to figure out.

If you're looking for something a little more complicated, consider "Bug" or "Enhancement". The Bug label is applied to issues that pertain to problems or failures found in the library. The Enhancement label is applied to feature requests.

Don't let the process intimidate you. If you're new to Git and GitHub, there is [a guide \(https://adafru.it/Dkh\)](https://adafru.it/Dkh) to walk you through the entire process. As well, there are always folks available on [Discord \(\)](#) to answer questions.

Library Infrastructure Issues

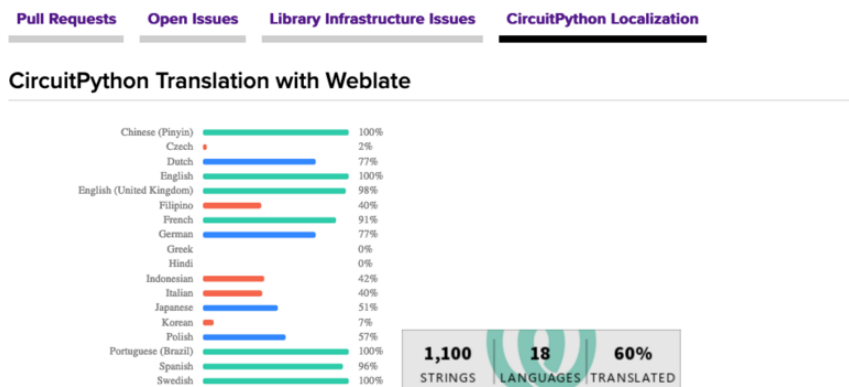
The third tab you'll find is a list of **library infrastructure issues**.



This section is generated by a script that runs checks on the libraries, and then reports back where there may be issues. It is made up of a list of subsections each containing links to the repositories that are experiencing that particular issue. This page is available mostly for internal use, but you may find some opportunities to contribute on this page. If there's an issue listed that sounds like something you could help with, mention it on Discord, or file an issue on GitHub indicating you're working to resolve that issue. Others can reply either way to let you know what the scope of it might be, and help you resolve it if necessary.

CircuitPython Localization

The fourth tab you'll find is the **CircuitPython Localization** tab.



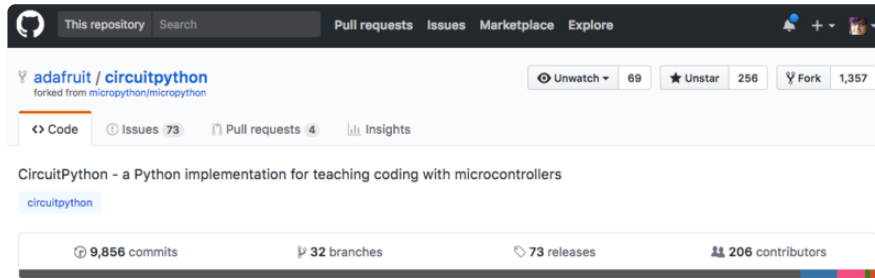
If you speak another language, you can help translate CircuitPython! The translations apply to informational and error messages that are within the CircuitPython core. It means that folks who do not speak English have the opportunity to have these messages shown to them in their own language when using CircuitPython. This is incredibly important to provide the best experience possible for all users.

CircuitPython uses Weblate to translate, which makes it much simpler to contribute translations. You will still need to know some CircuitPython-specific practices and a

few basics about coding strings, but as with any CircuitPython contributions, folks are there to help.

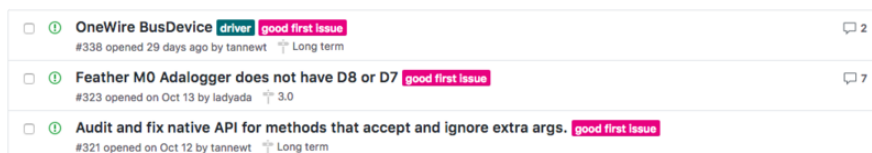
Regardless of your skill level, or how you want to contribute to the CircuitPython project, there is an opportunity available. The [Contributing page \(https://adafru.it/VD7\)](https://adafru.it/VD7) is an excellent place to start!

Adafruit GitHub



Whether you're just beginning or are life-long programmer who would like to contribute, there are ways for everyone to be a part of the CircuitPython project. The CircuitPython core is written in C. The libraries are written in Python. GitHub is the best source of ways to contribute to the [CircuitPython core \(https://adafru.it/tB7\)](https://adafru.it/tB7), and the [CircuitPython libraries \(https://adafru.it/VFv\)](https://adafru.it/VFv). If you need an account, visit <https://github.com/> (<https://adafru.it/d6C>) and sign up.

If you're new to GitHub or programming in general, there are great opportunities for you. For the CircuitPython core, head over to the CircuitPython repository on GitHub, click on "[Issues \(https://adafru.it/tBb\)](https://adafru.it/tBb)", and you'll find a list that includes issues labeled "[good first issue \(https://adafru.it/188e\)](https://adafru.it/188e)". For the libraries, head over to the [Contributing page Issues list \(https://adafru.it/VFv\)](https://adafru.it/VFv), and use the drop down menu to search for "[good first issue \(https://adafru.it/VFw\)](https://adafru.it/VFw)". These issues are things that have been identified as something that someone with any level of experience can help with. These issues include options like updating documentation, providing feedback, and fixing simple bugs. If you need help getting started with GitHub, there is an excellent guide on [Contributing to CircuitPython with Git and GitHub \(https://adafru.it/Dkh\)](https://adafru.it/Dkh).



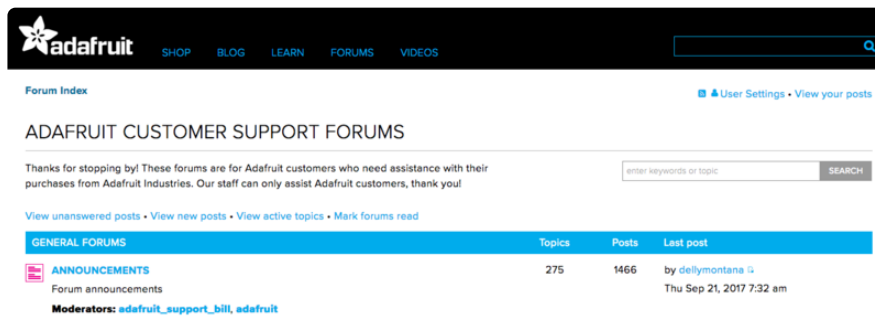
Already experienced and looking for a challenge? Checkout the rest of either issues list and you'll find plenty of ways to contribute. You'll find all sorts of things, from new driver requests, to library bugs, to core module updates. There's plenty of opportunities for everyone at any level!

When working with or using CircuitPython or the CircuitPython libraries, you may find problems. If you find a bug, that's great! The team loves bugs! Posting a detailed issue to GitHub is an invaluable way to contribute to improving CircuitPython. For CircuitPython itself, file an issue [here](https://adafru.it/tBb) (<https://adafru.it/tBb>). For the libraries, file an issue on the specific library repository on GitHub. Be sure to include the steps to replicate the issue as well as any other information you think is relevant. The more detail, the better!

Testing new software is easy and incredibly helpful. Simply load the newest version of CircuitPython or a library onto your CircuitPython hardware, and use it. Let us know about any problems you find by posting a new issue to GitHub. Software testing on both stable and unstable releases is a very important part of contributing CircuitPython. The developers can't possibly find all the problems themselves! They need your help to make CircuitPython even better.

On GitHub, you can submit feature requests, provide feedback, report problems and much more. If you have questions, remember that Discord and the Forums are both there for help!

Adafruit Forums



The [Adafruit Forums](https://adafru.it/jlf) (<https://adafru.it/jlf>) are the perfect place for support. Adafruit has wonderful paid support folks to answer any questions you may have. Whether your hardware is giving you issues or your code doesn't seem to be working, the forums are always there for you to ask. You need an Adafruit account to post to the forums. You can use the same account you use to order from Adafruit.

While Discord may provide you with quicker responses than the forums, the forums are a more reliable source of information. If you want to be certain you're getting an Adafruit-supported answer, the forums are the best place to be.

There are forum categories that cover all kinds of topics, including everything Adafruit. The [Adafruit CircuitPython](https://adafru.it/xXA) (<https://adafru.it/xXA>) category under "Supported Products & Projects" is the best place to post your CircuitPython questions.

Forum Index > Supported Products & Projects > Adafruit CircuitPython

Adafruit CircuitPython
Moderators: adafruit_support_bill, adafruit

POST A TOPIC Search this forum... SEARCH Mark topics read • 4154 topics • Page 1 of 84 • 12345 ... 84

Please be positive and constructive with your questions and comments.

ANNOUNCEMENTS	Replies	Views	Last post
CIRCUITPYTHON 7.2.0 ALPHA 1 RELEASED! by danhalbert • Tue Dec 28, 2021 11:55 pm	0	20	by danhalbert • Tue Dec 28, 2021 11:55 pm
CIRCUITPYTHON 7.1.0 RELEASED! by danhalbert • Tue Dec 28, 2021 12:01 pm	1	32	by rplloverbd • Wed Dec 29, 2021 5:53 am
SAMD51 (M4) BOARD USERS: UPDATE YOUR BOOTLOADERS TO >=V3.9.0 by danhalbert • Fri May 08, 2020 12:55 pm	10	2428	by Guest • Sat Aug 15, 2020 11:28 pm

TOPICS	Replies	Views	Last post
--------	---------	-------	-----------

Be sure to include the steps you took to get to where you are. If it involves wiring, post a picture! If your code is giving you trouble, include your code in your post! These are great ways to make sure that there's enough information to help you with your issue.

You might think you're just getting started, but you definitely know something that someone else doesn't. The great thing about the forums is that you can help others too! Everyone is welcome and encouraged to provide constructive feedback to any of the posted questions. This is an excellent way to contribute to the community and share your knowledge!

Read the Docs

Adafruit CircuitPython

2.x

Search docs

API AND USAGE

Core Modules

Support Matrix

Modules

- analogio — Analog hardware support
- audiobusio — Support for audio input and output over digital bus
- audioio** — Support for audio input and output
- bitbangio — Digital protocols implemented by the CPU

Docs » Core Modules » audioio — Support for audio input and output [Edit on GitHub](#)

audioio — Support for audio input and output

The `audioio` module contains classes to provide access to audio IO.

Libraries

- AudioOut** — Output an analog audio signal

All classes change hardware state and should be deinitialized when they are no longer needed if the program continues after use. To do so, either call `deinit()` or use a context manager. See [Lifetime and ContextManagers](#) for more info.

[Previous](#) [Next](#)

[Read the Docs \(https://adafru.it/Beg\)](https://adafru.it/Beg) is a an excellent resource for a more detailed look at the CircuitPython core and the CircuitPython libraries. This is where you'll find things like API documentation and example code. For an in depth look at viewing and understanding Read the Docs, check out the [CircuitPython Documentation \(https://adafru.it/VFx\)](https://adafru.it/VFx) page!

Here is blinky:

```
import time
import digitalio
import board

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT
while True:
    led.value = True
    time.sleep(0.1)
    led.value = False
    time.sleep(0.1)
```

CircuitPython Essentials



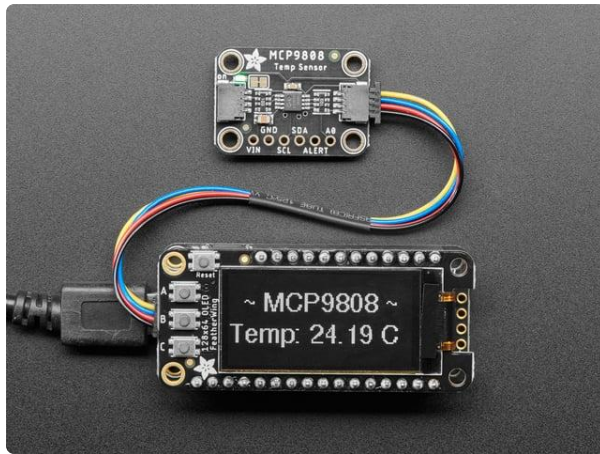
You've been introduced to CircuitPython, and worked through getting everything set up. What's next? CircuitPython Essentials!

There are a number of core modules built into CircuitPython, which can be used alongside the many CircuitPython libraries available. The following pages demonstrate some of these modules. Each page presents a different concept including a code example with an explanation. All of the examples are designed to work with your microcontroller board.

Time to get started learning the CircuitPython essentials!

Some examples require external components, such as switches or sensors. You'll find wiring diagrams where applicable to show you how to wire up the necessary components to work with each example.

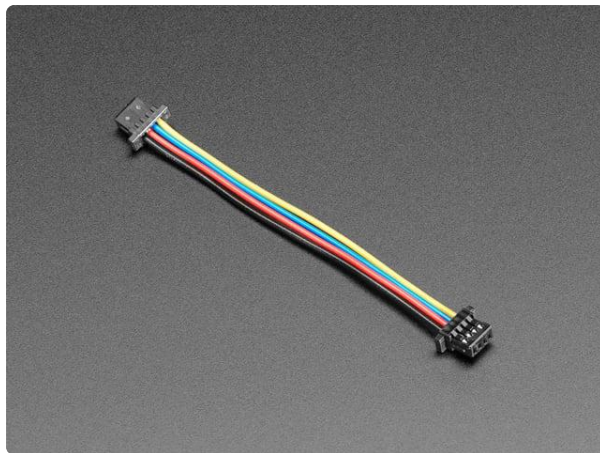
The following components are needed to complete all of the examples:



Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of $\pm 0.25^{\circ}\text{C}$ over the sensor's -40°C to...

<https://www.adafruit.com/product/5027>



STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

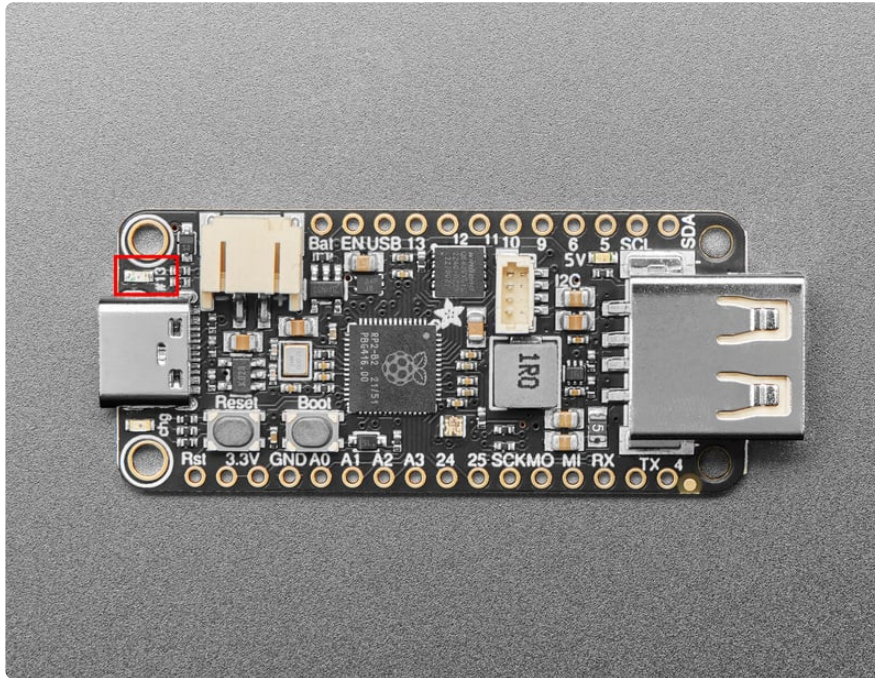
This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

<https://www.adafruit.com/product/4399>

Blink

In learning any programming language, you often begin with some sort of **Hello, World!** program. In CircuitPython, Hello, World! is blinking an LED. Blink is one of the simplest programs in CircuitPython. It involves three built-in modules, two lines of set up, and a short loop. Despite its simplicity, it shows you many of the basic concepts needed for most CircuitPython programs, and provides a solid basis for more complex projects. Time to get blinky!

LED Location

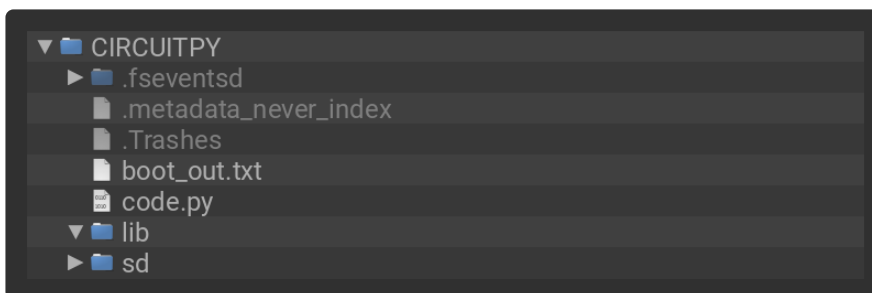


The red LED is above the USB-C connector on the left side of the board.

Blinking an LED

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/blink/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:



```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython Blink Example - the CircuitPython 'Hello, World!'"""
import time
import board
import digitalio

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

while True:
```

```
led.value = True
time.sleep(0.5)
led.value = False
time.sleep(0.5)
```

The built-in LED begins blinking!

Note that the code is a little less "Pythonic" than it could be. It could also be written as `led.value = not led.value` with a single `time.sleep(0.5)`. That way is more difficult to understand if you're new to programming, so the example is a bit longer than it needed to be to make it easier to read.

It's important to understand what is going on in this program.

First you `import` three modules: `time`, `board` and `digitalio`. This makes these modules available for use in your code. All three are built-in to CircuitPython, so you don't need to download anything to get started.

Next, you set up the LED. To interact with hardware in CircuitPython, your code must let the board know where to look for the hardware and what to do with it. So, you create a `digitalio.DigitalInOut()` object, provide it the LED pin using the `board` module, and save it to the variable `led`. Then, you tell the pin to act as an `OUTPUT`.

Finally, you create a `while True:` loop. This means all the code inside the loop will repeat indefinitely. Inside the loop, you set `led.value = True` which powers on the LED. Then, you use `time.sleep(0.5)` to tell the code to wait half a second before moving on to the next line. The next line sets `led.value = False` which turns the LED off. Then you use another `time.sleep(0.5)` to wait half a second before starting the loop over again.

With only a small update, you can control the blink speed. The blink speed is controlled by the amount of time you tell the code to wait before moving on using `time.sleep()`. The example uses `0.5`, which is one half of one second. Try increasing or decreasing these values to see how the blinking changes.

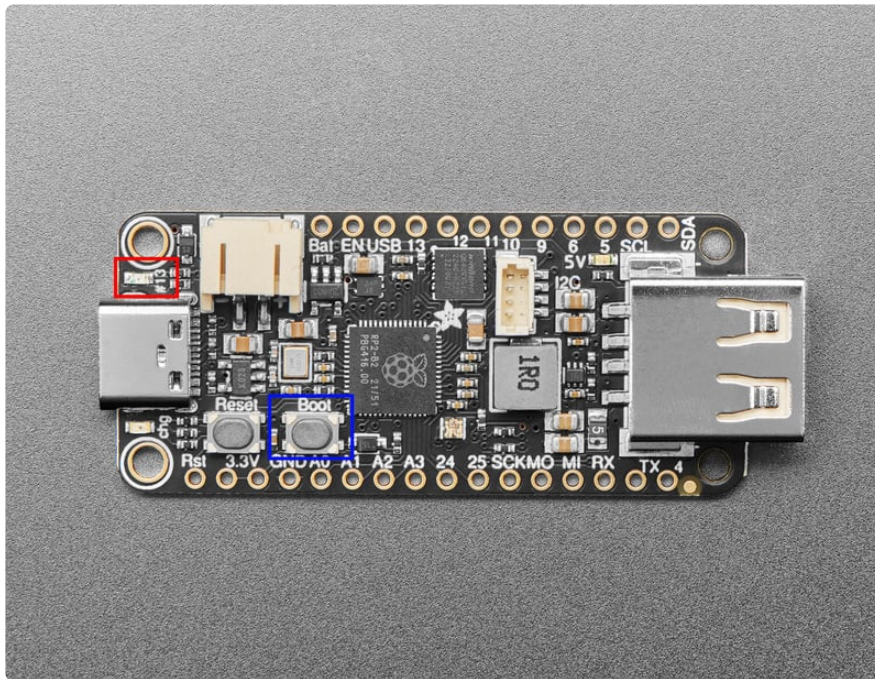
That's all there is to blinking an LED using CircuitPython!

Digital Input

The CircuitPython `digitalio` module has many applications. The basic Blink program sets up the LED as a digital output. You can just as easily set up a **digital input** such as a button to control the LED. This example builds on the basic Blink example, but now includes setup for a button switch. Instead of using the `time`

module to blink the LED, it uses the status of the button switch to control whether the LED is turned on or off.

LED and Button



- The **red LED** (highlighted in red above) is located above the USB-C connector.
- The **Boot button** (highlighted in blue above) is located to the right of the Reset button.

Controlling the LED with a Button

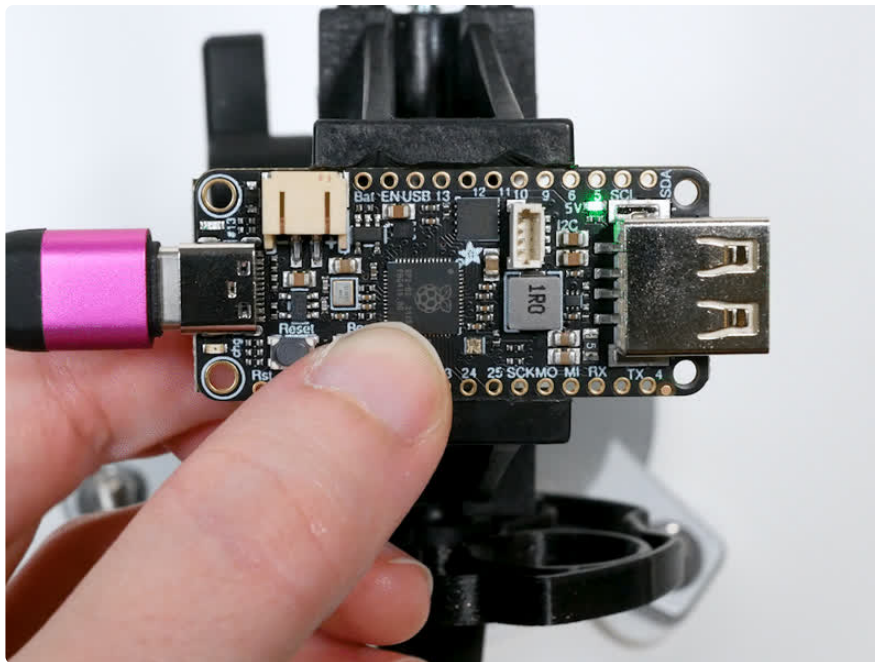
```
# SPDX-FileCopyrightText: 2022 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""
CircuitPython Digital Input Example - Blinking an LED using the built-in button.
"""
import board
import digitalio

led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

button = digitalio.DigitalInOut(board.BUTTON)
button.switch_to_input(pull=digitalio.Pull.UP)

while True:
    if not button.value:
        led.value = True
    else:
        led.value = False
```

Now, press the button. The LED lights up! Let go of the button and the LED turns off.



Note that the code is a little less "Pythonic" than it could be. It could also be written as `led.value = not button.value`. That way is more difficult to understand if you're new to programming, so the example is a bit longer than it needed to be to make it easier to read.

First you `import` two modules: `board` and `digitalio`. This makes these modules available for use in your code. Both are built-in to CircuitPython, so you don't need to download anything to get started.

Next, you set up the LED. To interact with hardware in CircuitPython, your code must let the board know where to look for the hardware and what to do with it. So, you create a `digitalio.DigitalInOut()` object, provide it the LED pin using the `board` module, and save it to the variable `led`. Then, you tell the pin to act as an `OUTPUT`.

You include setup for the button as well. It is similar to the LED setup, except the button is an `INPUT`, and requires a pull up.

Inside the loop, you check to see if the button is pressed, and if so, turn on the LED. Otherwise the LED is off.

That's all there is to controlling an LED with a button switch!

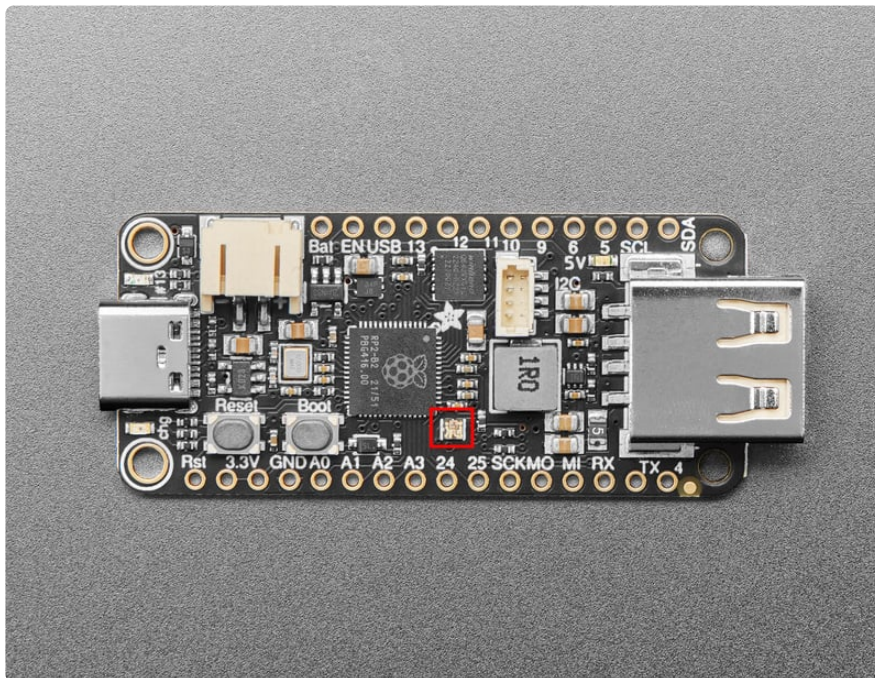
NeoPixel

Your board has a built-in RGB NeoPixel status LED. You can use CircuitPython code to control the color and brightness of this LED. It is also used to indicate the bootloader status and errors in your CircuitPython code.

A NeoPixel is what Adafruit calls the WS281x family of addressable RGB LEDs. It contains three LEDs - a red one, a green one and a blue one - along side a driver chip in a tiny package controlled by a single pin. They can be used individually (as in the built-in LED on your board), or chained together in strips or other creative form factors. NeoPixels do not light up on their own; they require a microcontroller. So, it's super convenient that the NeoPixel is built in to your microcontroller board!

This page will cover using CircuitPython to control the status RGB NeoPixel built into your microcontroller. You'll learn how to change the color and brightness, and how to make a rainbow. Time to get started!

NeoPixel Location



The **NeoPixel LED** is located above the D24 and D25 pin labels.

NeoPixel Color and Brightness

To use with CircuitPython, you need to first install a few libraries, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/status_led_one_neopixel_rgb/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:

```

▼ CIRCUITPY
  ► .fsevents
  .metadata_never_index
  .Trashes
  boot_out.txt
  code.py
  ▼ lib
    adafruit_pixelbuf.mpy
    neopixel.mpy
  ► sd

```

```

# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython status NeoPixel red, green, blue example."""
import time
import board
import neopixel

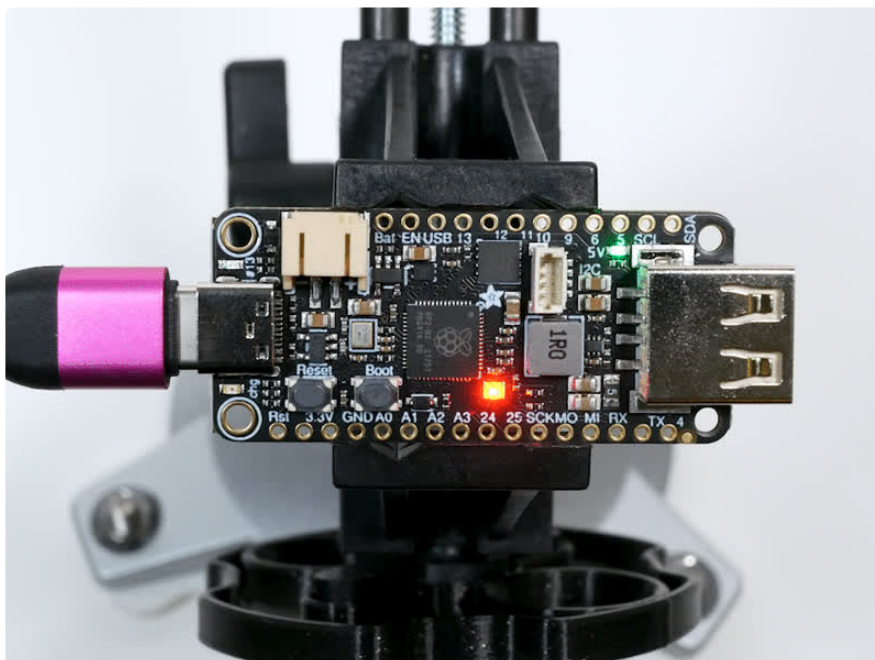
pixel = neopixel.NeoPixel(board.NEOPIXEL, 1)

pixel.brightness = 0.3

while True:
    pixel.fill((255, 0, 0))
    time.sleep(0.5)
    pixel.fill((0, 255, 0))
    time.sleep(0.5)
    pixel.fill((0, 0, 255))
    time.sleep(0.5)

```

The built-in NeoPixel begins blinking red, then green, then blue, and repeats!



First you import two modules, `time` and `board`, and one library, `neopixel`. This makes these modules and libraries available for use in your code. The first two are modules built-in to CircuitPython, so you don't need to download anything to use

those. The `neopixel` library is separate, which is why you needed to install it before getting started.

Next, you set up the NeoPixel LED. To interact with hardware in CircuitPython, your code must let the board know where to look for the hardware and what to do with it. So, you create a `neopixel.NeoPixel()` object, provide it the NeoPixel LED pin using the `board` module, and tell it the number of LEDs. You save this object to the variable `pixel`.

Then, you set the NeoPixel brightness using the `brightness` attribute. `brightness` expects float between `0` and `1.0`. A float is essentially a number with a decimal in it. The brightness value represents a percentage of maximum brightness; `0` is 0% and `1.0` is 100%. Therefore, setting `pixel.brightness = 0.3` sets the brightness to 30%. The default brightness, which is to say the brightness if you don't explicitly set it, is `1.0`. The default is really bright! That is why there is an option available to easily change the brightness.

Inside the loop, you turn the NeoPixel red for 0.5 seconds, green for 0.5 seconds, and blue for 0.5 seconds.

To turn the NeoPixel red, you "fill" it with an RGB value. Check out the section below for details on RGB colors. The RGB value for red is `(255, 0, 0)`. Note that the RGB value includes the parentheses. The `fill()` attribute expects the full RGB value including those parentheses. That is why there are two pairs of parentheses in the code.

You can change the RGB values to change the colors that the NeoPixel cycles through. Check out the list below for some examples. You can make any color of the rainbow with the right RGB value combination!

That's all there is to changing the color and setting the brightness of the built-in NeoPixel LED!

RGB LED Colors

RGB LED colors are set using a combination of red, green, and blue, in the form of an **(R, G, B)** tuple. Each member of the tuple is set to a number between 0 and 255 that determines the amount of each color present. Red, green and blue in different combinations can create all the colors in the rainbow! So, for example, to set an LED to red, the tuple would be `(255, 0, 0)`, which has the maximum level of red, and no green or blue. Green would be `(0, 255, 0)`, etc. For the colors between, you set a combination, such as cyan which is `(0, 255, 255)`, with equal amounts of green and blue. If you increase all values to the same level, you get white! If you decrease all the values to 0, you turn the LED off.

Common colors include:

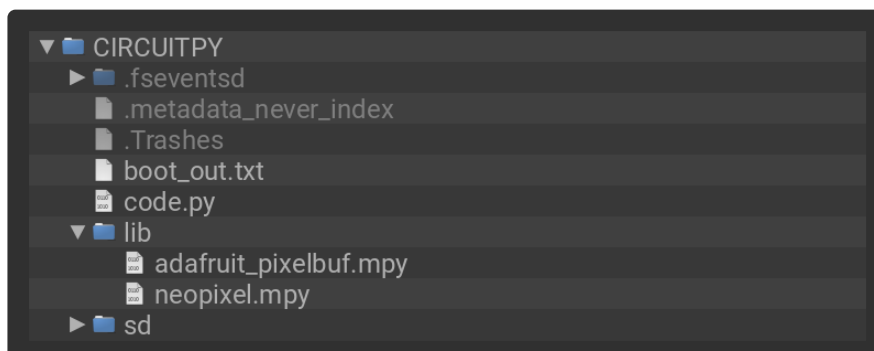
- red: (255, 0, 0)
- green: (0, 255, 0)
- blue: (0, 0, 255)
- cyan: (0, 255, 255)
- purple: (255, 0, 255)
- yellow: (255, 255, 0)
- white: (255, 255, 255)
- black (off): (0, 0, 0)

NeoPixel Rainbow

You should have already installed the library necessary to use the built-in NeoPixel LED. If not, follow the steps at the beginning of the NeoPixel Color and Brightness section to install it.

In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, open the directory **CircuitPython_Templates/status_led_one_neopixel_rainbow/** and then click on the directory that matches the version of CircuitPython you're using and copy the contents of that directory to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:



```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython status NeoPixel rainbow example."""
import time
import board
from rainbowio import colorwheel
import neopixel

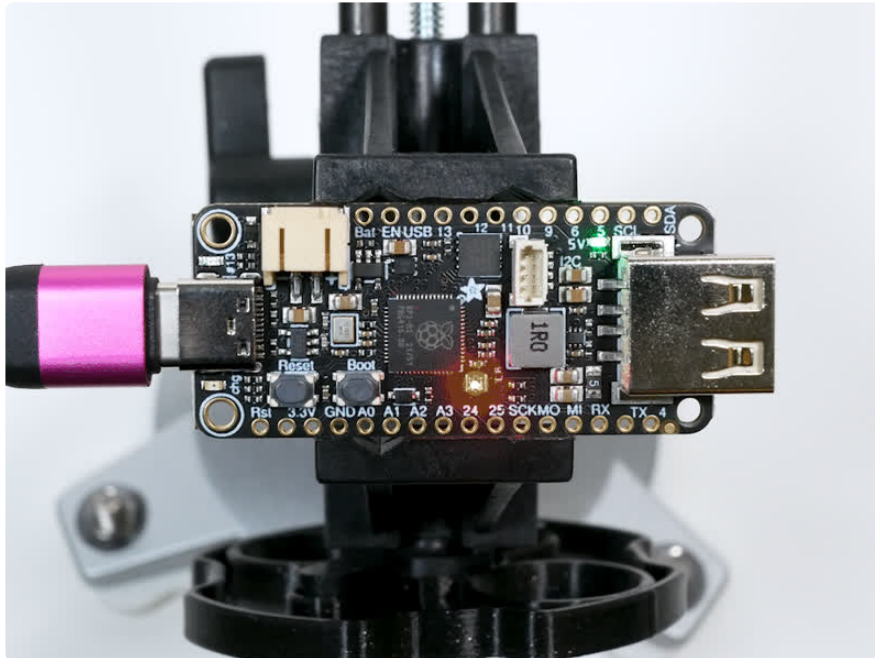
pixel = neopixel.NeoPixel(board.NEOPIXEL, 1)
pixel.brightness = 0.3

def rainbow(delay):
    for color_value in range(255):
        pixel[0] = colorwheel(color_value)
```

```
time.sleep(delay)

while True:
    rainbow(0.02)
```

The NeoPixel displays a rainbow cycle!



This example builds on the previous example.

First, you import the same three modules and libraries. In addition to those, you import `colorwheel`.

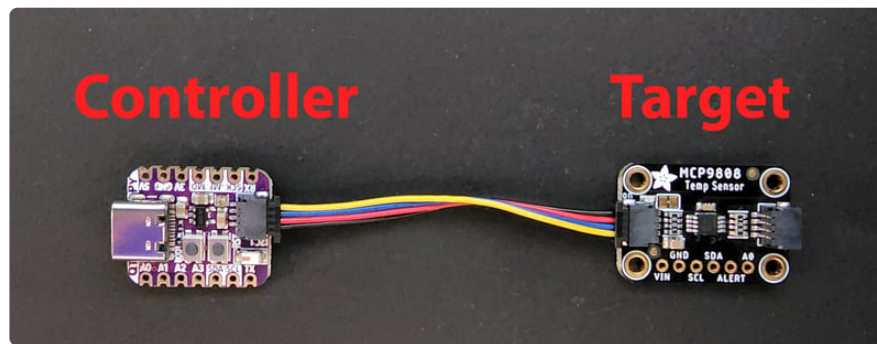
The NeoPixel hardware setup and brightness setting are the same.

Next, you have the `rainbow()` helper function. This helper displays the rainbow cycle. It expects a `delay` in seconds. The higher the number of seconds provided for `delay`, the slower the rainbow will cycle. The helper cycles through the values of the color wheel to create a rainbow of colors.

Inside the loop, you call the rainbow helper with a 0.2 second delay, by including `rainbow(0.2)`.

That's all there is to making rainbows using the built-in NeoPixel LED!

I2C



The **I2C**, or [inter-integrated circuit](https://adafruit.it/u2a) (<https://adafruit.it/u2a>), is a 2-wire protocol for communicating with simple sensors and devices, which means it uses two connections, or wires, for transmitting and receiving data. One connection is a clock, called **SCL**. The other is the data line, called **SDA**. Each pair of clock and data pins are referred to as a **bus**.

Typically, there is a device that acts as a **controller** and sends requests to the **target** devices on each bus. In this case, your microcontroller board acts as the controller, and the sensor breakout acts as the target. Historically, the controller is referred to as the master, and the target is referred to as the slave, so you may run into that terminology elsewhere. The official terminology is [controller and target](https://adafruit.it/TtF) (<https://adafruit.it/TtF>).

Multiple I2C devices can be connected to the same clock and data lines. Each I2C device has an address, and as long as the addresses are different, you can connect them at the same time. This means you can have many different sensors and devices all connected to the same two pins.

Both I2C connections require pull-up resistors, and most Adafruit I2C sensors and breakouts have pull-up resistors built in. If you're using one that does not, you'll need to add your own 2.2-10k Ω pull-up resistors from SCL and SDA to 3.3V.

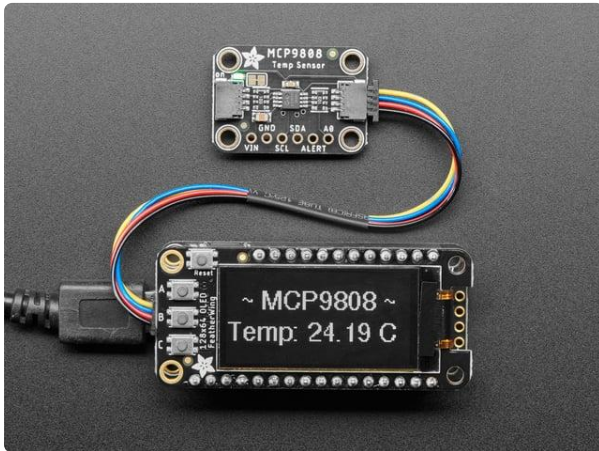
I2C and CircuitPython

CircuitPython supports many I2C devices, and makes it super simple to interact with them. There are libraries available for many I2C devices in the [CircuitPython Library Bundle](https://adafruit.it/Tra) (<https://adafruit.it/Tra>). (If you don't see the sensor you're looking for, keep checking back, more are being written all the time!)

In this section, you'll learn how to scan the I2C bus for all connected devices. Then you'll learn how to interact with an I2C device.

Necessary Hardware

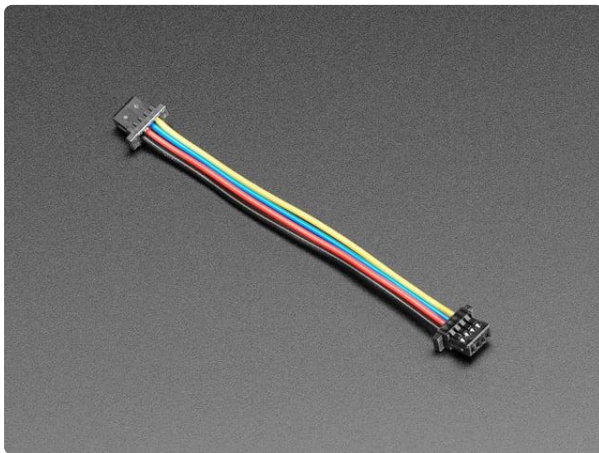
You'll need the following additional hardware to complete the examples on this page.



[Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout](https://www.adafruit.com/product/5027)

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of $\pm 0.25^{\circ}\text{C}$ over the sensor's -40°C to...

<https://www.adafruit.com/product/5027>



[STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long](https://www.adafruit.com/product/4399)

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

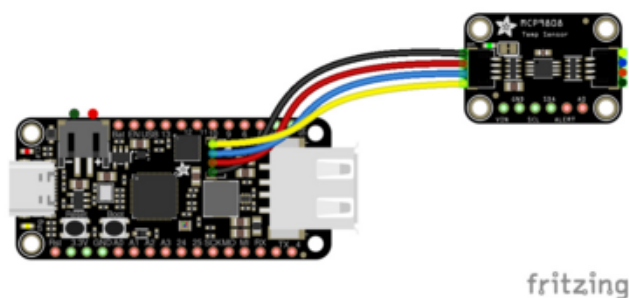
<https://www.adafruit.com/product/4399>

While the examples here will be using the [Adafruit MCP9808 \(http://adafru.it/5027\)](http://adafru.it/5027), a high accuracy temperature sensor, the overall process is the same for just about any I2C sensor or device.

The first thing you'll want to do is get the sensor connected so your board has I2C to talk to.

Wiring the MCP9808

The MCP9808 comes with a STEMMA QT connector, which makes wiring it up quite simple and solder-free.



Connect the STEMMA QT cable from the **STEMMA QT** port on your Feather to the **STEMMA QT** port on the MCP9808.

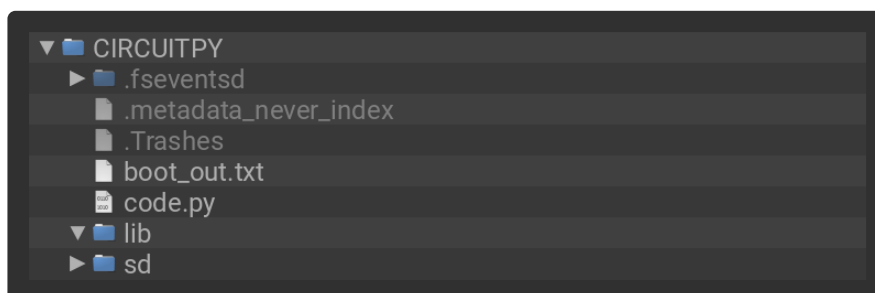
Find Your Sensor

The first thing you'll want to do after getting the sensor wired up, is make sure it's wired correctly. You're going to do an I2C scan to see if the board is detected, and if it is, print out its I2C address.

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your CircuitPython version, and copy the matching **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:



```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython I2C Device Address Scan"""
import time
import board

i2c = board.I2C() # uses board.SCL and board.SDA
# i2c = board.STEMMA_I2C() # For using the built-in STEMMA QT connector on a
# microcontroller

# To create I2C bus on specific pins
# import busio
# i2c = busio.I2C(board.GP1, board.GP0) # Pi Pico RP2040

while not i2c.try_lock():
    pass

try:
```

```

while True:
    print(
        "I2C addresses found:",
        [hex(device_address) for device_address in i2c.scan()],
    )
    time.sleep(2)

finally: # unlock the i2c bus when ctrl-c'ing out of the loop
    i2c.unlock()

```

```

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
I2C addresses found: ['0x18']

```

If you run this and it seems to hang, try manually unlocking your I2C bus by running the following two commands from the REPL.

```

import board
board.I2C().unlock()

```

First you create the `i2c` object, using `board.I2C()`. This convenience routine creates and saves a `busio.I2C` object using the default pins `board.SCL` and `board.SDA`. If the object has already been created, then the existing object is returned. No matter how many times you call `board.I2C()`, it will return the same object. This is called a singleton.

To be able to scan it, you need to lock the I2C down so the only thing accessing it is the code. So next you include a loop that waits until I2C is locked and then continues on to the scan function.

Last, you have the loop that runs the actual scan, `i2c_scan()`. Because I2C typically refers to addresses in hex form, the example includes this bit of code that formats the results into hex format: `[hex(device_address) for device_address in i2c.scan()]`.

Open the serial console to see the results! The code prints out an array of addresses. You've connected the MCP9808 which has a 7-bit I2C address of 0x18. The result for this sensor is `I2C addresses found: ['0x18']`. If no addresses are returned, refer back to the wiring diagrams to make sure you've wired up your sensor correctly.

I2C Sensor Data

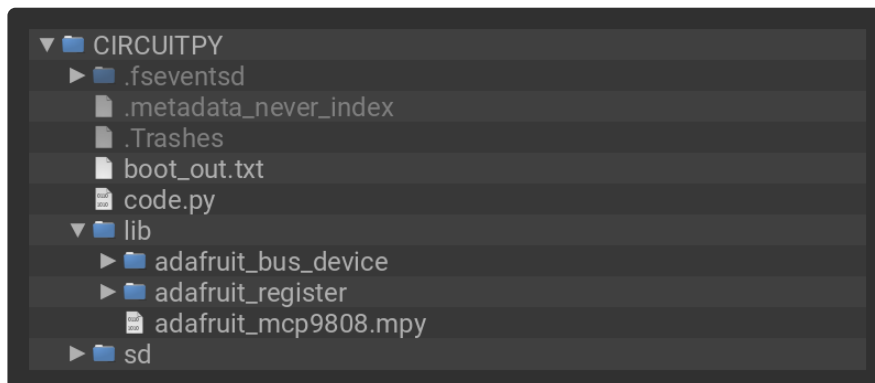
Now you know for certain that your sensor is connected and ready to go. Time to find out how to get the data from the sensor!

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your

CircuitPython version, and copy the matching **entire lib folder** and **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:



```
# SPDX-FileCopyrightText: 2021 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython I2C MCP9808 Temperature Sensor Example"""
import time
import board
import adafruit_mcp9808

i2c = board.I2C() # uses board.SCL and board.SDA
# i2c = board.STEMMA_I2C() # For using the built-in STEMMA QT connector on a
microcontroller
# import busio
# i2c = busio.I2C(board.SCL1, board.SDA1) # For QT Py RP2040, QT Py ESP32-S2
mcp9808 = adafruit_mcp9808.MCP9808(i2c)

while True:
    temperature_celsius = mcp9808.temperature
    temperature_fahrenheit = temperature_celsius * 9 / 5 + 32
    print("Temperature: {:.2f} C {:.2f} F ".format(temperature_celsius,
temperature_fahrenheit))
    time.sleep(2)
```

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disab
le.
code.py output:
Temperature: 23.38 C 74.07 F
```

The Feather RP2040 with USB Type A Host STEMMA QT connector is available on `board.STEMMA_I2C()`. Comment out the current `i2c` setup line, and uncomment the the `i2c = board.STEMMA_I2C()` line to use with your board's STEMMA QT connector.

This code begins the same way as the scan code, except this time, you create your sensor object using the sensor library. You call it `mcp9808` and provide it the `i2c` object.

Then you have a simple loop that prints out the temperature reading using the sensor object you created. Finally, there's a `time.sleep(2)`, so it only prints once every two

seconds. Connect to the serial console to see the results. Try touching the MCP9808 with your finger to see the values change!

Where's my I2C?

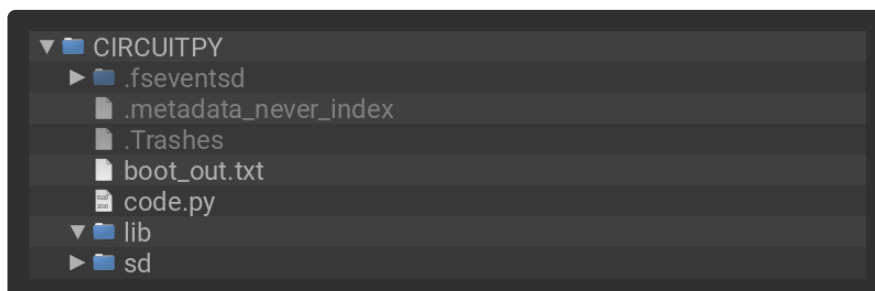
On many microcontrollers, you have the flexibility of using a wide range of pins for I2C. On some types of microcontrollers, any pin can be used for I2C! Other chips require using bitbangio, but can also use any pins for I2C. There are further microcontrollers that may have fixed I2C pins.

Given the many different types of microcontroller boards available, it's impossible to guarantee anything other than the labeled 'SDA' and 'SCL' pins. So, if you want some other setup, or multiple I2C interfaces, how will you find those pins? Easy! Below is a handy script.

Save the following to your **CIRCUITPY** drive as **code.py**.

Click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, find your CircuitPython version, and copy the matching **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY** drive should now look similar to the following image:



```
# SPDX-FileCopyrightText: 2021-2023 Kattni Rembor for Adafruit Industries
# SPDX-License-Identifier: MIT
"""CircuitPython I2C possible pin-pair identifying script"""
import board
import busio
from microcontroller import Pin

def is_hardware_i2c(scl, sda):
    try:
        p = busio.I2C(scl, sda)
        p.deinit()
        return True
    except ValueError:
        return False
    except RuntimeError:
        return True

def get_unique_pins():
    exclude = [
        getattr(board, p)
    ]
```

```

for p in [
    # This is not an exhaustive list of unexposed pins. Your results
    # may include other pins that you cannot easily connect to.
    "NEOPIXEL",
    "DOTSTAR_CLOCK",
    "DOTSTAR_DATA",
    "APA102_SCK",
    "APA102_MOSI",
    "LED",
    "SWITCH",
    "BUTTON",
    "ACCELEROMETER_INTERRUPT",
    "VOLTAGE_MONITOR",
    "MICROPHONE_CLOCK",
    "MICROPHONE_DATA",
    "RFM_RST",
    "RFM_CS",
    "RFM_IO0",
    "RFM_IO1",
    "RFM_IO2",
    "RFM_IO3",
    "RFM_IO4",
    "RFM_IO5",
    "TFT_I2C_POWER",
    "NEOPIXEL_POWER",
]
if p in dir(board)
]
pins = [
    pin
    for pin in [getattr(board, p) for p in dir(board)]
    if isinstance(pin, Pin) and pin not in exclude
]
unique = []
for p in pins:
    if p not in unique:
        unique.append(p)
return unique

for scl_pin in get_unique_pins():
    for sda_pin in get_unique_pins():
        if scl_pin is sda_pin:
            continue
        if is_hardware_i2c(scl_pin, sda_pin):
            print("SCL pin:", scl_pin, "\t SDA pin:", sda_pin)

```

Now, connect to the serial console and check out the output! The results print out a nice handy list of SCL and SDA pin pairs that support I2C.

The output for the Feather is extremely long! The screenshot shows only the beginning. Run the script yourself to see the full output!

```
CircuitPython REPL
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
SCL pin: board.A1    SDA pin: board.A0
SCL pin: board.A1    SDA pin: board.D10
SCL pin: board.A1    SDA pin: board.D6
SCL pin: board.A1    SDA pin: board.SCK
SCL pin: board.A1    SDA pin: board.SDA
SCL pin: board.A1    SDA pin: board.USB_HOST_5V_POWER
SCL pin: board.A3    SDA pin: board.A2
SCL pin: board.A3    SDA pin: board.TX
SCL pin: board.A3    SDA pin: board.D12
```

This example only runs once, so if you do not see any output when you connect to the serial console, try CTRL+D to reload.

USB Host Device Info

The main use case for this Feather is to use it as a USB host device. In the example below, you'll use scan for an attached USB device and print out its descriptors using the [Adafruit_CircuitPython_USB_Host_Descriptors](https://adafru.it/1a83) (<https://adafru.it/1a83>) library.

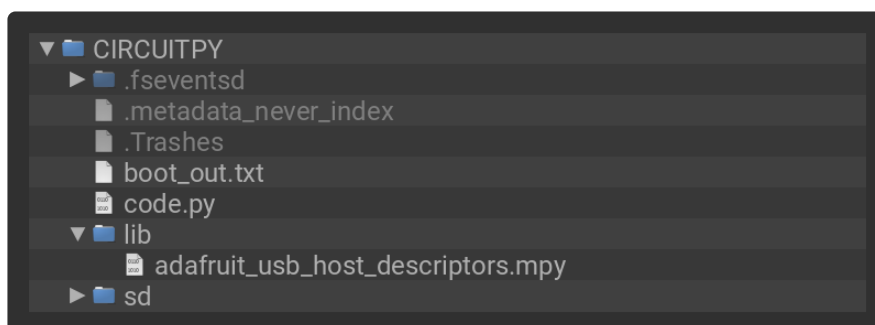
CircuitPython Usage

To use with CircuitPython, you need to first install the [USB host descriptors library](https://adafru.it/1a83) (<https://adafru.it/1a83>) into the **lib** folder onto your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following file:

- **adafruit_usb_host_descriptors.mpy**



Example Code

```
# SPDX-FileCopyrightText: 2017 Scott Shawcroft, written for Adafruit Industries
# SPDX-FileCopyrightText: Copyright (c) 2023 Scott Shawcroft for Adafruit Industries
#
# SPDX-License-Identifier: Unlicense

import time
import usb.core

import adafruit_usb_host_descriptors

DIR_IN = 0x80

while True:
    print("searching for devices")
    for device in usb.core.find(find_all=True):
        print("pid", hex(device.idProduct))
        print("vid", hex(device.idVendor))
        print("man", device.manufacturer)
        print("product", device.product)
        print("serial", device.serial_number)
        print("config[0]:")
        config_descriptor =
adafruit_usb_host_descriptors.get_configuration_descriptor(
    device, 0
)

        i = 0
        while i < len(config_descriptor):
            descriptor_len = config_descriptor[i]
            descriptor_type = config_descriptor[i + 1]
            if descriptor_type == adafruit_usb_host_descriptors.DESC_CONFIGURATION:
                config_value = config_descriptor[i + 5]
                print(f" value {config_value:d}")
            elif descriptor_type == adafruit_usb_host_descriptors.DESC_INTERFACE:
                interface_number = config_descriptor[i + 2]
                interface_class = config_descriptor[i + 5]
                interface_subclass = config_descriptor[i + 6]
                print(f" interface[{interface_number:d}]")
                print(
                    f"   class {interface_class:02x} subclass {interface_subclass:
02x}"
                )
            elif descriptor_type == adafruit_usb_host_descriptors.DESC_ENDPOINT:
                endpoint_address = config_descriptor[i + 2]
                if endpoint_address & DIR_IN:
                    print(f"   IN {endpoint_address:02x}")
                else:
                    print(f"   OUT {endpoint_address:02x}")
                i += descriptor_len
            print()
            time.sleep(5)
```

Plug in a USB device into the USB A host port on the Feather. You'll see its descriptor information print to the serial console.

```
searching for devices
pid 0xc21d
vid 0x46d
man Logitech
product Gamepad F310
serial EA42C34E
config[0]:
  value 1
  interface[0]
    class ff subclass 5d
    IN 81
    OUT 02
```

USB Host Read Data

Once you've got a USB device connected and you've gotten its device info, the next thing to do is start reading data from it. This example will illustrate how to read raw data packets from HID devices like generic USB game pads.

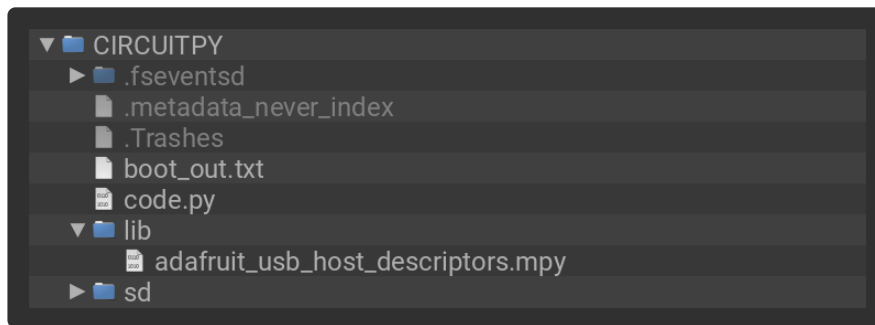


To use with CircuitPython, you need to first install the [USB host descriptors library](https://adafru.it/1a83) (<https://adafru.it/1a83>) into the **lib** folder onto your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the `code.py` file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the `code.py` file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following file:

- `adafruit_usb_host_descriptors.mpy`



Example Code

```
# SPDX-FileCopyrightText: Copyright (c) 2025 Tim Cocks for Adafruit Industries
#
# SPDX-License-Identifier: MIT
import array
import time
import usb.core
import adafruit_usb_host_descriptors

# Set to true to print detailed information about all devices found
VERBOSE_SCAN = True

# indexes within the reports to ignore when determining equality.
# some devices send alternating values with each report, this
# allows to ignore those and focus only on bytes that are
# affected by buttons. Value of [19] will ignore data at index 19.
# Check your own output for values that change even when you don't
# do anything on the controller and add their indexes here.
IGNORE_INDEXES = []

DIR_IN = 0x80
controller = None

if VERBOSE_SCAN:
    for device in usb.core.find(find_all=True):
        controller = device
        print("pid", hex(device.idProduct))
        print("vid", hex(device.idVendor))
        print("man", device.manufacturer)
        print("product", device.product)
        print("serial", device.serial_number)
        print("config[0]:")
        config_descriptor =
adafruit_usb_host_descriptors.get_configuration_descriptor(
    device, 0
)

    i = 0
    while i < len(config_descriptor):
```

```

        descriptor_len = config_descriptor[i]
        descriptor_type = config_descriptor[i + 1]
        if descriptor_type == adafruit_usb_host_descriptors.DESC_CONFIGURATION:
            config_value = config_descriptor[i + 5]
            print(f" value {config_value:d}")
        elif descriptor_type == adafruit_usb_host_descriptors.DESC_INTERFACE:
            interface_number = config_descriptor[i + 2]
            interface_class = config_descriptor[i + 5]
            interface_subclass = config_descriptor[i + 6]
            print(f" interface[{interface_number:d}]")
            print(
                f"   class {interface_class:02x} subclass {interface_subclass:
02x}"
            )
        elif descriptor_type == adafruit_usb_host_descriptors.DESC_ENDPOINT:
            endpoint_address = config_descriptor[i + 2]
            if endpoint_address & DIR_IN:
                print(f"   IN {endpoint_address:02x}")
            else:
                print(f"   OUT {endpoint_address:02x}")
        i += descriptor_len

# get the first device found
device = None
while device is None:
    for d in usb.core.find(find_all=True):
        device = d
        break
    time.sleep(0.1)

# set configuration so we can read data from it
device.set_configuration()
print(f"configuration set for {device.manufacturer}, {device.product},
{device.serial_number}")

# Test to see if the kernel is using the device and detach it.
if device.is_kernel_driver_active(0):
    device.detach_kernel_driver(0)

# buffer to hold 64 bytes
buf = array.array("B", [0] * 64)

def print_array(arr, max_index=None, fmt="hex"):
    """
    Print the values of an array
    :param arr: The array to print
    :param max_index: The maximum index to print. None means print all.
    :param fmt: The format to use, either "hex" or "bin"
    :return: None
    """
    out_str = ""
    if max_index is None or max_index >= len(arr):
        length = len(arr)
    else:
        length = max_index

    for _ in range(length):
        if fmt == "hex":
            out_str += f"{int(arr[_]):02x} "
        elif fmt == "bin":
            out_str += f"{int(arr[_]):08b} "
    print(out_str)

def reports_equal(report_a, report_b):
    """
    Test if two reports are equal. Accounting for any IGNORE_INDEXES

```



```

:param report_a: First report data
:param report_b: Second report data
:return: True if the reports are equal, otherwise False.
"""
if report_a is None and report_b is not None or \
    report_b is None and report_a is not None:
    return False
for _ in range(len(report_a)):
    if IGNORE_INDEXES is not None and _ not in IGNORE_INDEXES:
        if report_a[_] != report_b[_]:
            return False
return True

idle_state = None
prev_state = None

while True:
    try:
        count = device.read(0x81, buf)
        # print(f"read size: {count}")
    except usb.core.USBTimeoutError:
        continue

    if idle_state is None:
        idle_state = buf[:]
        print("Idle state:")
        print_array(idle_state, max_index=count)
        print()

    if not reports_equal(buf, prev_state) and not reports_equal(buf, idle_state):
        print_array(buf, max_index=count)

    prev_state = buf[:]

```

Plug in a USB device to the USB A host port on the Feather and run this example code. If `VERBOSE_SCAN` is `True` you'll see its device info. Then the code will read data from the USB device and print it out when it changes. Once the code is running, start pressing buttons on the controller to observe the different values being printed with each button or combination.

```

code.py output:
pid 0x575
vid 0x2563
man SHANWAN
product PS3 Gamepad
serial 000000000001
config[0]:
  value 1
  interface[0]
    class 03 subclass 00
    IN 01
    OUT 02
configuration set for SHANWAN, PS3 Gamepad, 000000000001
Idle state:
00 00 0f 80 80 80 80 00 00 00 00 00 00 00 00 00 00 02 80 01 00 02 00 02

00 00 00 80 80 80 80 00 00 ff 00 00 00 00 00 00 00 00 02 80 01 00 02 00 02
00 00 04 80 80 80 80 00 00 ff 00 00 00 00 00 00 00 00 01 02 80 01 00 02 00 02
00 00 06 80 80 80 80 00 ff 00 00 00 00 00 00 00 00 01 02 80 01 00 02 00 02
00 00 02 80 80 80 80 ff 00 00 00 00 00 00 00 00 00 01 02 80 01 00 02 00 02

```

Arduino IDE Setup

The [Arduino Philhower core \(https://adafru.it/ToC\)](https://adafru.it/ToC) provides support for RP2040 microcontroller boards. This page covers getting your Arduino IDE set up to include your board.

Arduino IDE Download

The first thing you will need to do is to download the latest release of the Arduino IDE. The Philhower core requires **version 1.8** or higher.

Arduino IDE Download

<https://adafru.it/Pd5>

Download and install it to your computer.

Once installed, open the Arduino IDE.

Adding the Philhower Board Manager URL

In the Arduino IDE, and navigate to the **Preferences** window. You can access it through **File > Preferences** on Windows or Linux, or **Arduino > Preferences** on OS X.

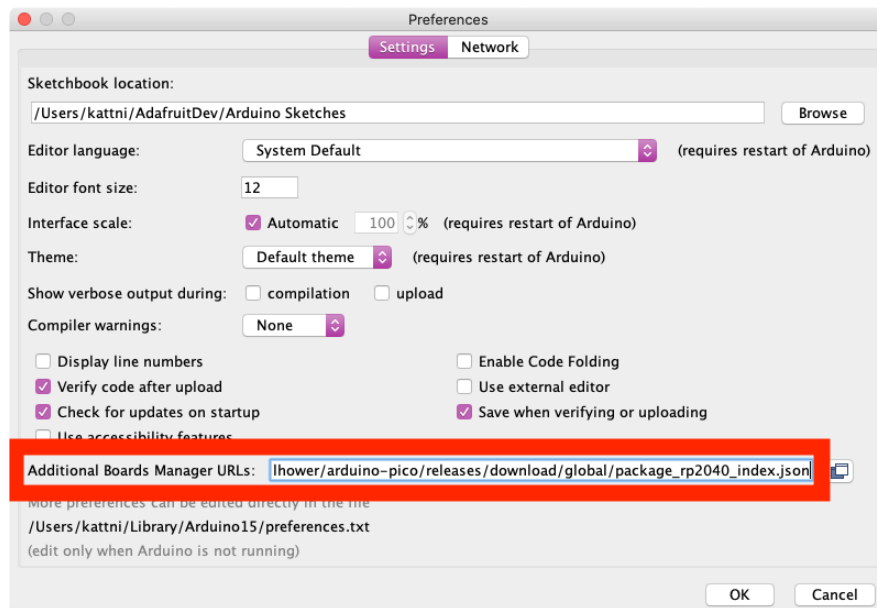
The **Preferences** window will open.

In the **Additional Boards Manager URLs** field, you'll want to add a new URL. The list of URLs is comma separated, and you will only have to add each URL once. The URLs point to index files that the Board Manager uses to build the list of available & installed boards.

Copy the following URL.

https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json

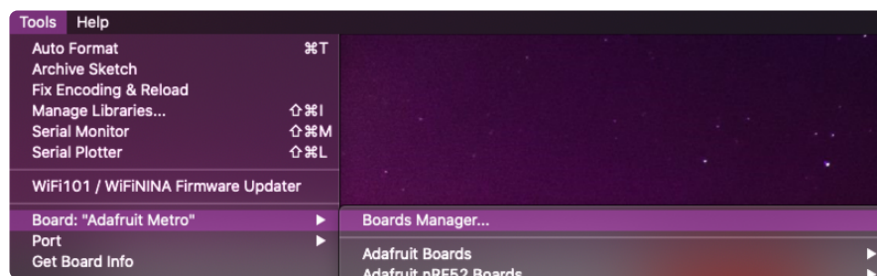
Add the URL to the the **Additional Boards Manager URLs** field (highlighted in red below).



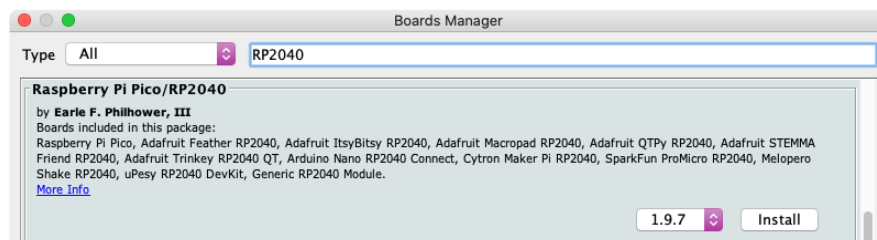
Click **OK** to save and close **Preferences**.

Add Board Support Package

In the Arduino IDE, click on **Tools > Board > Boards Manager**. If you have previously selected a board, the **Board** menu item may have a board name after it.



In the **Boards Manager**, search for RP2040. Scroll down to the **Raspberry Pi Pico/ RP2040** by Earle F Philhower, III entry. Click **Install** to install it.

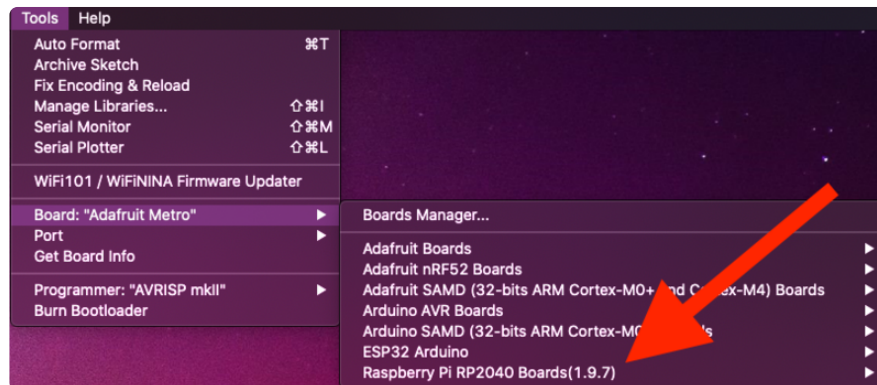


Installing a new board package can take a few minutes. Don't click Cancel!

Once installation is complete, click **Close** to close the Boards Manager.

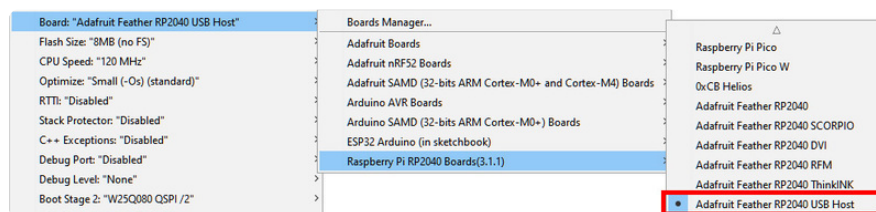
Choose Your Board

In the **Tools > Boards** menu, you should now see **Raspberry Pi RP2040 Boards** (possibly followed by a version number).



Navigate to the **Raspberry Pi RP2040 Boards** menu. You will see the available boards listed.

Navigate to the **Raspberry Pi RP2040 Boards** menu and choose **Adafruit Feather RP2040 USB Host**.



If there is no serial Port available in the dropdown, or an invalid one appears - don't worry about it! The RP2040 does not actually use a serial port to upload, so its OK if it does not appear if in manual bootload mode. You will see a serial port appear after uploading your first sketch.

Now you're ready to begin using Arduino with your RP2040 board!

Troubleshooting

If you have any strange errors after updating the **Raspberry Pi Pico/RP2040 by Earle F Philhower, III** board support package (BSP) from the boards manager you may need to start with a fresh install of the BSP. Close out of the Arduino IDE and navigate to your Arduino packages folder: **C:\Users[username]**

\AppData\Local\Arduino15\packages and delete the **/rp2040** folder. After that, open

the Arduino IDE and follow the steps above for installing the BSP. The errors should not occur.

Arduino Usage

Now that you've set up the Arduino IDE with the Philhower RP2040 Arduino core, you're ready to start using Arduino with your RP2040.

RP2040 Arduino Pins

There is no pin remapping for Arduino on the RP2040. Therefore, the pin names on the top of the board are **not** the pin names used for Arduino. The Arduino pin names are the RP2040 GPIO pin names.

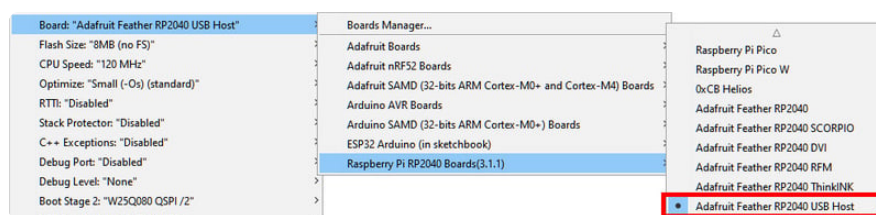
To find the Arduino pin name, check the PrettyPins diagram found on the Pinouts page. Each GPIO pin in the diagram has a **GPIOx** pin name listed, where **x** is the pin number. The Arduino pin name is the number following **GPIO**. For example, **GPIO1** would be Arduino pin **1**.

The Feather RP2040 USB Host has the GPIO pin names listed on the back of the board as **x**, where **x** is the pin number. So, for example, pin **5** would be Arduino pin **5**.

Choose Your Board

Navigate to the **Tools > Boards > Raspberry Pi RP2040 Boards** menu. The Raspberry Pi RP2040 Boards menu name may be followed by a version number.

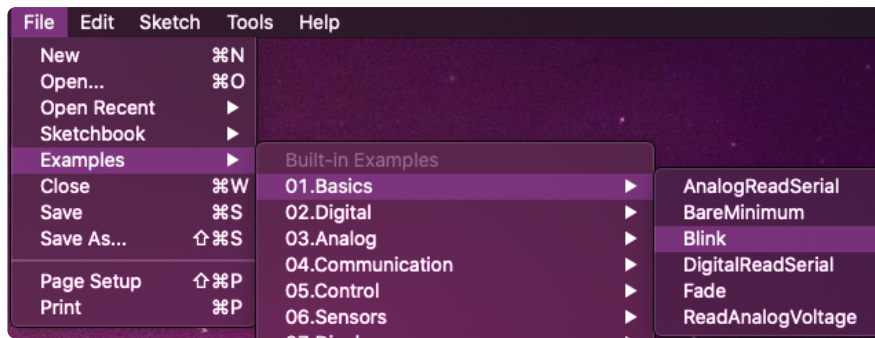
Choose **Feather RP2040 USB Host** from the menu.



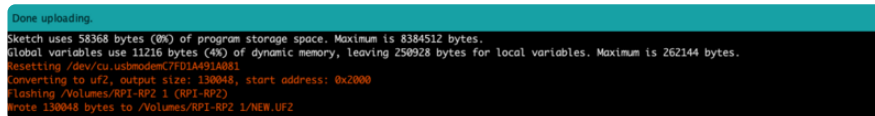
Load the Blink Sketch

Begin by plugging in your board to your computer, and wait a moment for it to be recognised by the OS. It will create a COM/serial port that you can now select from the **Tools > Port** menu dropdown.

Open the Blink sketch by clicking through **File > Examples > 01.Basics > Blink**.



Click Upload. A successful upload will result in text similar to the following.



Once complete, the little red LED will begin blinking once every second! Try changing up the `delay()` timing to change the rate at which the LED blinks.

Manually Enter the Bootloader

If you get into a state with the bootloader where you can no longer upload a sketch, or you have uploaded code that crashes and doesn't auto-reboot into the bootloader, you may have to manually enter the bootloader.

To enter the bootloader, hold down the **BOOT button**, and while continuing to hold it (don't let go!), press and release the **reset button**. **Continue to hold the BOOT button until the RPI-RP2 drive appears!**

Once the RPI-RP2 drive shows up, your board is in bootloader mode. There will not be a port available in bootloader mode, this is expected.

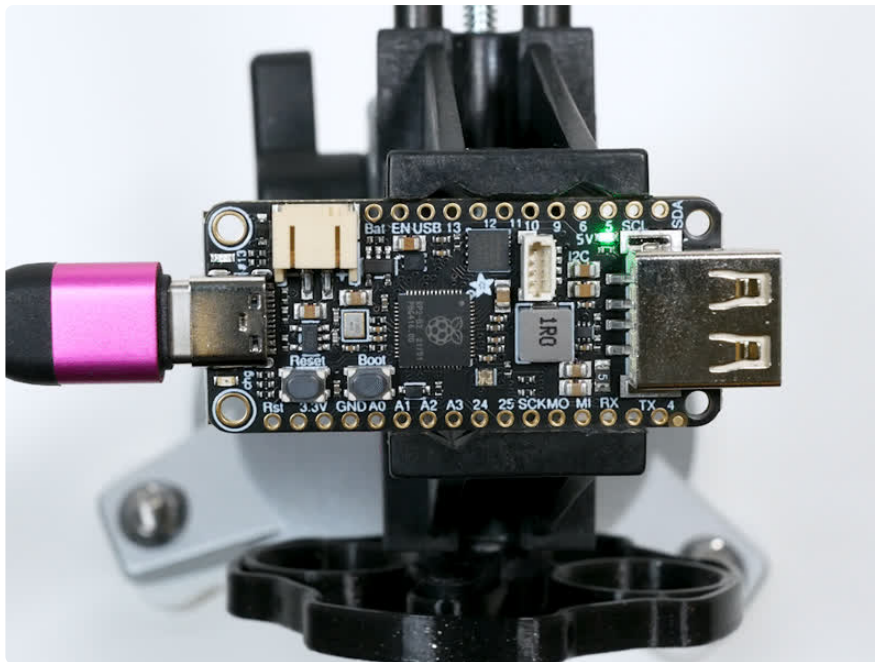
Once you see RPI-RP2 drive, make sure you are no longer holding down any buttons (reset or boot0 button).

Now, click Upload on your sketch to try again.

Blink

The first and most basic program you can upload to your Arduino is the classic Blink sketch. This takes something on the board and makes it, well, blink! On and off. It's a great way to make sure everything is working and you're uploading your sketch to the right board and right configuration.

When all else fails, you can always come back to Blink!



Pre-Flight Check: Get Arduino IDE & Hardware Set Up

This lesson assumes you have Arduino IDE set up. This is a generalized checklist, some elements may not apply to your hardware. If you haven't yet, check the previous steps in the guide to make sure you:

- **Install the very latest Arduino IDE for Desktop** (not all boards are supported by the Web IDE so we don't recommend it).
- **Install any board support packages (BSP) required for your hardware.** Some boards are built in defaults on the IDE, but lots are not! You may need to install plug-in support which is called the BSP.
- **Get a Data/Sync USB cable for connecting your hardware.** A significant amount of problems folks have stem from not having a USB cable with data pins. Yes, these cursed cables roam the land, making your life hard. If you find a USB cable that doesn't work for data/sync, throw it away immediately! There is no need to keep it around, cables are very inexpensive these days.
- **Install any drivers required** - If you have a board with a FTDI or CP210x chip, you may need to get separate drivers. If your board has native USB, it probably doesn't need anything. After installing, reboot to make sure the driver sinks in.
- **Connect the board to your computer.** If your board has a power LED, make sure its lit. Is there a power switch? Make sure its turned On!

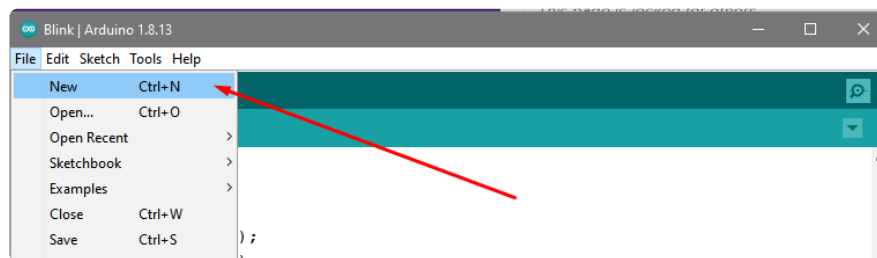
Start up Arduino IDE and Select Board/Port

OK now you are prepared! Open the Arduino IDE on your computer. Now you have to tell the IDE what board you are using, and how you want to connect to it.

In the IDE find the **Tools** menu. You will use this to select the board. If you switch boards, you must switch the selection! So always double-check before you upload code in a new session.

New Blink Sketch

OK lets make a new blink sketch! From the **File** menu, select **New**



Then in the new window, copy and paste this text:

```
int led = LED_BUILTIN;

void setup() {
  // Some boards work best if we also make a serial connection
  Serial.begin(115200);

  // set LED to be an output pin
  pinMode(led, OUTPUT);
}

void loop() {
  // Say hi!
  Serial.println("Hello!");

  digitalWrite(led, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(500);              // wait for a half second
  digitalWrite(led, LOW);  // turn the LED off by making the voltage LOW
  delay(500);              // wait for a half second
}
```

Note that in this example, we are not only blinking the LED but also printing to the Serial monitor, think of it as a little bonus to test the serial connection.

One note you'll see is that we reference the LED with the constant `LED_BUILTIN` rather than a number. That's because, historically, the built in LED was on pin 13 for Arduinos. But in the decades since, boards don't always have a pin 13, or maybe it

could not be used for an LED. So the LED could have moved to another pin. It's best to use **LED_BUILTIN** so you don't get the pin number confused!

On this Feather, the built in LED is on **pin 13**.

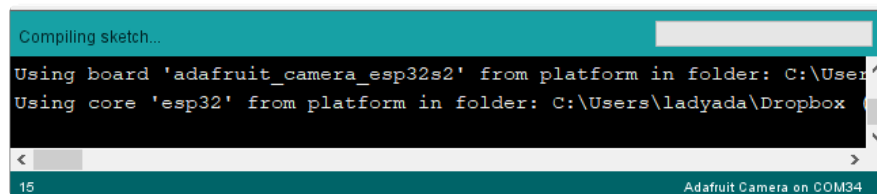
Verify (Compile) Sketch

OK now you can click the Verify button to convert the sketch into binary data to be uploaded to the board.

Note that Verifying a sketch is the same as Compiling a sketch - so we will use the words interchangeably



During verification/compilation, the computer will do a bunch of work to collect all the libraries and code and the results will appear in the bottom window of the IDE.



If something went wrong with compilation, you will get red warning/error text in the bottom window letting you know what the error was. It will also highlight the line with an error.

For example, here I had the wrong board selected - and the selected board does not have a built in LED!

```
File Edit Sketch Tools Help
sketch_dec25a$
int led = LED_BUILTIN;

void setup() {
  // Some boards work best if we also make a serial connection
  Serial.begin(115200);

  // set LED to be an output pin
  LED_BUILTIN;
}

exit status 1
'LED_BUILTIN' was not declared in this scope
15 Adafruit QT Py ESP32-S2 on COM34
```

Here's another common error, in my haste I forgot to add a `;` at the end of a line. The compiler warns me that it's looking for one - note that the error is actually a few lines up!

```
sketch_dec25a$
int led = LED_BUILTIN;

void setup() {
  // Some boards work best if we also make a serial connection
  Serial.begin(115200);

  // set LED to be an output pin
}

exit status 1
expected ',' or ';' before 'void'
3 Adafruit Feather ESP32-S2 on COM34
```

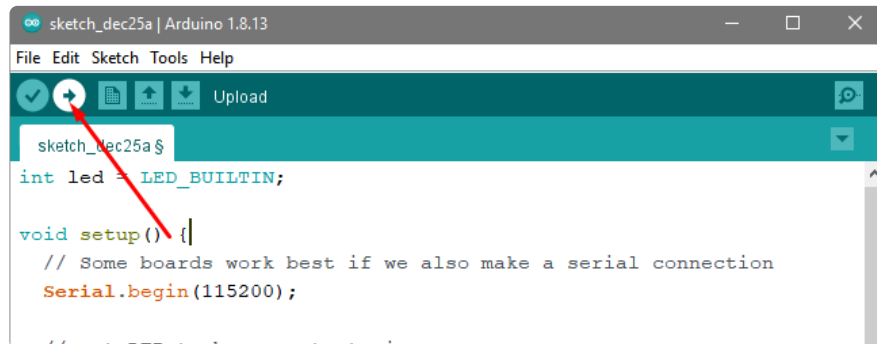
Turning on detailed compilation warnings and output can be very helpful sometimes - Its in Preferences under "Show Verbose Output During:" and check the Compilation button. If you ever need to get help from others, be sure to do this and then provide all the text that is output. It can assist in nailing down what happened!

On success you will see something like this white text output and the message **Done compiling.** in the message area.

```
Done compiling.
Sketch uses 219874 bytes (16%) of program storage space. Maximum is 1310720 bytes.
Global variables use 17956 bytes (5%) of dynamic memory, leaving 309724 bytes for local
6 Adafruit Feather ESP32-S2 on COM34
```

Upload Sketch

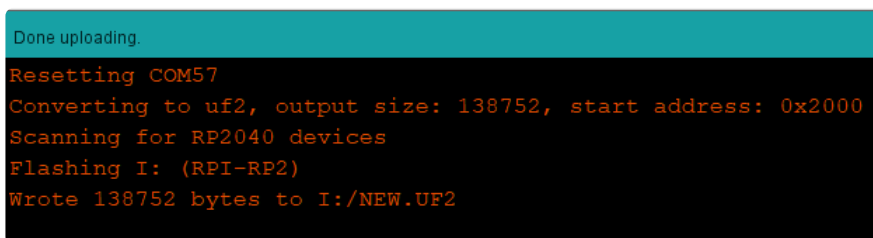
Once the code is verified/compiling cleanly you can upload it to your board. Click the **Upload** button.



The IDE will try to compile the sketch again for good measure, then it will try to connect to the board and upload a the file.

This is actually one of the hardest parts for beginners because it's where a lot of things can go wrong.

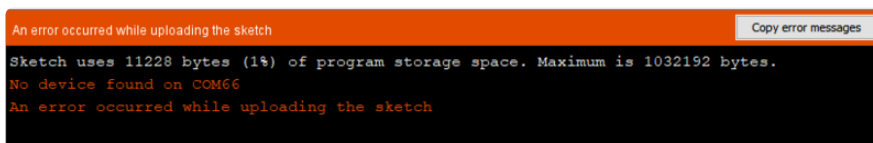
However, lets start with what it looks like on success! Here's what your board upload process looks like when it goes right:



Often times you will get a warning like this, which is kind of vague:

No device found on COM66 (or whatever port is selected)

An error occurred while uploading the sketch



This could be a few things.

First up, check again that you have the correct board selected! Many electronics boards have very similar names or look, and often times folks grab a board different from what they thought.

If you're positive the right board is selected, we recommend the next step is to put the board into manual bootloading mode.

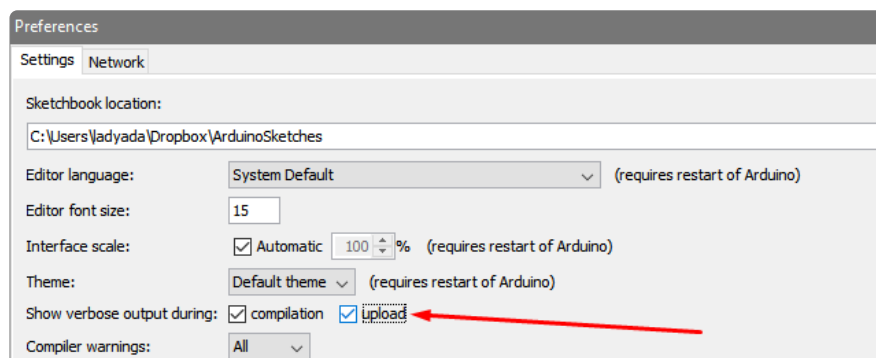
Native USB and manual bootloading

Historically, microcontroller boards contained two chips: the main micro chip (say, ATmega328 or ESP8266 or ESP32) and a separate chip for USB interface that would be used for bootloading (a CH430, FT232, CP210x, etc). With these older designs, the microcontroller is put into a bootloading state for uploading code by the separate chip. It allows for easier uploading but is more expensive as two chips are needed, and also the microcontroller can't act like a keyboard or disk drive.

Modern chips often have 'native' USB - that means that there is no separate chip for USB interface. It's all in one! Great for cost savings, simplicity of design, reduced size and more control. However, it means the chip must be self-aware enough to be able to put itself into bootload/upload mode on its own. That's fine 99% of the time but is very likely you will at some point get the board into an odd state that makes it too confused to bootload.

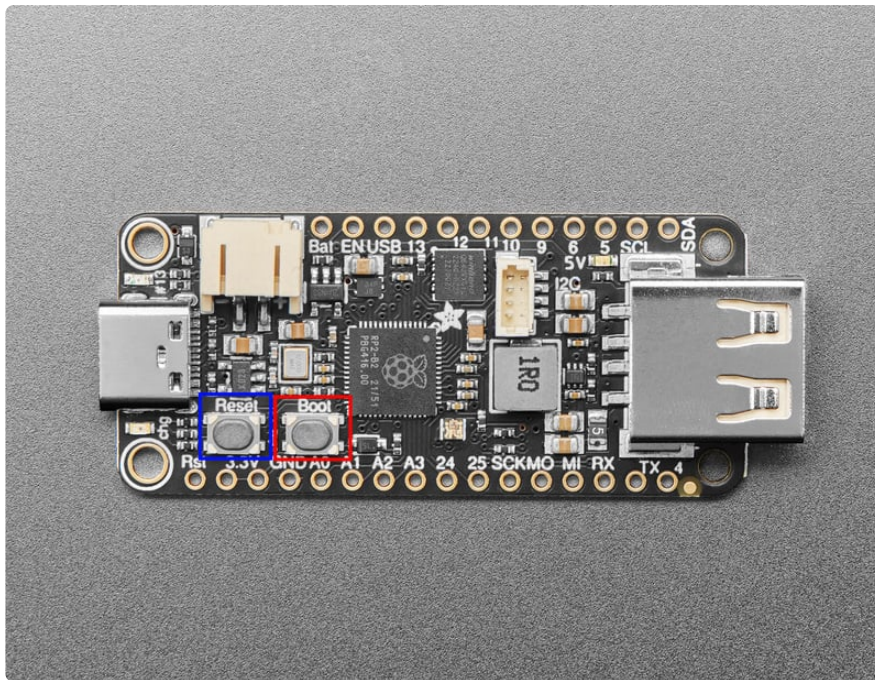
A lot of beginners have a little freakout the first time this happens, they think the board is ruined or 'bricked' - it's almost certainly not, it is just crashed and/or confused. You may need to perform a little trick to get the board back into a good state, at which point you won't need to manually bootload again.

Before continuing we really, really suggest turning on **Verbose Upload** messages, it will help in this process because you will be able to see what the IDE is trying to do. It's a checkbox in the **Preferences** menu.



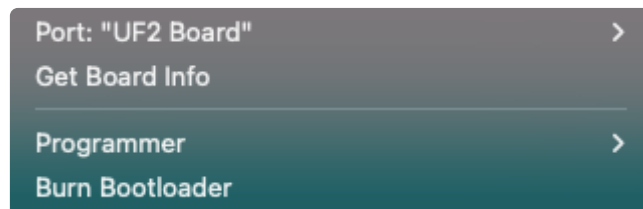
Enter Manual Bootload Mode

OK now you know it's probably time to try manual bootloading. No problem! Here is how you do that for this board:



To enter the bootloader on this Feather, hold down the **Boot button** (highlighted in red above), then press the **Reset button** (highlighted in blue above). **Continue holding the Boot button until the RPI-RP2 drive appears!** Then release the Boot button. The board is now in the bootloader!

Once you are in manual bootload mode, go to the Tools menu, and make sure you have selected the bootloader serial port. **It is almost certain that the serial port has changed now that the bootloader is enabled**



Now you can try uploading again!



Did you remember to select the new Port in the Tools menu since the bootloader port has changed?

This time, you should have success!

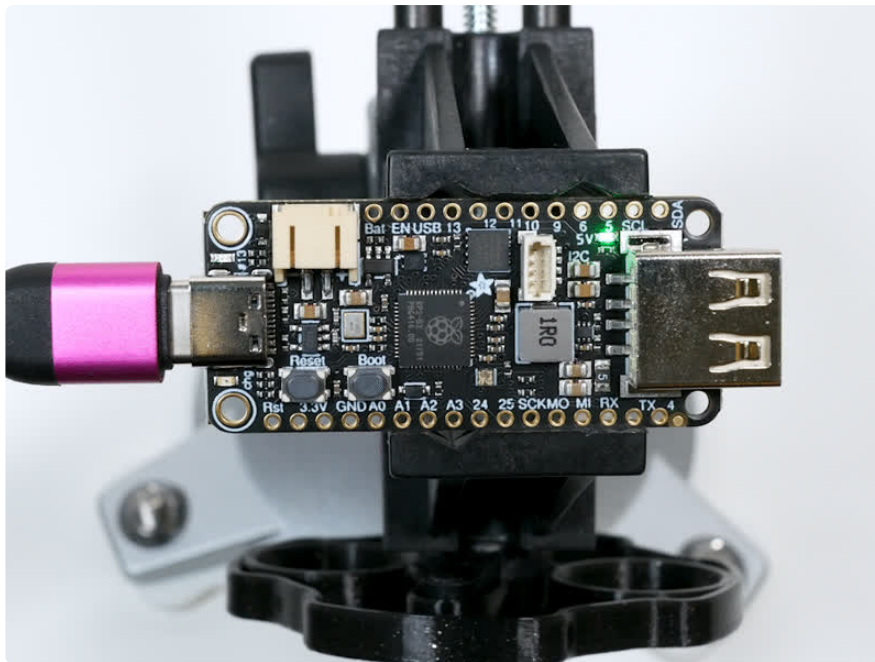
After uploading this way, be sure to **click the reset button** - it sort of makes sure that the board got a good reset and will come back to life nicely.

After uploading with Manual Bootloader - don't forget to re-select the old Port again

It's also a good idea to try to re-upload the sketch again now that you've performed a manual bootload to get the chip into a good state. It should perform an auto-reset the second time, so you don't have to manually bootload again.

Finally, a Blink!

OK it was a journey but now we're here and you can enjoy your blinking LED. Next up, try to change the delay between blinks and re-upload. It's a good way to make sure your upload process is smooth and practiced.



I2C Scan Test

A lot of sensors, displays, and devices can connect over I2C. I2C is a 2-wire 'bus' that allows multiple devices to all connect on one set of pins so it's very convenient for wiring!

When using your board, you'll probably want to connect up I2C devices, and it can be a little tricky the first time. The best way to debug I2C is go through a checklist and then perform an I2C scan

Common I2C Connectivity Issues

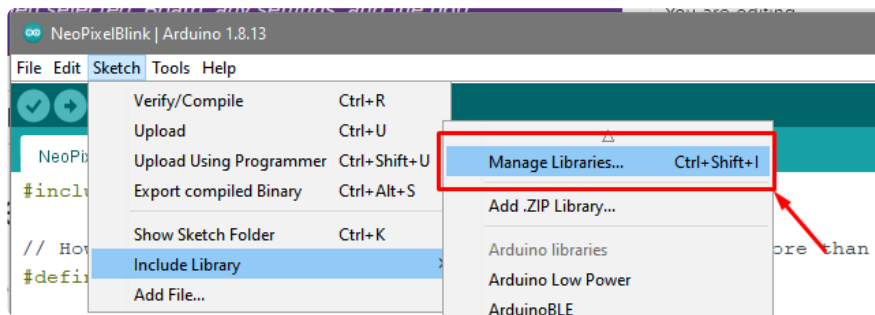
- **Have you connected four wires (at a minimum) for each I2C device?** Power the device with whatever is the logic level of your microcontroller board (probably 3.3V), then a ground wire, and a SCL clock wire, and and a SDA data wire.
- **If you're using a STEMMA QT board - check if the power LED is lit.** It's usually a green LED to the left side of the board.
- **Does the STEMMA QT/I2C port have switchable power or pullups?** To reduce power, some boards have the ability to cut power to I2C devices or the pullup resistors. Check the documentation if you have to do something special to turn on the power or pullups.
- **If you are using a DIY I2C device, do you have pullup resistors?** Many boards do not have pullup resistors built in and they are required! We suggest any common 2.2K to 10K resistors. You'll need two: one each connects from SDA to positive power, and SCL to positive power. Again, positive power (a.k.a VCC, VDD or V+) is often 3.3V
- **Do you have an address collision?** You can only have one board per address. So you cannot, say, connect two AHT20's to one I2C port because they have the same address and will interfere. Check the sensor or documentation for the address. Sometimes there are ways to adjust the address.
- **Does your board have multiple I2C ports?** Historically, boards only came with one. But nowadays you can have two or even three! This can help solve the "hey, but what if I want two devices with the same address" problem: just put one on each bus.
- **Are you hot-plugging devices?** I2C does not support dynamic re-connection, you cannot connect and disconnect sensors as you please. They should all be connected on boot and not change. ([Only exception is if you're using a hot-plug assistant but that'll cost you \(http://adafru.it/5159\)](http://adafru.it/5159)).
- **Are you keeping the total bus length reasonable?** I2C was designed for maybe 6" max length. We like to push that with plug-n-play cables, but really please keep them as short as possible! ([Only exception is if you're using an active bus extender \(http://adafru.it/4756\)](http://adafru.it/4756)).

Perform an I2C scan!

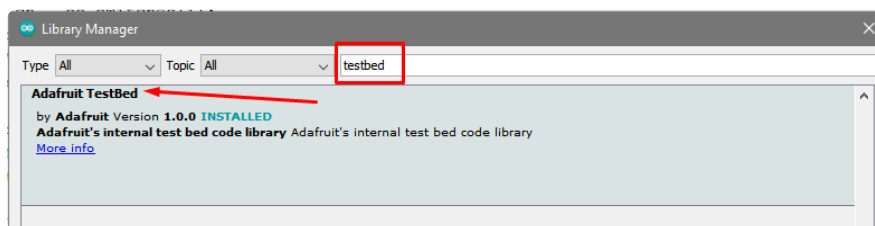
Install TestBed Library

To scan I2C, the Adafruit TestBed library is used. This library and example just makes the scan a little easier to run because it takes care of some of the basics. You will

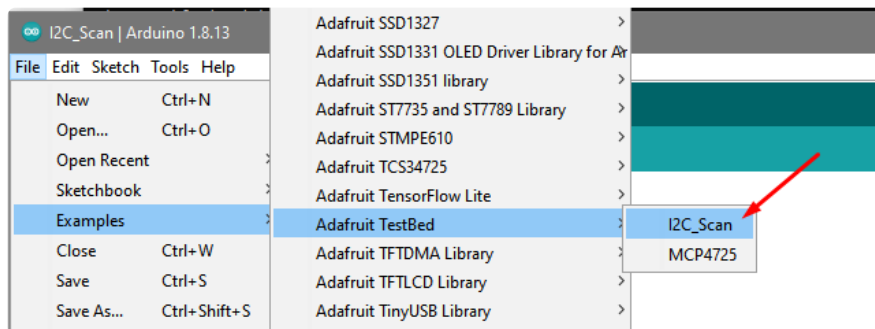
need to add support by installing the library. Good news: it is very easy to do it. Go to the **Arduino Library Manager**.



Search for **TestBed** and install the **Adafruit TestBed** library



Now open up the I2C Scan example



```
// SPDX-FileCopyrightText: 2023 Carter Nelson for Adafruit Industries
//
// SPDX-License-Identifier: MIT
// -----
// i2c_scanner
//
// Modified from https://playground.arduino.cc/Main/I2cScanner/
// -----

#include <Wire.h>

// Set I2C bus to use: Wire, Wire1, etc.
#define WIRE Wire

void setup() {
  WIRE.begin();

  Serial.begin(9600);
  while (!Serial)
    delay(10);
  Serial.println("\nI2C Scanner");
}
```

```

void loop() {
  byte error, address;
  int nDevices;

  Serial.println("Scanning...");

  nDevices = 0;
  for(address = 1; address < 127; address++ )
  {
    // The i2c_scanner uses the return value of
    // the Write.endTransmission to see if
    // a device did acknowledge to the address.
    WIRE.beginTransmission(address);
    error = WIRE.endTransmission();

    if (error == 0)
    {
      Serial.print("I2C device found at address 0x");
      if (address<16)
        Serial.print("0");
      Serial.print(address,HEX);
      Serial.println("  !");

      nDevices++;
    }
    else if (error==4)
    {
      Serial.print("Unknown error at address 0x");
      if (address<16)
        Serial.print("0");
      Serial.println(address,HEX);
    }
  }
  if (nDevices == 0)
    Serial.println("No I2C devices found\n");
  else
    Serial.println("done\n");

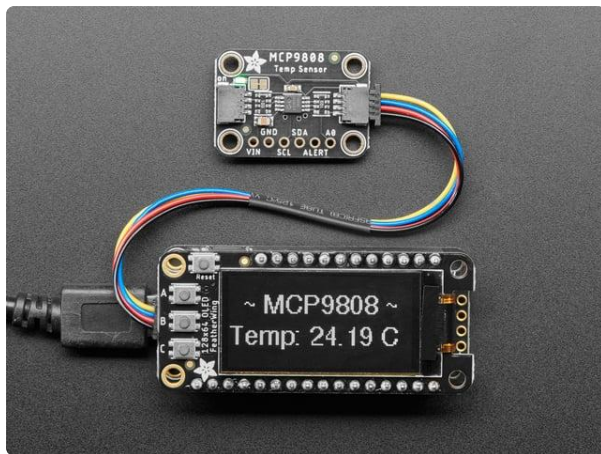
  delay(5000);          // wait 5 seconds for next scan
}

```

Wire up I2C device

While the examples here will be using the [Adafruit MCP9808](http://adafru.it/5027) (<http://adafru.it/5027>), a high accuracy temperature sensor, the overall process is the same for just about any I2C sensor or device.

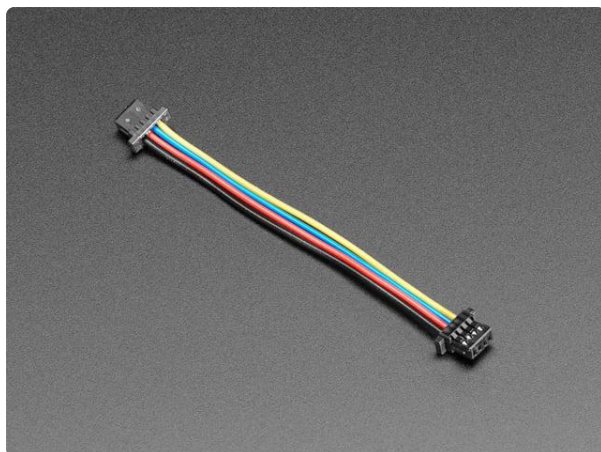
The first thing you'll want to do is get the sensor connected so your board has I2C to talk to.



Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of $\pm 0.25^{\circ}\text{C}$ over the sensor's -40°C to...

<https://www.adafruit.com/product/5027>



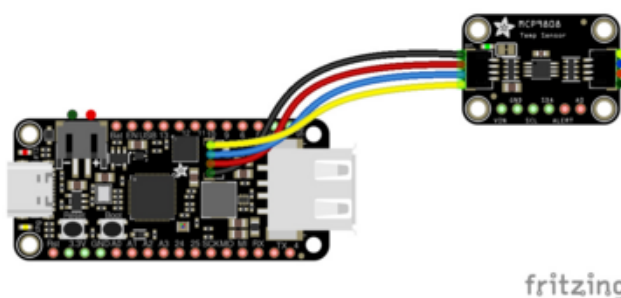
STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

<https://www.adafruit.com/product/4399>

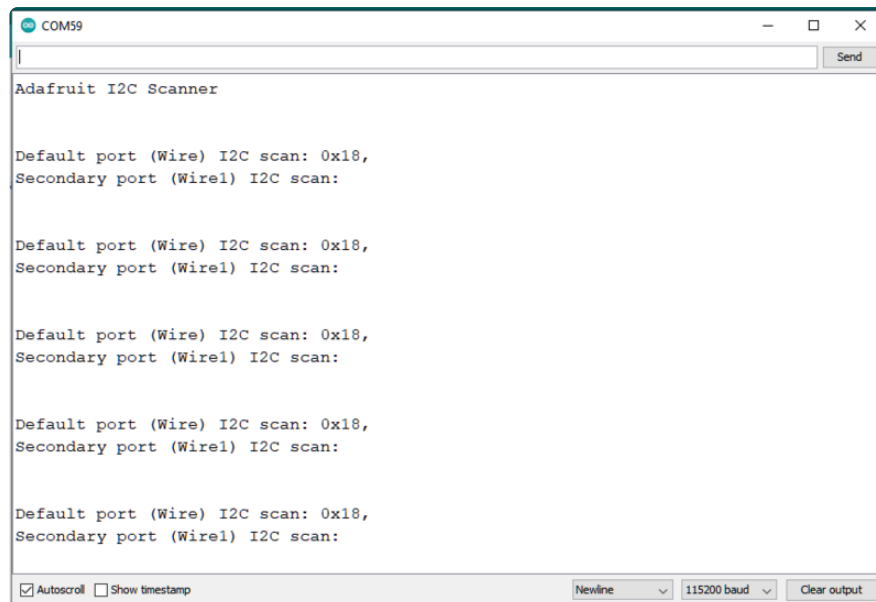
Wiring the MCP9808

The MCP9808 comes with a STEMMA QT connector, which makes wiring it up quite simple and solder-free.

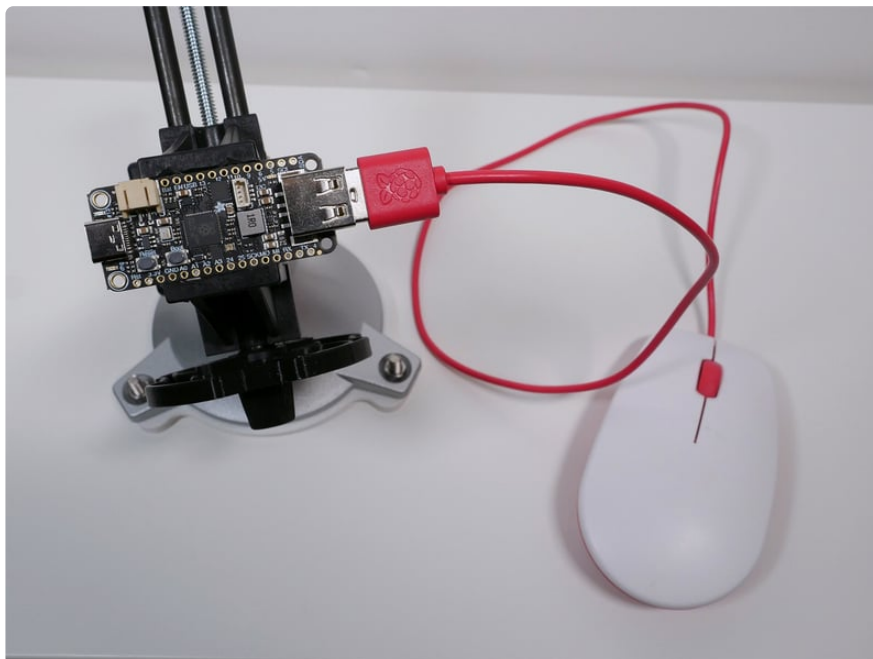


Plug a **STEMMA QT** cable from the **STEMMA QT** port on the Feather to the **STEMMA QT** port on the sensor.

Now upload the scanning sketch to your microcontroller and open the serial port to see the output. You should see something like this:



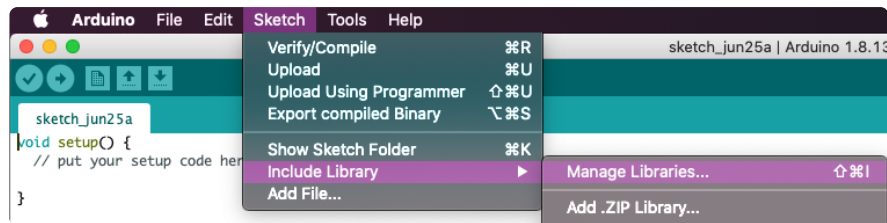
USB Host Device Info



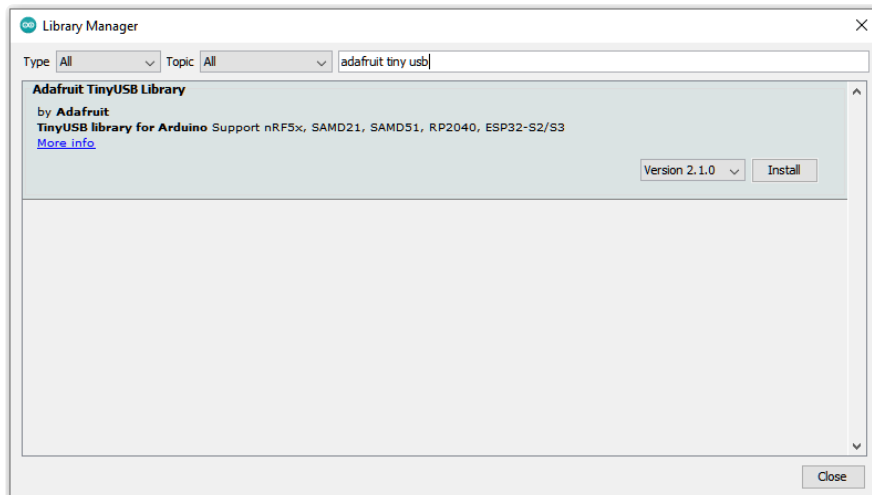
The main use case for this Feather is to act as a USB host. There are quite a few examples in the [Adafruit TinyUSB Library \(https://adafru.it/18Do\)](https://adafru.it/18Do) to get you started. In the example below, you'll run the [device info test \(https://adafru.it/19B5\)](https://adafru.it/19B5) to read the info from your attached USB device. You will need to have a USB device, such as a mouse or keyboard, to plug into the USB type A port on the Feather.

Install Adafruit_TinyUSB Library

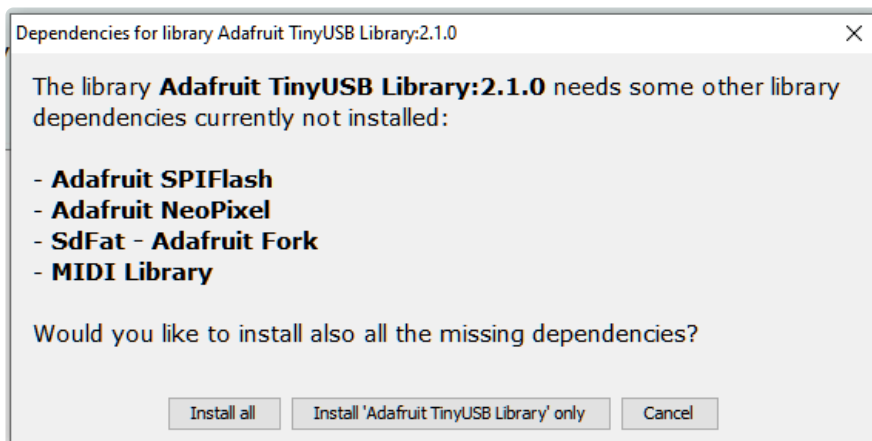
To use your Feather as a USB host, you'll need to [install the Adafruit TinyUSB library \(https://adafru.it/EWc\)](https://adafru.it/EWc). It can be installed using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit TinyUSB**, and select the **Adafruit TinyUSB Library**:



If asked about dependencies, click "Install all".

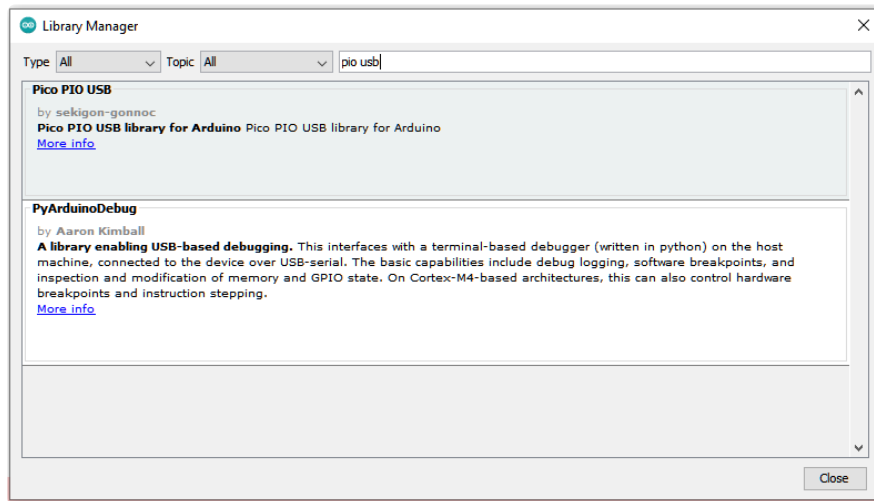


If the "Dependencies" window does not come up, then you already have the dependencies installed.

If the dependencies are already installed, you must make sure you update them through the Arduino Library Manager before loading the example!

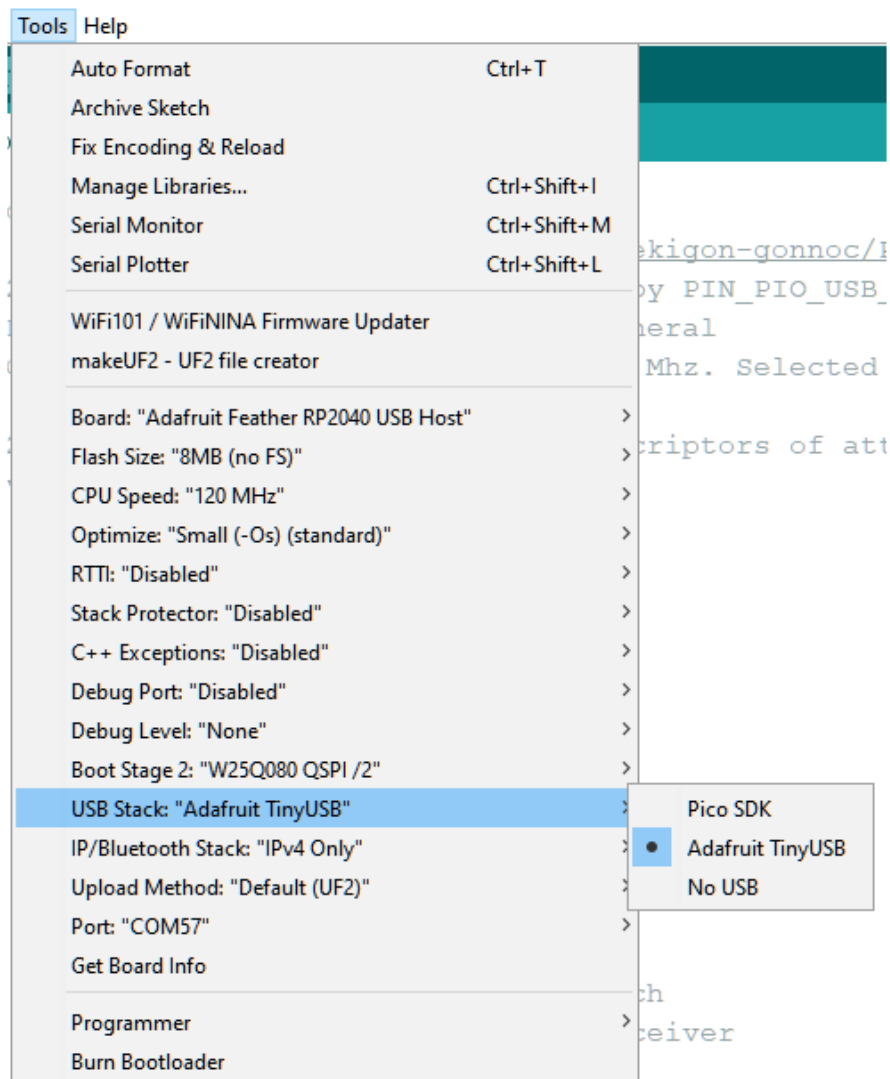
Install Pico PIO USB Library

Additionally, you'll need to install the Pico PIO USB library. Click the **Manage Libraries** ... menu item again, search for **PIO USB**, and select the **Pico PIO USB** library:



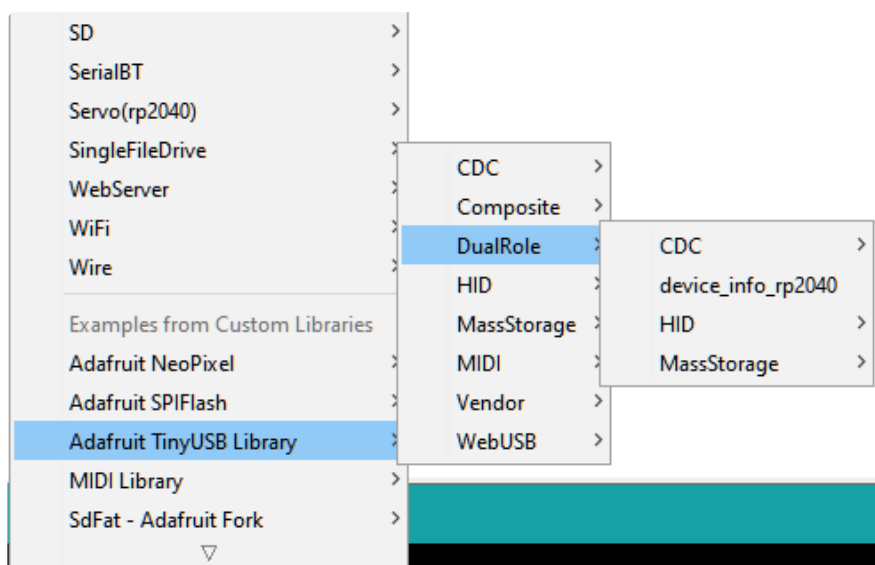
Board Upload Settings

In the **Tools** menu, select **Adafruit Feather RP2040 USB Host** under **Board**. For **CPU Speed**, you'll need to select **120 MHz** or **240 MHz**. Finally, under **USB Stack**, select **Adafruit TinyUSB**.



Device Info Example

You can open the device info example in the Arduino IDE by navigating to **File -> Examples -> Adafruit TinyUSB Library -> DualRole -> Simple** and select **device_info**.



```

/*****
Adafruit invests time and resources providing this open source code,
please support Adafruit and open-source hardware by purchasing
products from Adafruit!

MIT license, check LICENSE for more information
Copyright (c) 2019 Ha Thach for Adafruit Industries
All text above, and the splash screen below must be included in
any redistribution
*****/

/* This example demonstrates use of both device and host, where
 * - Device run on native usb controller (roothub port0)
 * - Host depending on MCUs run on either:
 *   - rp2040: bit-banging 2 GPIOs with the help of Pico-PIO-USB library (roothub
port1)
 *   - samd21/51, nrf52840, esp32: using MAX3421e controller (host shield)
 *
 * Requirements:
 * - For rp2040:
 *   - [Pico-PIO-USB](https://github.com/sekigon-gonnoc/Pico-PIO-USB) library
 *   - 2 consecutive GPIOs: D+ is defined by PIN_USB_HOST_DP, D- = D+ +1
 *   - Provide VBus (5v) and GND for peripheral
 *   - CPU Speed must be either 120 or 240 Mhz. Selected via "Menu -> CPU Speed"
 * - For samd21/51, nrf52840, esp32:
 *   - Additional MAX2341e USB Host shield or featherwing is required
 *   - SPI instance, CS pin, INT pin are correctly configured in usbh_helper.h
 */

/* Host example will get device descriptors of attached devices and print it out via
 * device cdc (Serial) as follows:
 *   Device 1: ID 046d:c52f
 *   Device Descriptor:
 *     bLength                18
 *     bDescriptorType         1
 *     bcdUSB                  0200
 *     bDeviceClass             0
 *     bDeviceSubClass          0
 *     bDeviceProtocol          0
 *     bMaxPacketSize0          8
 *     idVendor                 0x046d
 *     idProduct               0xc52f
 *     bcdDevice                2200
 *     iManufacturer           1      Logitech
 *     iProduct                2      USB Receiver
 *     iSerialNumber            0
 *     bNumConfigurations      1
 *
 */

// USBHost is defined in usbh_helper.h
#include "usbh_helper.h"

// Language ID: English
#define LANGUAGE_ID 0x0409

typedef struct {
  tusb_desc_device_t desc_device;
  uint16_t manufacturer[32];
  uint16_t product[48];
  uint16_t serial[16];
  bool mounted;
} dev_info_t;

// CFG_TUH_DEVICE_MAX is defined by tusb_config header
dev_info_t dev_info[CFG_TUH_DEVICE_MAX] = { 0 };

```

```

void setup() {
    Serial.begin(115200);

    #if defined(CFG_TUH_MAX3421) && CFG_TUH_MAX3421
        // init host stack on controller (rhport) 1
        // For rp2040: this is called in core1's setup1()
        USBHost.begin(1);
    #endif

    // while ( !Serial ) delay(10); // wait for native usb
    Serial.println("TinyUSB Dual Device Info Example");
}

#if defined(CFG_TUH_MAX3421) && CFG_TUH_MAX3421
//-----+
// Using Host shield MAX3421E controller
//-----+
void loop() {
    USBHost.task();
    Serial.flush();
}

#elif defined(ARDUINO_ARCH_RP2040)
//-----+
// For RP2040 use both core0 for device stack, core1 for host stack
//-----+

//----- Core0 -----//
void loop() {
}

//----- Core1 -----//
void setup1() {
    //while ( !Serial ) delay(10); // wait for native usb
    // configure pio-usb: defined in usbh_helper.h
    rp2040_configure_pio_usb();

    // run host stack on controller (rhport) 1
    // Note: For rp2040 pico-pio-usb, calling USBHost.begin() on core1 will have most
of the
    // host bit-banging processing works done in core1 to free up core0 for other
works
    USBHost.begin(1);
}

void loop1() {
    USBHost.task();
    Serial.flush();
}
#endif

//-----+
// TinyUSB Host callbacks
//-----+
void print_device_descriptor(tuh_xfer_t *xfer);

void utf16_to_utf8(uint16_t *temp_buf, size_t buf_len);

void print_lsusb(void) {
    bool no_device = true;
    for (uint8_t daddr = 1; daddr < CFG_TUH_DEVICE_MAX + 1; daddr++) {
        // TODO can use tuh_mounted(daddr), but tinyusb has an bug
        // use local connected flag instead
        dev_info_t *dev = &dev_info[daddr - 1];
        if (dev->mounted) {
            Serial.printf("Device %u: ID %04x:%04x %s %s\r\n", daddr,
                dev->desc_device.idVendor, dev->desc_device.idProduct,
                (char *) dev->manufacturer, (char *) dev->product);
        }
    }
}

```

```

        no_device = false;
    }
}

if (no_device) {
    Serial.println("No device connected (except hub)");
}
}

// Invoked when device is mounted (configured)
void tuh_mount_cb(uint8_t daddr) {
    Serial.printf("Device attached, address = %d\r\n", daddr);

    dev_info_t *dev = &dev_info[daddr - 1];
    dev->mounted = true;

    // Get Device Descriptor
    tuh_descriptor_get_device(daddr, &dev->desc_device, 18, print_device_descriptor,
0);
}

/// Invoked when device is unmounted (bus reset/unplugged)
void tuh_umount_cb(uint8_t daddr) {
    Serial.printf("Device removed, address = %d\r\n", daddr);
    dev_info_t *dev = &dev_info[daddr - 1];
    dev->mounted = false;

    // print device summary
    print_lsusb();
}

void print_device_descriptor(tuh_xfer_t *xfer) {
    if (XFER_RESULT_SUCCESS != xfer->result) {
        Serial.printf("Failed to get device descriptor\r\n");
        return;
    }

    uint8_t const daddr = xfer->daddr;
    dev_info_t *dev = &dev_info[daddr - 1];
    tusb_desc_device_t *desc = &dev->desc_device;

    Serial.printf("Device %u: ID %04x:%04x\r\n", daddr, desc->idVendor, desc-
>idProduct);
    Serial.printf("Device Descriptor:\r\n");
    Serial.printf("  bLength          %u\r\n", desc->bLength);
    Serial.printf("  bDescriptorType    %u\r\n", desc->bDescriptorType);
    Serial.printf("  bcdUSB             %04x\r\n", desc->bcdUSB);
    Serial.printf("  bDeviceClass       %u\r\n", desc->bDeviceClass);
    Serial.printf("  bDeviceSubClass    %u\r\n", desc->bDeviceSubClass);
    Serial.printf("  bDeviceProtocol    %u\r\n", desc->bDeviceProtocol);
    Serial.printf("  bMaxPacketSize0    %u\r\n", desc->bMaxPacketSize0);
    Serial.printf("  idVendor           0x%04x\r\n", desc->idVendor);
    Serial.printf("  idProduct          0x%04x\r\n", desc->idProduct);
    Serial.printf("  bcdDevice          %04x\r\n", desc->bcdDevice);

    // Get String descriptor using Sync API
    Serial.printf("  iManufacturer      %u", desc->iManufacturer);
    if (XFER_RESULT_SUCCESS ==
        tuh_descriptor_get_manufacturer_string_sync(daddr, LANGUAGE_ID, dev-
>manufacturer, sizeof(dev->manufacturer))) {
        utf16_to_utf8(dev->manufacturer, sizeof(dev->manufacturer));
        Serial.printf((char *) dev->manufacturer);
    }
    Serial.printf("\r\n");

    Serial.printf("  iProduct           %u", desc->iProduct);
    if (XFER_RESULT_SUCCESS ==
        tuh_descriptor_get_product_string_sync(daddr, LANGUAGE_ID, dev->product,
sizeof(dev->product))) {

```

```

    utf16_to_utf8(dev->product, sizeof(dev->product));
    Serial.printf((char *) dev->product);
}
Serial.printf("\r\n");

Serial.printf("  iSerialNumber      %u      ", desc->iSerialNumber);
if (XFER_RESULT_SUCCESS ==
    tuh_descriptor_get_serial_string_sync(daddr, LANGUAGE_ID, dev->serial,
sizeof(dev->serial))) {
    utf16_to_utf8(dev->serial, sizeof(dev->serial));
    Serial.printf((char *) dev->serial);
}
Serial.printf("\r\n");

Serial.printf("  bNumConfigurations  %u\r\n", desc->bNumConfigurations);

// print device summary
print_lsusb();
}

//-----+
// String Descriptor Helper
//-----+

static void _convert_utf16le_to_utf8(const uint16_t *utf16, size_t utf16_len,
uint8_t *utf8, size_t utf8_len) {
    // TODO: Check for runover.
    (void) utf8_len;
    // Get the UTF-16 length out of the data itself.

    for (size_t i = 0; i < utf16_len; i++) {
        uint16_t chr = utf16[i];
        if (chr < 0x80) {
            *utf8++ = chr & 0xff;
        } else if (chr < 0x800) {
            *utf8++ = (uint8_t) (0xC0 | (chr >> 6 & 0x1F));
            *utf8++ = (uint8_t) (0x80 | (chr >> 0 & 0x3F));
        } else {
            // TODO: Verify surrogate.
            *utf8++ = (uint8_t) (0xE0 | (chr >> 12 & 0x0F));
            *utf8++ = (uint8_t) (0x80 | (chr >> 6 & 0x3F));
            *utf8++ = (uint8_t) (0x80 | (chr >> 0 & 0x3F));
        }
        // TODO: Handle UTF-16 code points that take two entries.
    }
}

// Count how many bytes a utf-16-le encoded string will take in utf-8.
static int _count_utf8_bytes(const uint16_t *buf, size_t len) {
    size_t total_bytes = 0;
    for (size_t i = 0; i < len; i++) {
        uint16_t chr = buf[i];
        if (chr < 0x80) {
            total_bytes += 1;
        } else if (chr < 0x800) {
            total_bytes += 2;
        } else {
            total_bytes += 3;
        }
        // TODO: Handle UTF-16 code points that take two entries.
    }
    return total_bytes;
}

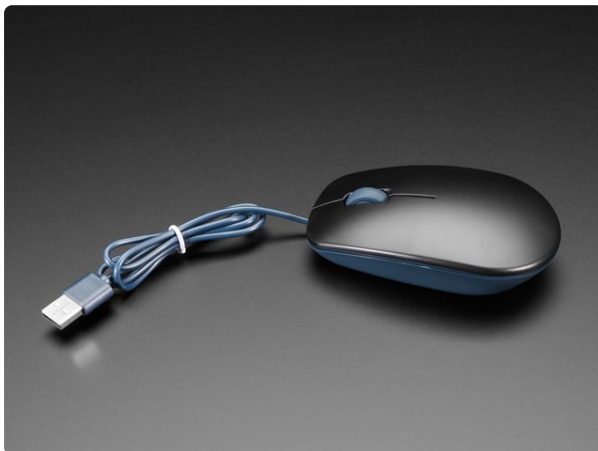
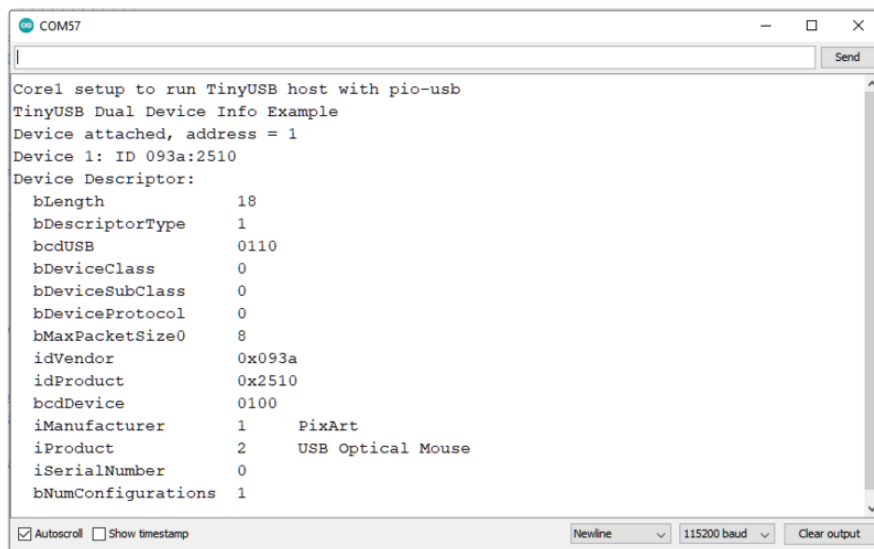
void utf16_to_utf8(uint16_t *temp_buf, size_t buf_len) {
    size_t utf16_len = ((temp_buf[0] & 0xff) - 2) / sizeof(uint16_t);
    size_t utf8_len = _count_utf8_bytes(temp_buf + 1, utf16_len);

    _convert_utf16le_to_utf8(temp_buf + 1, utf16_len, (uint8_t *) temp_buf, buf_len);
}

```

```
((uint8_t *) temp_buf)[utf8_len] = '\0';  
}
```

Upload the sketch to your board and plug in a USB device to the USB type A port on the Feather. Then, open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. You should see the starting text at the top of the window, followed by the attached USB device information. The device tested below was the Official Raspberry Pi USB mouse.

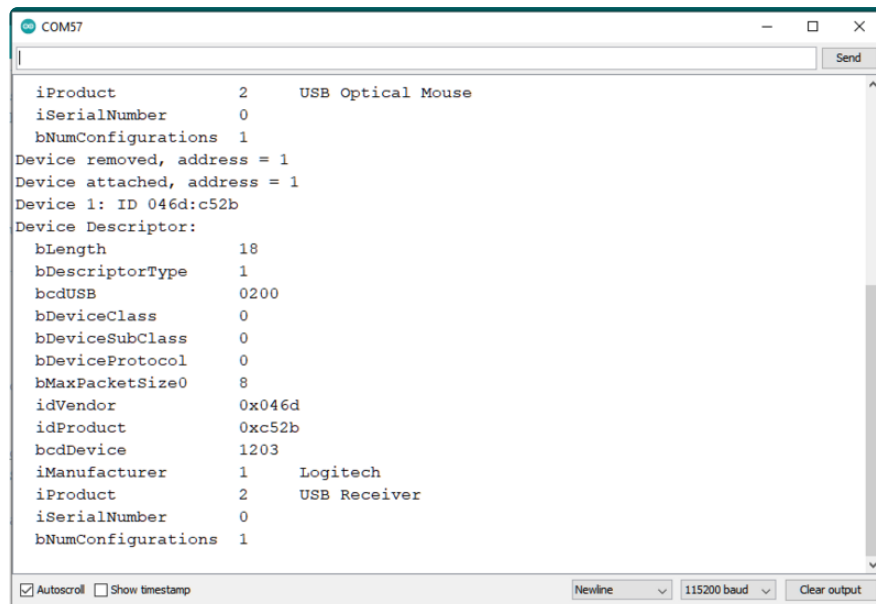


Official Raspberry Pi USB Optical Mouse - Black and Gray

This is a mouse. But, hark, it's no generic mouse. It's the Official Raspberry Pi Mouse! It's optical for good resolution and precision, three...

<https://www.adafruit.com/product/4113>

If you unplug the device from the USB type A port, "Device removed, address = #" will be printed to the Serial monitor. If you plug in another device, its info will appear in the monitor.



```
COM57
iProduct      2      USB Optical Mouse
iSerialNumber  0
bNumConfigurations 1
Device removed, address = 1
Device attached, address = 1
Device 1: ID 046d:c52b
Device Descriptor:
  bLength        18
  bDescriptorType 1
  bcdUSB          0200
  bDeviceClass    0
  bDeviceSubClass 0
  bDeviceProtocol 0
  bMaxPacketSize0 8
  idVendor        0x046d
  idProduct       0xc52b
  bcdDevice       1203
  iManufacturer   1      Logitech
  iProduct        2      USB Receiver
  iSerialNumber   0
  bNumConfigurations 1
```

Factory Reset

The Feather RP2040 USB Host ships running code that performs a USB device scan to quickly test the USB host functionality and get enumeration data in the Serial Monitor. It's lovely, but you probably had other plans for the board. As you start working with your board, you may want to return to the original code to begin again, or you may find your board gets into a bad state. Either way, this page has you covered.

Completing a factory reset will erase your board's firmware which is also used for storing CircuitPython/Arduino/Files! Be sure to back up your data first.

Step 1. Download the factory-reset.uf2 file

Save the following file wherever is convenient for you. You will need to access it to copy it to your board.

Click to download Adafruit Feather
RP2040 USB Host Factory
Reset.uf2

<https://adafru.it/18DL>

Step 2. Enter RP2040 bootloader mode

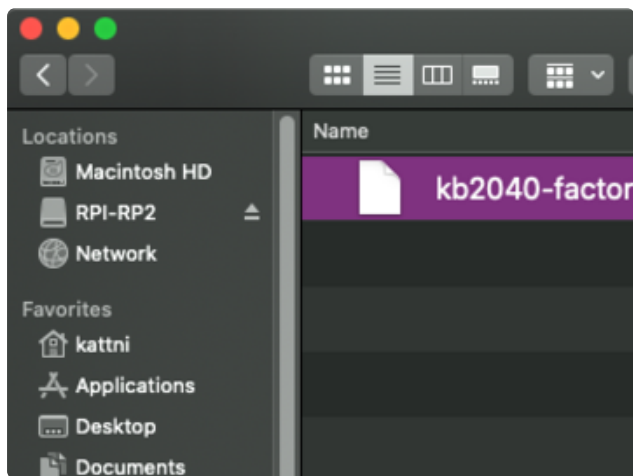
Entering the RP2040 bootloader is easy. Complete the following steps.

Before you start, make sure your microcontroller is plugged into USB port to your computer using a data/sync cable. Charge-only cables will not work!

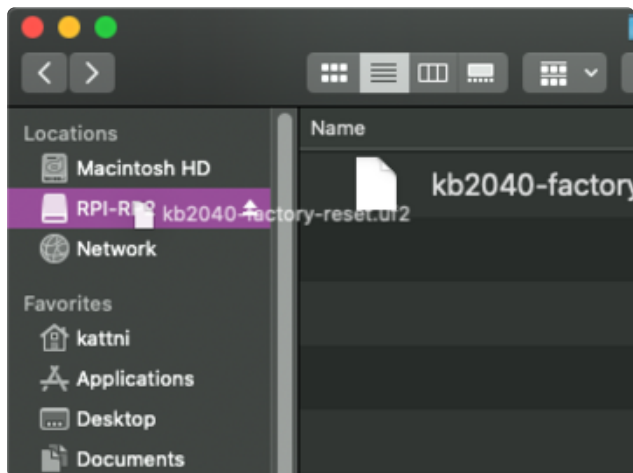
To enter the bootloader:

1. Press and hold the **Boot** button down. Don't let go of it yet!
2. Press and release the **Reset** button. You should still have the Boot button pressed while you do this.
3. Continue holding the **Boot** button until you see the **RPI-RP2** drive appear.
4. You can now release but Boot button.

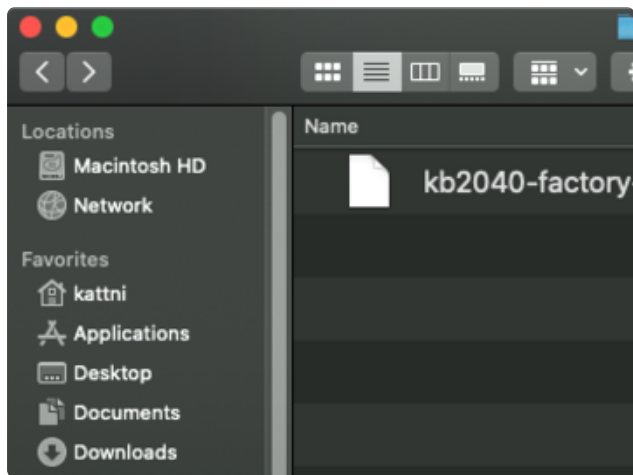
Step 3. Drag UF2 file to RPI-RP2



Navigate to the folder where you downloaded the **factory-reset.uf2** file from Step 1.



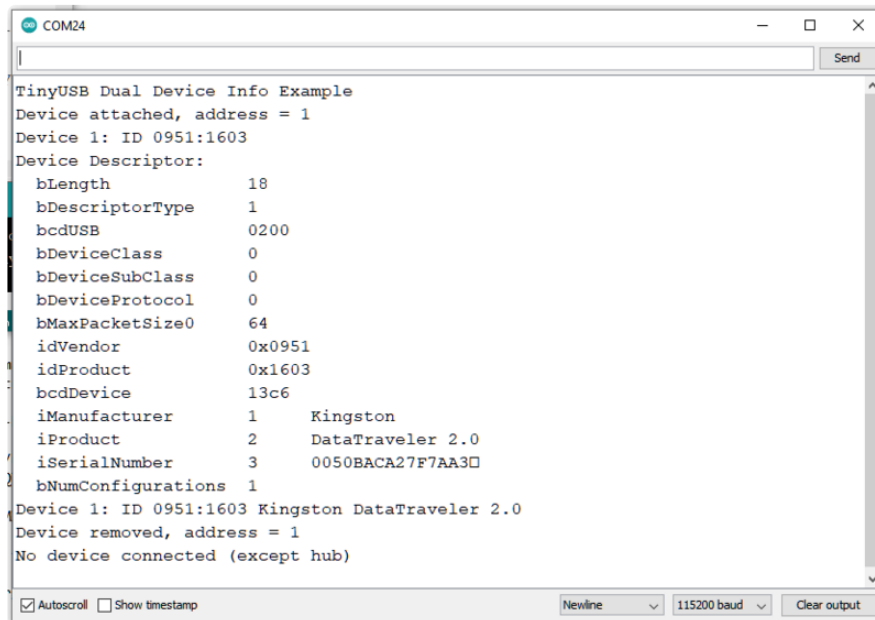
Drag the **factory-reset.uf2** file to the **RPI-RP2** drive.



The **RPI-RP2** drive will disappear.

The board will automatically reboot.

You can use it to quickly test the USB host functionality and get enumeration data in the Serial Monitor.



You've successfully returned your board to a factory reset state!

Flash Resetting UF2

If your board ever gets into a really weird state and doesn't even show up when loading code, try loading this 'nuke' UF2 which will do a 'deep clean' on your Flash Memory. **You will lose all the files on the board**, but at least you'll be able to revive it! Download the file below, and follow the instructions in Step 2 and Step 3 above to load this UF2. Then, start again at Step 1 to return your board to factory reset state.

[Download flash erasing "nuke" UF2](https://adafru.it/RLE)

<https://adafru.it/RLE>

Downloads

Files

- [RP2040 Datasheet \(https://adafru.it/QTf\)](https://adafru.it/QTf)
- [EagleCAD PCB Files on GitHub \(https://adafru.it/18DM\)](https://adafru.it/18DM)
- [Fritzing object in the Adafruit Fritzing Library \(https://adafru.it/18DN\)](https://adafru.it/18DN)
- [PrettyPins PDF on GitHub \(https://adafru.it/18Dt\)](https://adafru.it/18Dt)
- [PrettyPins SVG \(https://adafru.it/18DO\)](https://adafru.it/18DO)

Schematic and Fab Print

Dimensions are in inches.

